[Considerations for Updating a Multitenant Solution - Azure Architecture Center | Microsoft Learn]

Trong một giải pháp MT, cần phải làm rõ chính sách cập nhật bởi vì hoặc họ sẽ miễn cưỡng cho phép cập nhật, hoặc họ sẽ yêu cầu các điều kiện và chỉ được cập nhật khi các điều kiện đó được đáp ứng. Do đó, để lên một chiến lược cập nhật cần phải:

- Hiểu rõ yêu cầu khách hàng
- Hiểu rõ khả năng hệ thống có thể cung cấp
- Tìm sự cân bằng giữa cả hai
- Giao tiếp với khách hàng (thông báo cập nhật, biết rõ họ đang sử dụng phiên bản nào khi giao tiếp,...)

Các kỹ thuật cập nhật cho SaaS MT:

- **Deployment Stamps (Cập nhật theo cụm):** Từ một bản lớn triển khai nhiều bản copy của hệ thống (kiểu từ một bản chia ra thành nhiều bản nhỏ), mỗi bản có thể phục vụ một tệp tenant khác nhau (của mình có thể đó là Tiers). Từ đó, triển khai cập nhật dần dần trên từng bản copy, không ảnh hưởng đến các bản copy khác.
- Feature flags:
- Deployment rings:
- API versions:

Đo lường mức tiêu thụ theo tenant và cách áp dụng vào ứng dụng Bridge

Nguồn phân tích chính: Microsoft Azure Architecture Center — Measure the consumption of each tenant
Ngày đối chiếu: 2026-10-02
Phạm vi: ứng dụng quản lý công việc đa thuê bao Bridge với ba cách bố trí dữ liệu POOL, SCHEMA_PER_TENANT và SILO_DATABASE.

Tài liệu này đề xuất kiến trúc đo lường, chưa phải kết quả thực nghiệm và chưa phải bảng giá chính thức. Các con số trong ví dụ chỉ dùng để giải thích bản chất. Chỉ những chức năng được ghi rõ là **đã hiện thực** mới được xem là hành vi hiện tại của ứng dụng.

Một tenant/workspace là không gian làm việc của một nhóm hoặc tổ chức. Trong tenant, người dùng có thể tạo project, quản lý task trên bảng công việc, mời thành viên, dùng chung tài nguyên và nhận nhắc hạn qua nhiều kênh.

Đo lường mức tiêu thụ trả lời ba câu hỏi khác nhau:

> Tenant đang sử dụng sản phẩm như thế nào?
>
> Tenant đang tạo ra bao nhiêu tải và chi phí cho hệ thống?
>
> Tenant có đang vượt giới hạn gói hoặc cần thanh toán thêm hay không?

Ba câu hỏi này có liên quan nhưng không đồng nghĩa. Một metric phù hợp để vận hành chưa chắc đủ chính xác để tính tiền; một metric dễ tính tiền chưa chắc phản ánh đúng chi phí hạ tầng.

1. Vì sao phải đo mức tiêu thụ theo tenant?

Theo Microsoft, có hai động lực chính:

> Biết chi phí để phục vụ từng tenant, từ đó đánh giá khả năng duy trì và lợi nhuận.
>
> Xác định số tiền cần thu khi sản phẩm áp dụng giá dựa trên mức tiêu thụ.

Đối với Bridge, đo lường còn phục vụ thêm ba mục đích:

> Kiểm soát quota để một tenant không sử dụng quá mức hoặc ảnh hưởng tenant khác.
>
> Phát hiện noisy neighbor trong mô hình tài nguyên dùng chung.
>
> Cung cấp dữ liệu thực nghiệm để so sánh ba placement trên cùng workload.

Khó khăn nằm ở chỗ nhiều tenant dùng chung API, worker, PostgreSQL, MinIO và tài nguyên máy chủ. Hạ tầng thường chỉ cho biết tổng CPU, RAM hoặc dung lượng, không tự biết phần nào thuộc về tenant nào. Vì vậy Bridge không thể chỉ đọc hóa đơn máy chủ rồi chia đều cho số tenant.

2. Chỉ số đại diện cho mức tiêu thụ

Định nghĩa

Indicative metric là một chỉ số dễ đo, có quan hệ đủ gần với mức tiêu thụ thật và được dùng để đại diện cho nhiều loại chi phí phức tạp phía sau.

Ví dụ, một hệ thống quản lý tài liệu có thể dùng dung lượng tài liệu đang lưu làm chỉ số đại diện. Chỉ số này không đo chính xác toàn bộ CPU, truy vấn database và băng thông, nhưng dễ hiểu và thường có tương quan với quy mô tenant.

Nếu áp dụng vào ứng dụng

Các chỉ số đại diện phù hợp với Bridge gồm:

storage_active_bytes: tổng dung lượng file chưa bị xóa mềm.

active_members: số thành viên còn hiệu lực hoặc có hoạt động nghiệp vụ trong kỳ.

active_projects: số project còn hoạt động.

Ví dụ, gói Starter có giới hạn minh họa là 2 GB tài nguyên và 10 thành viên. Tenant đang dùng 1,6 GB và có 9 thành viên thì trang Usage hiển thị lần lượt 80% và 90% hạn mức.

Luồng nghiệp vụ của người dùng

Owner mở trang **Mức sử dụng**.

Hệ thống hiển thị dung lượng, thành viên và project đang dùng so với hạn mức.

Khi đạt ngưỡng cảnh báo, Owner nhận thông báo trong ứng dụng hoặc email.

Khi đạt hard limit, hệ thống chặn đúng thao tác làm tăng usage, ví dụ upload file mới; các thao tác đọc dữ liệu vẫn hoạt động.

Owner dọn tài nguyên, thu hồi thành viên hoặc yêu cầu thay đổi gói.

Bản chất thể hiện qua ví dụ

Chỉ số đại diện dễ hiểu và rẻ để triển khai, nhưng không thể hiện đầy đủ mọi tải hệ thống. Hai tenant cùng lưu 1 GB có thể tạo chi phí khác nhau nếu một tenant thường xuyên tải file và chạy automation, còn tenant kia gần như không hoạt động. Do đó cần kiểm tra định kỳ xem chỉ số đại diện còn tương quan với chi phí thực hay không.

3. Chỉ số giao dịch nghiệp vụ

Định nghĩa

Transaction metric đếm một hành động nghiệp vụ quan trọng, lặp lại và có thể ghi nhận tại một điểm rõ ràng trong ứng dụng.

Đơn vị không nhất thiết là giao dịch thanh toán. Trong Bridge, nó có thể là một lần hoàn tất upload, một automation được thực thi hoặc một thông báo được gửi thành công.

Nếu áp dụng vào ứng dụng

Các giao dịch có thể đo gồm:

tasks_created_total.

resource_upload_completed_total và resource_upload_bytes.

automation_executions_total.

deadline_reminders_delivered_total.

approval_runs_started_total.

Ví dụ, trong tháng một tenant tạo 600 task, hoàn tất 80 lượt upload, chạy 250 automation và gửi thành công 900 nhắc hạn. Hệ thống tổng hợp các sự kiện theo tenant và ngày thay vì phải đọc lại toàn bộ bảng nghiệp vụ vào cuối tháng.

Luồng nghiệp vụ của người dùng

Thành viên thực hiện một thao tác nghiệp vụ.

Giao dịch nghiệp vụ thành công và usage event được ghi trong cùng transaction hoặc thông qua transactional outbox.

Worker xử lý event theo cơ chế idempotent để không đếm trùng khi retry.

Số liệu được cộng vào bản tổng hợp theo ngày.

Owner và System Admin xem số liệu đã tổng hợp theo kỳ.

Bản chất thể hiện qua ví dụ

Metric gắn với hành động có ý nghĩa nên dễ giải thích hơn số request kỹ thuật. Tuy nhiên không phải mọi hành động đều nên trở thành đơn vị tính phí. Nếu việc tạo task bình thường làm hóa đơn tăng ngay, người dùng có thể ngại sử dụng chức năng cốt lõi. Với Bridge, transaction metric trước tiên nên phục vụ phân tích mức sử dụng, quota cho tính năng tốn tài nguyên và nghiên cứu; chưa nên mặc định là billable metric.

4. Chỉ số theo request API

Định nghĩa

Per-request metric đếm số request gửi đến API. Đây là cách đo thuận tiện đối với sản phẩm cung cấp API trực tiếp cho khách hàng và mỗi request có ý nghĩa tương đối ổn định.

Nếu áp dụng vào ứng dụng

Bridge là SPA: một lần người dùng mở bảng Kanban có thể tạo nhiều request để lấy project, board, task, member và notification. Một thay đổi cách frontend cache hoặc gom request có thể làm số request giảm dù người dùng vẫn thực hiện cùng nghiệp vụ.

Ví dụ:

Phiên bản A tải trang bằng 8 request.

Sau khi tối ưu, phiên bản B tải cùng trang bằng 4 request.

Người dùng nhận cùng một giá trị nhưng nếu tính tiền theo request thì chi phí bị giảm một nửa chỉ vì thay đổi kỹ thuật nội bộ.

Luồng nghiệp vụ nếu dùng để kiểm soát fair-use

Request đã được xác thực và xác định tenant.

Bộ giới hạn kiểm tra ngân sách request của tenant trong cửa sổ thời gian.

Request hợp lệ được xử lý và metric vận hành được tăng.

Nếu vượt giới hạn, API trả 429 cùng thời gian thử lại.

System Admin dùng số liệu để phát hiện tenant gây tải bất thường.

Bản chất thể hiện qua ví dụ

Request count phản ánh áp lực lên API tốt hơn mức độ giá trị mà người dùng nhận được. Hai request cũng không nhất thiết có cùng chi phí xử lý. Vì vậy Bridge nên dùng request metric cho vận hành, fair-use, noisy-neighbor và thí nghiệm; không dùng nó làm đơn vị tính tiền chính ở giai đoạn hiện tại.

Microsoft cũng cảnh báo telemetry có sampling không phù hợp làm nguồn dữ liệu billing cần độ chính xác cao. Prometheus của Bridge vì thế không được xem là sổ cái thanh toán.

5. Ước lượng mức tiêu thụ

Định nghĩa

Estimated consumption phân bổ tổng chi phí dùng chung theo một tỷ lệ hợp lý thay vì cố đo chính xác từng đơn vị tài nguyên mà tenant đã dùng.

Phương pháp này phù hợp khi:

Ứng dụng chưa tính giá trực tiếp theo usage.

Mỗi tenant chỉ chiếm một phần nhỏ của deployment.

Chi phí đo chính xác cao hơn giá trị thu được.

Mẫu sử dụng giữa các tenant không quá khác biệt.

Nếu áp dụng vào ứng dụng

Bridge có thể ước lượng tỷ trọng tải của tenant bằng tổ hợp:

Tỷ lệ request đã xử lý.

Dung lượng đang lưu.

Số transaction nghiệp vụ nặng.

Thời gian xử lý hoặc truy vấn được lấy mẫu.

Ví dụ minh họa: tenant A chiếm 8% request, 10% dung lượng và 7% automation của cụm Pool. Nhóm có thể ước lượng tenant A gây ra khoảng 8–10% phần chi phí dùng chung, nhưng phải ghi rõ đây là ước lượng chứ không phải phép đo chi phí tuyệt đối.

Bản chất thể hiện qua ví dụ

Ước lượng đủ tốt cho capacity planning, so sánh tương đối và đánh giá COGS ban đầu. Nó không đủ đáng tin để tranh chấp hóa đơn nếu công thức, nguồn dữ liệu và sai số chưa được công bố.

6. Phân bổ trực tiếp chi phí tài nguyên

Định nghĩa

Direct cost attribution gán chi phí của tài nguyên dành riêng cho một tenant trực tiếp cho tenant đó. Trên nền tảng đám mây, việc gán thường dựa trên subscription, resource group, resource name hoặc tag.

Nếu áp dụng vào ứng dụng

Với POOL, database và bảng được dùng chung nên không thể gán trực tiếp toàn bộ chi phí database cho một tenant.

Với SCHEMA_PER_TENANT, schema tách riêng nhưng database server và compute vẫn dùng chung.

Với SILO_DATABASE, database logic tách riêng nên dễ theo dõi hơn; tuy nhiên nếu nhiều database vẫn nằm trên cùng một PostgreSQL server thì CPU, RAM và I/O vật lý vẫn chưa hoàn toàn dành riêng.

Chỉ khi một tenant có database instance, storage hoặc hạ tầng thực sự riêng thì phần chi phí đó mới phù hợp để pass-through trực tiếp.

Bản chất thể hiện qua ví dụ

Placement thể hiện biên cô lập dữ liệu, không tự động đồng nghĩa với biên chi phí vật lý. Không được tuyên bố "Silo đo chi phí chính xác hoàn toàn" nếu API, worker, MinIO hoặc PostgreSQL host vẫn dùng chung.

7. Ba hệ thống số liệu phải được tách biệt

7.1 Số liệu vận hành

Trả lời các câu hỏi như:

API có chậm không?

Error rate có tăng không?

Database connection có gần cạn không?

Worker có backlog hay dead letter không?

Nguồn phù hợp là Micrometer, Prometheus, Grafana và structured logs.

7.2 Sổ mức sử dụng theo tenant

Trả lời các câu hỏi như:

Tenant đã dùng bao nhiêu dung lượng trong kỳ?

Tenant đã chạy bao nhiêu automation?

Số liệu nào đã được dùng để áp quota hoặc tính tiền?

Nguồn phù hợp là một usage store bền vững, có chống trùng, tổng hợp theo kỳ và đối soát được.

7.3 Dữ liệu thực nghiệm nghiên cứu

Trả lời các câu hỏi như:

Pool, Schema-per-tenant và Silo khác nhau thế nào trên cùng workload?

Metering làm tăng bao nhiêu latency hoặc database write?

Rate limit giảm ảnh hưởng noisy neighbor đến đâu?

Nguồn phù hợp là các run có manifest, raw data, checksum, số lần lặp và điều kiện thí nghiệm đã khóa. Số liệu development không được đổi nhãn thành kết quả thực nghiệm chính thức.

8. Trạng thái đo lường hiện tại của Bridge

8.1 Đã hiện thực

API có Micrometer/Actuator và gắn các nhãn tenant_id, tenant_tier, tenant_placement vào HTTP observation.

Prometheus/Grafana có dashboard request rate, HTTP p95, database connection và memory.

Resource service tính tổng size_bytes của resource active để kiểm tra và hiển thị quota.

Tenant rate limit đã tồn tại theo tier.

Bộ công cụ nghiên cứu đã định nghĩa các metric như latency, throughput, error rate, CPU, RAM và connection count.

Transactional outbox và worker idempotent đã tồn tại cho các luồng notification, resource cleanup và automation; đây là nền tảng có thể tái sử dụng cho usage event.

8.2 Chưa hiện thực

Chưa có bảng usage_event, tenant_usage_daily hoặc usage_policy.

Chưa có lịch sử usage theo ngày và tháng.

Chưa có trang Usage tổng hợp dành cho Owner hoặc System Admin.

Quota dung lượng theo tier còn nằm trong mã nguồn thay vì policy có version trong control plane.

Rate limit và khóa upload quota còn nằm trong bộ nhớ của một API process, chưa bảo đảm đúng khi scale nhiều instance.

Worker chưa có metrics endpoint riêng.

Chưa có cơ chế reservation và reconciliation cho quota.

Chưa có dữ liệu production/VPS hoặc thực nghiệm chính thức để hiệu chỉnh mô hình chi phí.

8.3 Chưa được phép kết luận

Placement nào rẻ hoặc nhanh nhất trong điều kiện triển khai thật.

Mỗi task, request, automation hoặc GB thực sự gây ra bao nhiêu chi phí.

Các mức quota và giá mô phỏng hiện tại đã phù hợp với người dùng.

9. Kiến trúc đo lường đề xuất

Kiến trúc đo lường mức tiêu thụ được tổ chức thành các luồng xử lý và các tầng lưu trữ rõ ràng như sau:

1. **Luồng dữ liệu nghiệp vụ (Application & Control Plane):**
   - Thao tác từ người dùng phát sinh dữ liệu và cập nhật trực tiếp trên Application Plane.
   - Đồng thời, một sự kiện mức sử dụng (usage event) được ghi vào bảng transactional outbox trong cùng giao dịch cục bộ (local transaction).
   - Worker Idempotent tiếp nhận sự kiện từ outbox và đẩy dữ liệu về Module Quản lý Usage trong Control Plane.
2. **Tầng lưu trữ & Xử lý dữ liệu Usage (Control Plane):**
   - **usage_event / deduplication:** Tiêu thụ sự kiện và loại bỏ các sự kiện trùng lặp.
   - **tenant_usage_daily:** Tổng hợp dữ liệu sử dụng theo từng tenant theo ngày.
   - **usage_policy:** Kiểm tra và áp dụng chính sách hạn mức (quota/limit) cho từng tenant.
3. **Đầu ra & Giao diện báo cáo:**
   - **Giao diện Tenant:** Hiển thị trang Usage chi tiết cho Owner của Tenant.
   - **Giao diện Admin:** Cung cấp báo cáo tổng quan dành cho System Admin.
4. **Tầng hạ tầng & Vận hành (Infrastructure Monitoring):**

• Các thành phần hạ tầng (API + Worker + PostgreSQL + MinIO) liên tục đẩy metrics về hệ thống Prometheus/Grafana để theo dõi sức khỏe hệ thống, độ nhạy (latency), tỷ lệ lỗi và năng lực xử lý (capacity).

Nguyên tắc thiết kế

Raw dữ liệu nghiệp vụ vẫn nằm trong application plane của tenant.

Control plane chỉ nhận metadata usage tối thiểu: tenant, mã metric, đơn vị, số lượng, thời điểm, placement và idempotency key.

Không đưa tên task, nội dung comment, filename hoặc dữ liệu nhạy cảm vào usage store.

Giao dịch nghiệp vụ và outbox event phải được ghi cùng local transaction.

Worker xử lý at-least-once nhưng phép cộng usage phải idempotent.

Prometheus không phải nguồn sự thật cho quota hoặc billing.

Usage policy phải có version và ngày hiệu lực để giải thích vì sao một tenant bị giới hạn tại một thời điểm.

10. Mô hình dữ liệu tối thiểu

Bảng usage_event

Trường	Ý nghĩa
event_id	Khóa duy nhất để chống ghi nhận trùng
tenant_id	Tenant phát sinh mức sử dụng
metric_code	Ví dụ RESOURCE_UPLOAD_BYTES
quantity	Số lượng theo đơn vị metric
unit	BYTE , COUNT , MILLISECOND ...
occurred_at	Thời điểm nghiệp vụ xảy ra
placement	Placement tại thời điểm phát sinh
source	API, worker, reconciliation...
idempotency_key	Khóa chống đếm trùng khi retry

Bảng tenant_usage_daily

Trường	Ý nghĩa
tenant_id	Tenant sở hữu số liệu
usage_date	Ngày tổng hợp
metric_code	Loại metric
quantity	Tổng trong ngày hoặc snapshot cuối ngày
updated_at	Thời điểm tổng hợp gần nhất

Khóa duy nhất nên bao gồm tenant_id, usage_date, metric_code và dimension hữu hạn nếu metric cần phân nhóm.

Bảng usage_policy

Trường	Ý nghĩa
tier	Gói áp dụng
metric_code	Metric bị giới hạn
soft_limit	Ngưỡng cảnh báo
hard_limit	Ngưỡng chặn
enforcement_action	Cảnh báo, throttle hoặc từ chối thao tác tăng usage
effective_from	Thời điểm policy bắt đầu có hiệu lực

11. Các metric đề xuất cho Bridge

11.1 Metric authoritative dùng cho quota

Metric	Cách đo	Cách dùng
storage_active_bytes	Tổng dung lượng resource active, có reconciliation	Storage quota
active_member_count	Snapshot membership hiệu lực	Seat/plan limit nếu sản phẩm chọn
automation_executions	Event sau khi execution được chấp nhận	Quota tính năng nâng cao
resource_upload_bytes	Event khi upload hoàn tất	Theo dõi lưu lượng ghi hoặc add-on storage

11.2 Metric dùng phân tích sản phẩm

Metric	Cách dùng
tasks_created_total	Đo adoption của nghiệp vụ cốt lõi
projects_active	Đánh giá quy mô sử dụng workspace
approval_runs_started_total	Đo mức dùng capability Approval
notifications_delivered_total	Đánh giá mức dùng reminder đa kênh

feature_active_users	Đo người dùng thực sự nhận giá trị từ từng module

11.3 Metric chỉ dùng vận hành và nghiên cứu

Metric	Cách dùng
HTTP p50/p95/p99	Đánh giá đáp ứng và SLO
Error rate	Phát hiện suy giảm chất lượng
Request rate	Fair-use và noisy-neighbor
CPU/RAM	Capacity và so sánh placement
Database connection	Kiểm tra connection budget
Outbox/worker lag	Phát hiện backlog
Retry/dead-letter	Đánh giá độ tin cậy

Không đưa trực tiếp các metric vận hành này vào hóa đơn người dùng.

12. Luồng kiểm soát quota được đề xuất

Ví dụ upload tài nguyên

Người dùng chọn file trong một project mà họ có quyền ghi.

API xác định tenant từ host và phiên đã xác thực; không nhận tenantId làm nguồn quyết định từ client.

Hệ thống tạo reservation dung lượng bằng thao tác atomic trong database.

Nếu vượt hard limit, API từ chối trước khi lưu object và trả thông báo dễ hiểu.

Nếu còn quota, file được lưu vào namespace riêng của tenant.

Metadata và usage event được hoàn tất idempotent.

Nếu upload lỗi, reservation được giải phóng.

Job reconciliation định kỳ so sánh usage counter với metadata resource thực tế.

Khóa ReentrantLock trong một JVM không đủ khi có nhiều API instance. Reservation hoặc counter phải được bảo vệ bằng transaction, row lock, optimistic version hoặc câu lệnh cập nhật có điều kiện trong PostgreSQL.

Ví dụ đạt quota automation

Automation rule được trigger.

Worker kiểm tra entitlement và quota hiện hành.

Nếu còn quota, execution được tạo và event usage được ghi.

Nếu hết quota, execution không chạy; lịch sử ghi lý do QUOTA_EXCEEDED.

Owner nhận thông báo và có thể điều chỉnh rule hoặc gói.

13. Quản lý cardinality và quyền riêng tư

tenant_id trên Prometheus hữu ích khi demo hoặc thực nghiệm với ít tenant, nhưng trở thành label có cardinality cao khi số tenant tăng. Điều này làm tăng bộ nhớ, dung lượng và độ phức tạp truy vấn.

Hướng áp dụng:

Metric vận hành toàn hệ thống ưu tiên label hữu hạn như placement, tier, route_group và status_family.

Chi tiết usage theo tenant nằm trong PostgreSQL và được kiểm soát quyền.

Chỉ bật label tenant cho môi trường nghiên cứu, nhóm tenant giới hạn hoặc cơ chế chẩn đoán có thời hạn.

Không dùng raw URL, project ID, task ID, email hoặc filename làm metric label.

Owner/Admin chỉ được xem aggregate usage của tenant mình. System Admin được xem aggregate giữa tenant nhưng không nhờ đó có quyền đọc task, comment hoặc nội dung tài nguyên của tenant.

14. So sánh các hướng đo đối với ứng dụng

Hướng đo	Thứ được đo	Người dùng nhìn thấy	Điểm mạnh	Vấn đề chính	Quyết định cho Bridge
Chỉ số đại diện	Dung lượng, member, project	Thanh quota dễ hiểu	Rẻ, trực quan	Không phản ánh mọi chi phí	**Dùng làm trục chính**
Giao dịch nghiệp vụ	Upload, automation, reminder	Số hoạt động theo kỳ	Gắn với giá trị sản phẩm	Có thể làm người dùng ngại thao tác nếu tính phí quá chi tiết	**Dùng cho quota nâng cao và analytics**
Request API	Mỗi HTTP request	Khó liên hệ với thao tác UI	Tốt cho fair-use và tải API	Request không đồng đều; SPA phụ thuộc cách gọi API	**Không dùng làm đơn vị giá chính**

Ước lượng	Tỷ trọng trên tổng chi phí	Báo cáo COGS tương đối	Dễ bắt đầu	Có sai số, khó kiểm toán hóa đơn	**Dùng cho capacity và nghiên cứu ban đầu**
Phân bổ trực tiếp	Tài nguyên dành riêng	Phụ phí isolation/dedicated	Rõ với tài nguyên riêng	Phần dùng chung vẫn khó phân bổ	**Chỉ dùng khi thật sự có tài nguyên riêng**

15. Giải pháp phù hợp nhất cho Bridge

Giải pháp sáng giá nhất là **mô hình đo lường lai gồm chỉ số đại diện, giao dịch nghiệp vụ và hiệu chỉnh định kỳ bằng số liệu hạ tầng**.

Thành phần chính

Dùng storage_active_bytes và giới hạn member làm các chỉ số quota dễ hiểu.

Ghi transaction metric cho upload, automation, reminder và các capability tốn tài nguyên.

Dùng request count cho rate limit, noisy-neighbor và nghiên cứu, không làm đơn vị tính tiền chính.

Lưu usage có thể kiểm toán trong PostgreSQL; không dùng Prometheus làm billing ledger.

Tận dụng transactional outbox và worker hiện có, chưa cần Kafka hoặc dịch vụ metering bên thứ ba.

Định kỳ đối chiếu usage với CPU, RAM, connection, storage và worker load để kiểm tra giả định.

Chỉ phân bổ trực tiếp chi phí dedicated khi tài nguyên vật lý thực sự dành riêng cho tenant.

Vì sao lựa chọn này phù hợp?

Phù hợp với mô hình giá theo gói tính năng/mức dịch vụ đã được đề xuất cho Bridge.

Người dùng mới hiểu được quota mà không cần biết API request hay compute time.

Người dùng chuyên môn vẫn xem được metric chi tiết, placement và nguyên nhân giới hạn.

Không làm phức tạp hệ thống quá sớm.

Có thể phát triển thành billing usage sau này mà không phải bỏ nền tảng đã xây.

Hỗ trợ trực tiếp mục tiêu nghiên cứu về cô lập, hiệu năng và mức tiêu thụ theo tenant.

16. Lộ trình triển khai

Giai đoạn 1 — Chốt hợp đồng đo lường

Tạo metric catalog.

Ghi rõ đơn vị, nguồn, độ chính xác, mục đích, retention và quyền xem.

Chọn metric tối thiểu; không đo mọi hành động chỉ vì có thể đo.

Giai đoạn 2 — Xây usage module tối thiểu

Thêm Flyway migration cho usage policy, event/deduplication và daily aggregate.

Ghi usage event qua outbox cho các giao dịch quan trọng.

Thêm worker aggregation và reconciliation.

Chuyển quota hard-code thành policy có version.

Giai đoạn 3 — Hoàn thiện vận hành

Thêm worker metrics endpoint.

Thêm metric outbox lag, retry và dead-letter.

Giới hạn cardinality Prometheus.

Làm rate limit và quota an toàn khi chạy nhiều instance.

Giai đoạn 4 — Hoàn thiện giao diện

Trang Usage cho Owner/Admin.

Dashboard tổng hợp cho System Admin.

Cảnh báo 80%, 90% và 100%.

Hiển thị metric, đơn vị, kỳ tính và cách xử lý khi vượt giới hạn.

Giai đoạn 5 — Thực nghiệm và hiệu chỉnh

Chạy workload giống nhau trên ba placement.

So sánh khi tắt và bật metering để đo overhead.

Lặp lại theo protocol, lưu raw data và manifest.

Dùng kết quả thật để hiệu chỉnh quota, không suy ra quota từ demo local.

17. Thiết kế phép đo phục vụ nghiên cứu

Mỗi placement cần chạy cùng seed, tenant count, workload, VU, duration và rate-limit policy. Tối thiểu thu thập:

HTTP p50, p95 và p99.

Throughput và error rate.

CPU, RAM và database connection.

Database write phát sinh do metering.

Dung lượng bảng usage sau mỗi run.

Outbox lag và worker processing lag.

Tỷ lệ event trùng, retry và reconciliation mismatch.

Cần có hai nhóm chạy:

Baseline: chưa bật usage event mới.

Metering enabled: bật đầy đủ usage event và aggregation.

Chênh lệch giữa hai nhóm cho biết chi phí kỹ thuật của cơ chế đo lường. Kết quả development hoặc pilot chỉ dùng điều chỉnh phương pháp; chỉ run hợp lệ theo protocol mới được đưa vào kết luận nghiên cứu.

18. Kết luận cuối cùng

Bridge không nên tìm một metric duy nhất để đồng thời đại diện cho giá trị người dùng, chi phí hạ tầng, quota và kết quả nghiên cứu. Cách đúng hơn là tách vai trò:

Quota và gói: dung lượng, member và một số capability tốn tài nguyên.

Product analytics: transaction nghiệp vụ.

Fair-use và noisy-neighbor: request rate.

Vận hành: Prometheus/Grafana và logs.

Kiểm toán/billing tương lai: usage store bền vững trong PostgreSQL.

Nghiên cứu: bộ run độc lập, tái lập và có manifest.

Vì vậy, phương án được chọn là:

> **Indicative + transaction metering trong một usage module tenant-aware, sử dụng PostgreSQL, transactional outbox và worker hiện tại; Prometheus chỉ phục vụ quan sát vận hành; chi phí hạ tầng được dùng để hiệu chỉnh định kỳ.**

Ở thời điểm hiện tại, ứng dụng mới có nền tảng quan sát và quota dung lượng, chưa có consumption metering hoàn chỉnh. Bước triển khai đúng tiếp theo là chốt metric catalog, xây usage module tối thiểu, làm quota an toàn khi scale ngang, sau đó mới chạy thực nghiệm và quyết định có sử dụng usage cho pricing hay không.

Pricing models for a multitenant solution - Azure Architecture Center | Microsoft Learn

Các mô hình định giá và cách áp dụng vào ứng dụng SaaS đa thuê bao

Một **tenant/workspace** là không gian làm việc của một nhóm hoặc tổ chức.

Người dùng có thể tạo dự án, quản lý task trên bảng công việc, mời thành viên, chia sẻ tài nguyên và nhận nhắc deadline.

Ứng dụng có ba cách bố trí dữ liệu: POOL, SCHEMA_PER_TENANT và SILO_DATABASE.

Các con số trong ví dụ chỉ dùng để giải thích bản chất mô hình, không phải bảng giá chính thức.

1. Mô hình tính theo mức tiêu thụ

Định nghĩa

Mô hình **consumption-based** hoặc **pay-as-you-go** tính tiền theo lượng tài nguyên hay số lần dịch vụ thực sự được sử dụng. Khách hàng dùng nhiều trả nhiều, dùng ít trả ít.

Đơn vị tính có thể là dung lượng lưu trữ, số thông báo đã gửi, số lần chạy automation, thời gian xử lý hoặc số API request.

Nếu áp dụng vào ứng dụng

Giả sử ứng dụng quy định:

2.000 đồng cho mỗi GB lưu trữ trong một tháng.

10.000 đồng cho mỗi 1.000 lượt nhắc deadline.

20 đồng cho mỗi lần chạy automation.

Một nhóm tạo workspace, quản lý năm dự án, tải lên 3 GB tài liệu và phát sinh 2.000 lượt nhắc deadline. Cuối tháng, hệ thống đọc sổ usage và lập hoá đơn đúng theo lượng đã dùng.

Luồng nghiệp vụ của người dùng sẽ là:

Owner tạo workspace mà không cần chọn một gói cố định lớn.

Thành viên tạo task, tải tài liệu và cấu hình nhắc deadline.

Mỗi hoạt động có tính phí được hệ thống ghi nhận.

Owner theo dõi chi phí tạm tính trên trang usage.

Cuối kỳ, hệ thống tổng hợp usage và xuất hoá đơn.

Bản chất thể hiện qua ví dụ

Tiền gắn trực tiếp với hành vi sử dụng. Mô hình phản ánh chi phí tốt nhưng người dùng có thể ngại tải tài liệu, tạo automation hoặc bật nhắc deadline vì mỗi thao tác có thể làm tăng hoá đơn. Việc đo và đối soát cũng phải rất chính xác.

2. Mô hình tính theo người dùng đăng ký

Định nghĩa

Mô hình **per-user** tính tiền theo số tài khoản hoặc số seat được cấp trong workspace, bất kể từng người có sử dụng thường xuyên hay không.

Nếu áp dụng vào ứng dụng

Giả sử giá là 30.000 đồng/người/tháng. Owner mời 20 thành viên vào workspace thì phí tháng là:

20 thành viên × 30.000 đồng = 600.000 đồng/tháng

Trong tháng đó, dù chỉ có 12 người đăng nhập và cập nhật task, workspace vẫn trả cho đủ 20 seat.

Luồng nghiệp vụ của người dùng sẽ là:

Owner mua một số lượng seat.

Owner mời thành viên vào workspace.

Khi hết seat, hệ thống yêu cầu mua thêm trước khi mời người mới.

Thành viên bị vô hiệu hoá hoặc bị xoá thì seat mới được giải phóng.

Bản chất thể hiện qua ví dụ

Đối tượng được tính tiền là **quyền truy cập đã cấp**, không phải hoạt động thực tế. Mô hình dễ hiểu và doanh thu dễ dự đoán, nhưng có thể khiến nhóm ngại mời sinh viên, cộng tác viên hoặc người chỉ tham gia một giai đoạn ngắn.

3. Mô hình tính theo người dùng hoạt động

Định nghĩa

Mô hình **per-active-user** chỉ tính những người thực sự hoạt động trong kỳ, thường gọi là monthly active user (MAU).

Ứng dụng phải định nghĩa rõ thế nào là “hoạt động”. Ví dụ, một người được tính active khi họ tạo, cập nhật hoặc bình luận task; việc refresh token tự động không được xem là hoạt động nghiệp vụ.

Nếu áp dụng vào ứng dụng

Owner mời 40 người vào workspace nhưng trong tháng chỉ có 18 người đăng nhập và thao tác với dự án. Nếu giá là 30.000 đồng/active user thì phí là:

18 active users × 30.000 đồng = 540.000 đồng/tháng

22 người còn lại vẫn giữ tài khoản và có thể quay lại vào tháng sau nhưng không bị tính phí trong tháng hiện tại.

Luồng nghiệp vụ của người dùng sẽ là:

Owner có thể mời thành viên mà chưa phát sinh phí ngay.

Khi một thành viên thực hiện hành động nghiệp vụ đầu tiên trong kỳ, họ được ghi nhận là active.

Trang quản trị hiển thị danh sách và số active user tạm tính.

Cuối kỳ, hệ thống chốt snapshot và tính tiền theo số active user.

Bản chất thể hiện qua ví dụ

Tiền gắn với **người thực sự nhận giá trị trong kỳ**. Mô hình công bằng hơn per-user đối với nhóm có thành viên tham gia không thường xuyên, nhưng doanh thu khó dự đoán hơn và hệ thống phải lưu số liệu có thể kiểm toán.

4. Mô hình tính theo đơn vị nghiệp vụ

Định nghĩa

Mô hình **per-unit** tính tiền theo một đơn vị có ý nghĩa đối với nghiệp vụ thay vì theo từng người. Đơn vị có thể là workspace, project, nhóm, giao dịch hoặc một đối tượng khác mà khách hàng dễ hiểu.

Nếu áp dụng vào ứng dụng

Có hai cách gần với ứng dụng:

Tính theo workspace: mỗi workspace trả một mức cố định. Một giảng viên tạo workspace cho nhóm nghiên cứu thì trả một phí thuê bao, không phụ thuộc workspace có 8 hay 15 thành viên.

Tính theo project: gói cơ bản bao gồm ba project. Khi nhóm tạo project thứ tư, hệ thống yêu cầu mua thêm một project hoặc nâng gói.

Ví dụ luồng tính theo project:

Owner đăng ký workspace với ba project đi kèm.

Thành viên sử dụng đầy đủ task, tài nguyên và deadline trong ba project đó.

Khi Manager tạo project thứ tư, UI hiển thị phần chi phí tăng thêm.

Owner xác nhận mua thêm project rồi hệ thống mới tạo project.

Bản chất thể hiện qua ví dụ

Tiền gắn với **đơn vị tạo ra giá trị nghiệp vụ**. Cách tính theo workspace rất dễ hiểu, nhưng một workspace nhỏ và một workspace có hàng trăm thành viên, nhiều tệp và automation vẫn có thể trả cùng giá. Tính theo project lại có thể khiến người dùng dồn nhiều công việc không liên quan vào một project để tránh phí.

5. Mô hình tính theo gói tính năng hoặc mức dịch vụ

Định nghĩa

Mô hình **feature-based/service-level pricing** chia sản phẩm thành một số gói. Mỗi gói có tập tính năng, hạn mức, mức cô lập dữ liệu, hỗ trợ hoặc cam kết dịch vụ khác nhau.

Khách hàng không trả tiền cho từng thao tác. Họ trả tiền để được sử dụng một mức năng lực của sản phẩm trong kỳ thuê bao.

Nếu áp dụng vào ứng dụng

Có thể tổ chức sản phẩm thành ba gói như sau:

Gói minh hoạ	Nghiệp vụ người dùng nhận được	Cấu hình phù hợp
Starter	Tạo project, bảng công việc, task, thành viên, chia sẻ tài nguyên và nhắc deadline; hạn mức cơ bản	POOL
Professional	Toàn bộ Starter, hạn mức cao hơn và tuỳ biến dữ liệu theo nhu cầu nhóm	SCHEMA_PER_TENANT
Enterprise	Toàn bộ Professional, thêm phê duyệt, automation, cô lập dữ liệu riêng và mức hỗ trợ cao hơn	SILO_DATABASE

Ví dụ nghiệp vụ:

Một nhóm sinh viên chọn Starter để quản lý đề tài và deadline.

Khi cần thêm trường dữ liệu riêng cho task, Owner mở trang nâng gói.

UI giải thích Professional hỗ trợ custom data và mức cô lập schema riêng.

Owner xác nhận nâng gói; entitlement mới được kích hoạt.

Một doanh nghiệp cần quy trình duyệt task và database riêng sẽ chọn Enterprise.

Bản chất thể hiện qua ví dụ

Tiền gắn với **mức giá trị và mức dịch vụ mà khách hàng mua**, không gắn với từng lần nhấn nút. Người mới dễ chọn theo nhu cầu; người có yêu cầu chuyên sâu nhìn thấy rõ khác biệt về tính năng và cô lập. Rủi ro của mô hình là tạo quá nhiều gói hoặc mô tả khác biệt không rõ ràng.

6. Mô hình freemium

Định nghĩa

Mô hình **freemium** cung cấp một phiên bản miễn phí lâu dài với tính năng hoặc hạn mức thấp. Khách hàng chỉ trả tiền khi muốn mở rộng quy mô hoặc dùng tính năng nâng cao.

Nếu áp dụng vào ứng dụng

Ví dụ gói Free có thể gồm:

Một workspace dùng POOL.

Một project.

Tối đa năm thành viên hoạt động.

50 MB tài nguyên.

Chức năng task và deadline cơ bản.

Một nhóm dùng miễn phí để quản lý bài tập. Khi muốn tạo project thứ hai hoặc mời thành viên thứ sáu, ứng dụng hiển thị màn hình nâng lên Starter.

Luồng nghiệp vụ:

Người dùng đăng ký và tạo workspace miễn phí.

Hệ thống cấp các chức năng cốt lõi với quota thấp.

Khi người dùng chạm quota hoặc chọn chức năng nâng cao, UI giải thích quyền lợi của gói trả phí.

Workspace vẫn giữ dữ liệu khi nâng gói.

Bản chất thể hiện qua ví dụ

Người dùng được trải nghiệm giá trị trước khi trả tiền. Đổi lại, hệ thống phải chịu chi phí cho nhiều tenant miễn phí hoặc bị bỏ quên. Với kiến trúc multi-tenant có provisioning và lưu trữ, freemium chỉ an toàn khi có quota, chính sách dọn dữ liệu và offboarding rõ ràng.

7. Mô hình tính theo giá vốn

Định nghĩa

Mô hình **COGS-based**, **pass-through** hoặc **chargeback** phân bổ chi phí thực tế để phục vụ từng tenant rồi chuyển chi phí đó cho đơn vị sử dụng. Mục tiêu chính thường là thu hồi chi phí hơn là tối đa hoá lợi nhuận.

Mô hình này thường phù hợp với hệ thống nội bộ của một trường, doanh nghiệp hoặc tập đoàn.

Nếu áp dụng vào ứng dụng

Giả sử trường đại học vận hành app cho các khoa:

Khoa A dùng POOL, ít dung lượng và ít thông báo nên được phân bổ chi phí thấp.

Khoa B yêu cầu SILO_DATABASE, backup riêng và nhiều dung lượng nên chịu chi phí cao hơn.

Cuối tháng, hệ thống tổng hợp chi phí database, storage, thông báo và vận hành để chargeback cho từng khoa.

Người dùng nghiệp vụ vẫn tạo project và task bình thường. Người quản trị tài chính của trường nhận báo cáo chi phí theo khoa thay vì hoá đơn theo từng tính năng.

Bản chất thể hiện qua ví dụ

Tiền gắn với **chi phí mà tenant gây ra cho đơn vị vận hành**. Mô hình hợp với triển khai nội bộ nhưng khó dùng cho SaaS thương mại vì chi phí hạ tầng dùng chung khó phân bổ chính xác và giá không thể hiện trực tiếp giá trị khách hàng nhận được.

8. Mô hình giá cố định

Định nghĩa

Mô hình **flat-rate** áp dụng một mức giá cố định cho sản phẩm, thường không phụ thuộc số người dùng, tính năng hoặc mức sử dụng.

Nếu áp dụng vào ứng dụng

Ví dụ mọi workspace đều trả 300.000 đồng/tháng và được dùng toàn bộ chức năng:

- Nhóm A có 5 người, 2 project và 100 MB tài nguyên vẫn trả 300.000 đồng.
- Tổ chức B có 100 người, 30 project, 20 GB tài nguyên, automation và SILO_DATABASE cũng trả 300.000 đồng.

Luồng nghiệp vụ rất đơn giản: Owner đăng ký, trả một mức tiền duy nhất rồi sử dụng tất cả chức năng mà không cần theo dõi quota tính tiền.

Bản chất thể hiện qua ví dụ

Tiền **không thay đổi theo quy mô hay mức dùng**. Đây là cách dễ hiểu nhất nhưng tenant lớn có thể tạo COGS cao hơn rất nhiều so với doanh thu và gây noisy-neighbor

cho tenant khác. Flat-rate chỉ an toàn nếu phạm vi sử dụng hẹp hoặc vẫn có giới hạn sử dụng hợp lý.

9. Chiết khấu và giá cho môi trường thử nghiệm

Chiết khấu không phải một mô hình định giá độc lập. Nó được áp dụng trên một mô hình cơ sở, ví dụ:

Giảm giá khi trả trước theo năm.

Giá trên mỗi active user giảm khi số người tăng theo dải.

Gói non-production rẻ hơn nhưng quota thấp và không có SLA.

Trial miễn phí trong thời gian xác định thay vì duy trì gói Free vô thời hạn.

Ví dụ, một workspace Professional trả theo tháng. Nếu Owner chuyển sang trả trước một năm, hệ thống áp dụng chiết khấu nhưng tính năng, placement và hạn mức của Professional không thay đổi.

10. So sánh bản chất các mô hình trong ứng dụng

Mô hình	Thứ làm phát sinh tiền	Người dùng nhìn thấy trong nghiệp vụ	Vấn đề chính đối với app
Theo mức tiêu thụ	Mỗi đơn vị tài nguyên/hành động	Upload, reminder hoặc automation làm usage tăng	Khó đo, hoá đơn khó đoán, có thể làm người dùng ngại thao tác
Theo người dùng	Mỗi tài khoản/seat được cấp	Mời thành viên mới làm phí tăng	Phải trả cả người ít hoạt động
Theo người dùng hoạt động	Mỗi người có hành động trong kỳ	Chỉ thành viên thực sự dùng app mới bị tính	Cần định nghĩa và đo MAU chính xác
Theo đơn vị nghiệp vụ	Mỗi workspace/project	Tạo thêm workspace hoặc project làm phí tăng	Đơn vị không luôn phản ánh COGS
Theo gói tính năng/mức dịch vụ	Quyền lợi và mức dịch vụ đã chọn	Nâng gói khi cần custom data, approval, automation hoặc isolation	Phải thiết kế gói rõ và không quá nhiều

Freemium	Chỉ trả khi vượt giới hạn Free	Chạm quota hoặc chọn chức năng nâng cao thì nâng gói	Tenant miễn phí vẫn gây chi phí
Theo COGS	Chi phí được phân bổ cho tenant	Quản trị viên nhận báo cáo chargeback	Khó phân bổ hạ tầng dùng chung; không phản ánh giá trị
Flat-rate	Một mức cố định	Dùng ít hay nhiều vẫn cùng giá	Tenant nặng có thể làm dịch vụ lỗ

11. Kết luận cuối cùng

Mô hình phù hợp nhất với ứng dụng là **tính theo gói tính năng/mức dịch vụ trên mỗi workspace/tenant**.

Lý do lựa chọn:

- Chức năng của app đã chia được thành các mức giá trị rõ ràng: nghiệp vụ quản lý công việc cốt lõi, custom data, phê duyệt và automation.
- Ba placement tạo ra ba mức chi phí và cô lập khác nhau, phù hợp để giải thích trong các gói hoặc dưới dạng phụ phí isolation.
- Người dùng mới chỉ cần chọn gói theo nhu cầu, không phải hiểu API request, compute hay cơ chế tính COGS.
- Người dùng chuyên môn vẫn thấy rõ capability, quota và mức cô lập dữ liệu mình nhận được.
- Thu nhập từ thuê bao workspace dễ dự đoán hơn mô hình consumption hoặc active-user thuần tuý.
- Hạn mức đi kèm giúp tránh rủi ro tenant dùng quá nhiều làm dịch vụ thua lỗ.

Cách áp dụng được đề xuất:

Phí phải trả = phí gói workspace + phụ phí placement/cô lập (nếu có)

Starter nên gắn với nghiệp vụ cốt lõi và POOL; Professional phù hợp với custom data và SCHEMA_PER_TENANT; Enterprise phù hợp với approval, automation và SILO_DATABASE. Mỗi gói cần có hạn mức người dùng, lưu trữ và thông báo đủ rộng để người dùng không phải cân nhắc chi phí cho từng thao tác bình thường.

Không chọn consumption, per-user, freemium, COGS-based hoặc flat-rate làm mô hình chính. Sau khi có dữ liệu nghiên cứu thực tế, **per-active-user hoặc storage chỉ nên được bổ sung như phần mua thêm**, không thay thế mô hình thuê bao phân tầng.

Các mức 100.000, 300.000 và 1.000.000 VND hiện có trong app chỉ là giá mô phỏng. Tài liệu này lựa chọn **cấu trúc định giá**, chưa khẳng định các mức tiền đó là phù hợp.

Đánh giá Control Plane Hiện Tại và Hướng Hoàn Thiện Cho Bridge

1. Kết luận trước

Bridge **không phải chưa có Control Plane**. Cách mô tả chính xác hơn là:

Bridge đã có một **Control Plane tùy chỉnh ở mức cơ bản**, được triển khai bên trong modular monolith. Nó gồm tenant catalog trong control_db, các API/UI quản trị và worker cấp phát tài nguyên. Phần này đã được kiểm chứng ở local, nhưng chưa hoàn chỉnh về toàn bộ vòng đời tenant, đo mức sử dụng, quan sát worker, backup/restore, độ sẵn sàng và cách ly production.

Vì vậy, nhóm không cần xây lại từ đầu hoặc chuyển sang một nền tảng khác. Hướng phù hợp là **chuẩn hóa và phát triển dần Control Plane hiện có**, giữ nguyên Spring Boot/PostgreSQL và React/Vite/MUI.

2. Control Plane là gì?

Một ứng dụng đa thuê bao có thể được nhìn thành hai phần:

- **Data Plane:** nơi người dùng làm công việc chính của sản phẩm. Với Bridge, đó là Project, Board, Task, Comment, Resource, Approval, Automation và Notification.
- **Control Plane:** nơi hệ thống hoặc quản trị viên quản lý tenant ở cấp toàn hệ thống: tạo tenant, cấu hình, cấp phát tài nguyên, theo dõi trạng thái, bảo trì và xử lý lỗi.

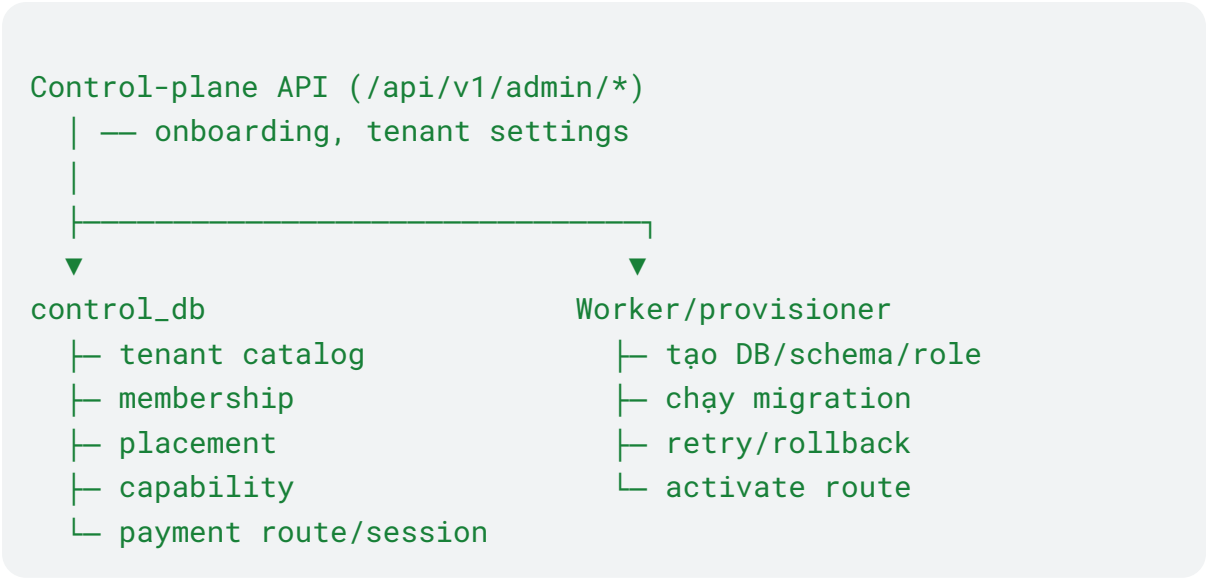
Microsoft mô tả Control Plane là phần thực hiện các tác vụ cấp cao như quản lý truy cập, provisioning và bảo trì trên nhiều tenant. Các trách nhiệm thường gặp gồm quản lý tài nguyên, cấu hình tenant, vòng đời tenant, telemetry và theo dõi mức sử dụng. Không có một mẫu Control Plane duy nhất cho mọi sản phẩm. Xem Considerations for multitenant control planes.

Ví dụ dễ hiểu:

- **Data Plane:** Người dùng tạo task, kéo task, bình luận...
- **Control Plane:** System Admin tạo/suspend tenant, cấp DB, retry…

3. Control Plane hiện tại của Bridge nằm ở đâu?

```
None
Accounts host / System Admin UI
  |
  ▼
```



Các phần đã có trong repo:

- control_db lưu user, tenant, membership, tier, route, placement, payment, provisioning, session, capability và branding.
- AdminController và các endpoint /api/v1/admin/* cung cấp API quản trị cấp hệ thống.
- AdminPage hiển thị tenant, tier, placement, trạng thái, payment, provisioning, lỗi và thao tác retry.
- SYSTEM_ADMIN được tách khỏi vai trò tenant OWNER/ADMIN/MEMBER và project MANAGER/MEMBER/VIEWER.
- Provisioning worker tạo Pool/Schema/Silo, chạy migration, health check, retry và rollback có kiểm soát.
- Có audit, lịch sử chuyển trạng thái provisioning và dead-letter/requeue cho cleanup file.
- Có onboarding self-service ở local và cơ chế chuyển người dùng vào đúng tenant host.

Tài liệu/mã đối chiếu:

- Tổng quan kiến trúc
- OpenAPI
- AdminController.java
- AdminService.java
- AdminPage.tsx
- Trạng thái dự án

4. Đây đã phải Control Plane hoàn chỉnh chưa?

Chưa. Hiện tại đây là **Basic Custom Control Plane chạy local**, chưa phải Control Plane production được cách ly và vận hành đầy đủ.

Trách nhiệm	Trạng thái Bridge	Nhận xét
Tenant catalog	**Đã có**	`control_db` là nguồn metadata trung tâm.

Trách nhiệm	Trạng thái Bridge	Nhận xét
Tenant onboarding	**Đã có local**	Có đăng ký, payment giả lập, provisioning và chuyển host.
Provisioning database/schema	**Đã có local**	Worker tạo resource, migration, retry và rollback.
Tenant routing	**Đã có local**	Host được đối chiếu với route, token và membership.
Tenant configuration	**Có phần lớn**	Membership, branding, capability và notification settings.
System Admin portal	**Có cơ bản**	Danh sách/detail tenant, filter, retry, capability và cleanup lỗi.
Long-running workflow	**Khá đầy đủ ở local**	Có job state, lease, idempotency, retry và rollback.
Audit	**Có một phần đáng kể**	Cần rà độ bao phủ mọi thao tác Control Plane.
Telemetry API	**Có nền tảng**	Prometheus hiện scrape API.
Telemetry worker	**Thiếu**	Worker chưa có endpoint metric riêng.
Consumption tracking	**Có một phần**	Có quota byte file; chưa có sổ usage tổng hợp đầy đủ theo tenant/kỳ.
Suspend/reactivate	**Một phần**	Có trạng thái `SUSPENDED`; chưa đủ workflow quản trị vòng đời.
Offboarding/archive/purge	**Thiếu**	Chưa có retention và quy trình xóa an toàn hoàn chỉnh.
Placement tự động theo capacity	**Chưa có**	Placement hiện được chọn lúc onboarding.
Chuyển tenant giữa placement	**Ngoài phạm vi**	Chưa có data migration Pool ↔ Schema ↔ Silo.
Backup/restore theo tenant	**Chưa chứng minh**	Có tài liệu/runbook nhưng cần restore drill trên môi trường thật.
Secret/certificate rotation	**Chưa đầy đủ**	Chưa có workflow tự động toàn diện.
SLO, RTO, RPO của Control Plane	**Chưa khóa**	Cần pilot và quyết định của nhóm.
Cách ly Control Plane khỏi Data Plane	**Mới tách logic/quyền**	Vẫn chung codebase/hạ tầng; chưa có bulkhead vận hành mạnh.

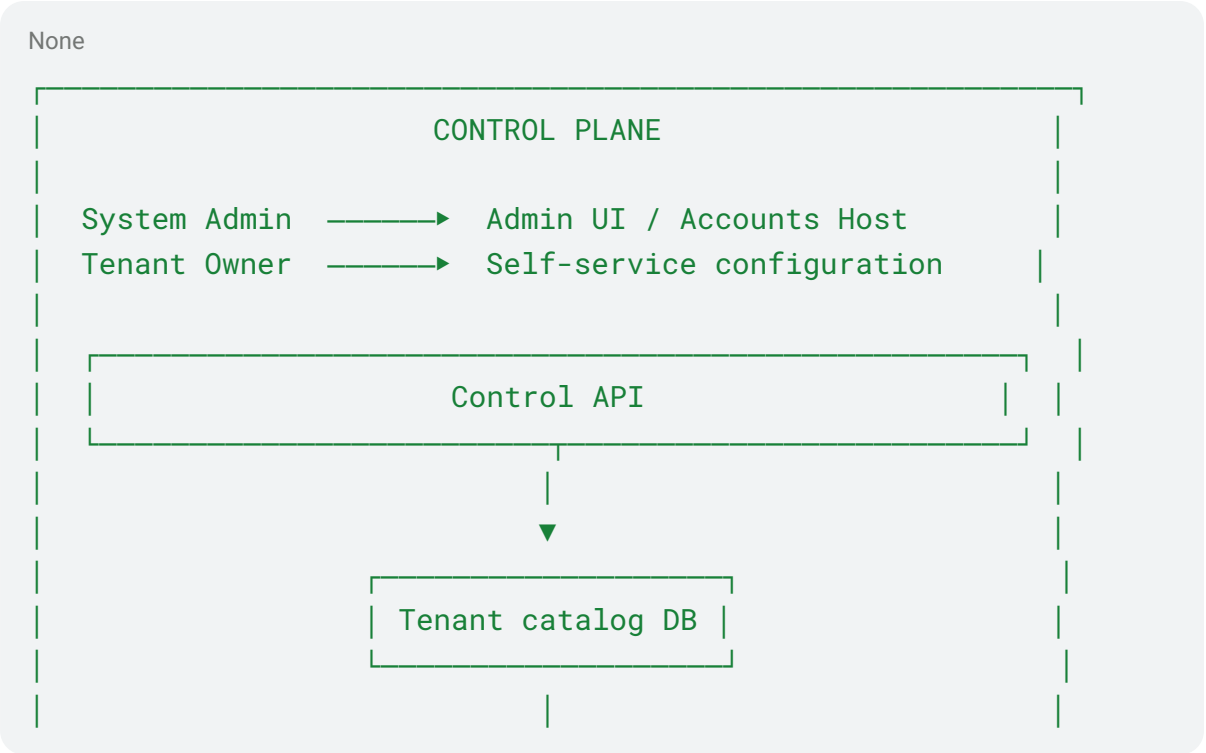
5. Microsoft đề xuất những cách làm nào?

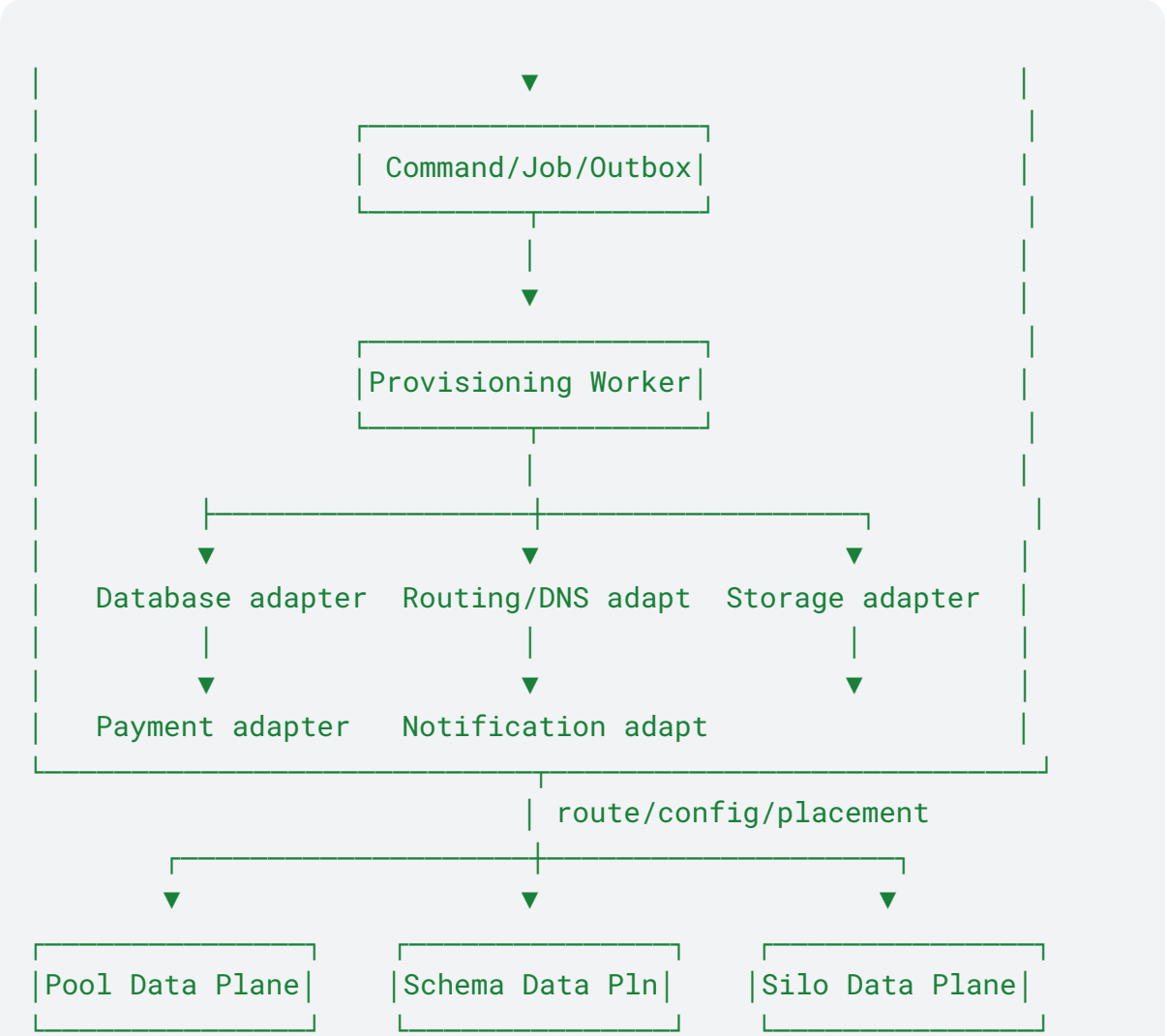
Microsoft phân loại các hướng triển khai như sau. Xem Architectural approaches for control planes.

Hướng	Đặc điểm	Phù hợp với Bridge?
Manual	Catalog bằng file/Excel và chạy script thủ công	Chỉ phù hợp giai đoạn rất sớm; Bridge đã vượt qua mức này.
Low-code	Dùng Power Platform/Power Automate hoặc công cụ tương tự	Không cần thiết vì nhóm đã có Spring Boot API, worker và portal.
Custom	Tự xây catalog, API, portal và background processing	**Phù hợp nhất** với trạng thái hiện tại.
Hybrid	Kết hợp workflow tự động với một số bước phê duyệt/thủ công	Hữu ích cho thao tác nguy hiểm như purge, restore hoặc retry đặc biệt.

Hướng đề xuất cho Bridge là **Custom Control Plane phát triển dần**, có thể giữ một số bước manual approval cho thao tác nguy hiểm. Microsoft nhấn mạnh Control Plane tùy chỉnh có độ linh hoạt và khả năng kiểm thử cao, nhưng đội dự án phải tự chịu trách nhiệm về bảo mật, độ tin cậy và bảo trì.

6. Kiến trúc Control Plane đề xuất cho Bridge





Không nhất thiết tách thành microservice ngay. Có thể giữ cùng repository và modular monolith, nhưng cần giữ các ranh giới:

- Control-plane API không được lẫn với API project/task bình thường.
- System Admin authorization được kiểm ở backend.
- API phục vụ người dùng không được giữ credential tạo/drop database hoặc role.
- Worker/provisioner dùng credential riêng và quyền tối thiểu.
- control_db không chứa nội dung project/task của tenant.
- Data Plane không tự quyết định placement từ dữ liệu client gửi lên.
- Mọi thao tác nguy hiểm phải audit và có state transition hợp lệ.

7. Control Plane nên quản lý dữ liệu gì?

Tenant catalog

Mỗi tenant nên có:

- Tenant ID, tên, slug và trạng thái.
- Owner và số thành viên hiện hành.
- Tier/gói và trạng thái subscription/payment.

- Placement Pool/Schema/Silo.
- Route/domain reference.
- Database/schema/resource reference an toàn.
- Schema version.
- Capability được hỗ trợ, được cấp và đang bật.
- Quota và usage tổng hợp.
- Trạng thái provisioning gần nhất.
- Deployment/stamp nếu tương lai có nhiều máy.
- Ngày suspend/offboard và retention policy.
- Phiên bản cấu hình để chống cập nhật đồng thời.

Không nên đưa nội dung task/comment/file vào tenant catalog. Control Plane chỉ cần metadata cần thiết để quản lý và vận hành tenant.

Lifecycle state

```
None
DRAFT
  |
  ▼
PENDING_PAYMENT
  |
  ▼
PROVISIONING ——→ RETRYABLE_FAILED / FAILED_ROLLED_BACK /
MANUAL_INTERVENTION
  |
  ▼
ACTIVE ————————→ SUSPENDED ——→ ACTIVE
  |
  ▼
OFFBOARDING ———→ ARCHIVED ———→ PURGED
```

Mỗi chuyển trạng thái cần quy định:

- Ai được thực hiện.
- Điều kiện trước khi chuyển.
- Dữ liệu/job/session nào bị ảnh hưởng.
- Có thể rollback không.
- Audit gì được ghi.
- Người dùng nhìn thấy trạng thái/lỗi như thế nào.

8. Các chức năng cần bổ sung

Ưu tiên 1 — Hoàn chỉnh tenant lifecycle

- System Admin suspend/reactivate tenant.
- Thu hồi session khi tenant bị suspend.
- Worker không tiếp tục job nghiệp vụ không còn hợp lệ.
- Offboarding có ngày hiệu lực và thời gian giữ dữ liệu.
- Archive trước khi purge.
- Gỡ route/domain an toàn.
- Xử lý file, notification, job chờ và backup.
- Purge cần xác nhận nhiều bước, idempotency và audit.

Ưu tiên 2 — Hoàn chỉnh telemetry và metering

Metric vận hành:

- Request count, latency và error rate.
- Connection pool active/pending.
- Provisioning duration.
- Job queue age.
- Job retry/dead-letter.
- Notification delivery latency/error.
- Storage error/quota rejection.

Usage theo tenant:

- Dung lượng file đang dùng.
- Số request theo ngày/kỳ.
- Số thành viên hoạt động nếu nghiệp vụ cần.
- Số thao tác nghiệp vụ được nhóm chọn làm đơn vị sử dụng.

Metric vận hành và usage cần tách nhau. Không đưa token, email, tên file hoặc nội dung task vào metric label. Nếu gắn tenant ID, phải kiểm soát cardinality, quyền xem và retention.

Ưu tiên 3 — Hoàn chỉnh System Admin portal

- Tenant list/detail và filter — đã có.
- Provisioning timeline/retry — đã có cơ bản.
- Capability grant/revoke — đã có.
- Suspend/reactivate — cần bổ sung.
- Usage/quota dashboard — cần bổ sung.
- Schema version và migration health — cần bổ sung rõ hơn.
- Worker queue/dead-letter summary — cần bổ sung.
- Backup/restore status — cần bổ sung sau khi có adapter thật.
- Offboarding/retention status — cần bổ sung.
- Cảnh báo và runbook link — cần bổ sung.

System Admin chỉ nên xem metadata vận hành cần thiết, không mặc nhiên được đọc nội dung project/task của tenant.

Ưu tiên 4 — Bảo mật Control Plane

- Giữ SYSTEM_ADMIN tách khỏi tenant/project role.
- Kiểm quyền ở service/backend, không dựa vào menu UI.
- MFA cho System Admin khi chạy Internet thật.
- Rate limit riêng cho admin endpoints.
- Không hiện JDBC URL/password/secret trên UI hoặc log.
- Credential provisioning chỉ ở worker.
- Audit mọi thay đổi lifecycle, placement metadata, capability và retry thủ công.
- Hạn chế actuator/metrics/admin endpoint bằng mạng hoặc authentication phù hợp.
- Có break-glass access nếu cần, nhưng phải giới hạn thời gian và audit.
- Backup và kiểm thử restore control_db.

Microsoft lưu ý Control Plane thường có đặc quyền rất lớn; nếu bị xâm nhập, nhiều tenant có thể cùng bị ảnh hưởng. Tài liệu cũng khuyến nghị tách tài nguyên Control Plane khỏi tenant workloads để giảm lan truyền noisy neighbor và hạn chế leo thang đặc quyền. Xem phần security/isolation trong Control plane considerations.

9. Provisioning nên thực hiện thế nào?

```
None
VALIDATE
  |
  ▼
RESERVE_ROUTE
  |
  ▼
SELECT/CREATE_PLACEMENT_RESOURCE
  |
  ▼
APPLY_MIGRATIONS
  |
  ▼
SEED_TENANT
  |
  ▼
HEALTH_CHECK
  |
  ▼
ACTIVATE_ROUTE
  |
  ▼
```

```
SUCCEEDED
```

Mỗi step cần lưu:

- Job/attempt ID.
- Idempotency key.
- Trạng thái trước/sau.
- Thời gian bắt đầu/kết thúc.
- Resource ID được tạo.
- Error code và thông điệp đã loại bí mật.
- Lần retry tiếp theo.
- Ownership marker để rollback không xóa nhầm tài nguyên.

Nếu một bước lỗi, workflow phải chọn một trong ba cách đã định trước: retry, tiếp tục khi lỗi không chặn, hoặc dừng để người vận hành xử lý. Không giữ HTTP request mở trong toàn bộ quá trình cấp phát. Microsoft cũng coi workflow dài và failure handling là trách nhiệm trọng yếu của Control Plane.

Bridge đã có state machine, job lease, retry và rollback. Vì vậy nên củng cố phần hiện tại thay vì thay bằng Azure Durable Functions chỉ để giống ví dụ Microsoft. Công nghệ Microsoft nêu là ví dụ; yêu cầu cốt lõi là workflow bền vững, quan sát được và xử lý lỗi nhất quán.

10. Placement trong Control Plane

Hiện tại placement được chọn lúc onboarding và bất biến sau khi tenant có dữ liệu. Control Plane lưu quyết định đó và resolver sử dụng nó để chọn datasource.

Giai đoạn này không nên quảng bá rằng hệ thống tự chuyển tenant từ Pool sang Silo theo usage. Chức năng đó cần:

- Snapshot và khóa mutation.
- Copy dữ liệu/file/event an toàn.
- Kiểm tra checksum/count.
- Chuyển route nguyên tử.
- Rollback.
- Downtime hoặc dual-write policy.
- Backup và audit.

Đây là một đề tài phức tạp riêng và đang ngoài phạm vi phiên bản hiện tại.

Sau khi có dữ liệu vận hành, Control Plane có thể **gợi ý placement** dựa trên:

- Mức nhạy cảm/cách ly yêu cầu.
- Số thành viên và tải dự kiến.
- Dung lượng dữ liệu.
- Connection budget.
- Yêu cầu backup/restore.
- Chi phí vận hành.
- Capacity của database hoặc deployment stamp.

Không nên tự động quyết định bằng một công thức chưa được đo.

11. Độ tin cậy và cách ly

Control Plane hiện tách control_db, module và quyền provisioner, nhưng vẫn dùng chung codebase và phần hạ tầng với Data Plane. Điều này phù hợp giai đoạn nghiên cứu/local nhưng tạo các câu hỏi cần kiểm chứng:

- Control DB lỗi thì tenant đang đăng nhập có tiếp tục dùng Data Plane không?
- API/worker bị quá tải có làm cả ba placement chậm không?
- Một Silo DB lỗi có thực sự chỉ ảnh hưởng tenant đó không?
- MinIO dùng chung lỗi ảnh hưởng những tenant nào?
- Worker dừng thì provisioning/reminder/cleanup khôi phục ra sao?
- Backup control_db và application data có khôi phục nhất quán không?

Khi đưa lên staging, nên tăng cách ly từng bước:

- control_db tách khỏi application database.
- API principal không có quyền provisioning.
- Worker principal có quyền theo adapter và tối thiểu cần thiết.
- Admin host/API được bảo vệ riêng.
- Metric/actuator không công khai ngoài ý muốn.
- Nếu dữ liệu tải chứng minh cần thiết, tách deployment control worker/API khỏi tenant-serving API.

Chưa cần Kubernetes, Redis hay nhiều microservice nếu chưa có yêu cầu tải và vận hành cụ thể.

12. Lộ trình triển khai đề xuất

Giai đoạn 1 — Chuẩn hóa phần đang có

- Ghi ADR ranh giới Control Plane/Data Plane.
- Lập inventory bảng, API, job và credential thuộc từng plane.
- Chuẩn hóa tên control_db, System Admin API, provisioning worker.
- Rà audit và authorization toàn bộ /admin/*.

Đầu ra: sơ đồ, ADR, permission matrix và test matrix được nhóm duyệt.

Giai đoạn 2 — Tenant lifecycle

- Suspend/reactivate.
- Session revoke và worker guard.
- Offboarding/archive/purge.
- Retention policy.
- Trạng thái và lỗi hiển thị rõ trên portal.

Đầu ra: state machine, API/OpenAPI, migration, UI, audit và test retry/idempotency.

Giai đoạn 3 — Quan sát và usage

- Bổ sung metric worker.
- Daily usage aggregate theo tenant.
- Dashboard queue/provisioning/capacity.
- Alert có người nhận và runbook.
- Định nghĩa SLO, RTO và RPO.

Đầu ra: dashboard/alert, dữ liệu thô, định nghĩa metric và retention.

Giai đoạn 4 — Staging và recovery

- Domain/TLS và secret production-like.
- Provider sandbox.
- Backup/restore drill.
- Fault injection control DB, worker, Pool/Schema/Silo và storage.
- Security matrix tenant A/B và System Admin.

Đầu ra: biên bản staging/pilot và bằng chứng recovery, chưa gọi là production nếu còn provider giả.

Giai đoạn 5 — Chức năng nâng cao khi có bằng chứng

- Capacity-aware placement.
- Deployment stamp.
- Tự động mở rộng resource pool.
- Tenant relocation/rebalancing.
- Multi-region hoặc nhiều Control Plane.

Microsoft khuyến nghị giảm số lượng Control Plane vì phối hợp nhiều Control Plane làm tăng phức tạp. Với quy mô Bridge hiện tại, một Control Plane là đủ.

13. Tiêu chí xem Control Plane v1 là hoàn thành

Control Plane v1 có thể xem là hoàn thành khi:

- Tenant catalog là nguồn sự thật duy nhất cho route/placement/lifecycle.
- Onboarding, retry, suspend/reactivate và offboarding có state machine rõ.
- Mọi thao tác System Admin được backend kiểm quyền và audit.
- API không nắm credential provisioning.
- Worker xử lý retry/crash mà không tạo resource trùng.
- Metric API và worker đủ để phát hiện job kẹt, lỗi và connection saturation.
- Có usage tối thiểu theo tenant với định nghĩa rõ.
- Backup/restore control_db và application data đã được diễn tập.
- Cross-tenant tests đạt zero unauthorized read/write/download/delivery.
- Có SLO/RTO/RPO được nhóm phê duyệt sau pilot.
- Domain/TLS, secret và provider được kiểm chứng trong staging.

14. Nội dung có thể báo cáo với nhóm

Có thể trình bày ngắn như sau:

Bridge hiện đã có Control Plane cơ bản chứ không phải hoàn toàn chưa có. control_db đóng vai trò tenant catalog; System Admin API/UI quản lý tenant; provisioning worker tạo và duy trì placement. Tuy nhiên, Control Plane này mới được kiểm chứng local và còn thiếu vòng đời offboarding, metering đầy đủ, metric worker, recovery objective và cách ly production. Nhóm nên phát triển dần kiến trúc hiện tại theo hướng Custom Control Plane, ưu tiên tenant lifecycle, observability, bảo mật quản trị và backup/restore trước khi nghiên cứu placement tự động hoặc nhiều deployment stamp.

15. Nguồn tham khảo

- Microsoft Azure Architecture Center, Considerations for multitenant control planes, truy cập ngày 30/09/2026.
- Microsoft Azure Architecture Center, Architectural approaches for control planes in multitenant solutions, truy cập ngày 30/09/2026.
- Kiến trúc Bridge hiện tại và cách đưa vào vận hành.
- Tổng quan kiến trúc Bridge.

Trạng thái triển khai dự án.

Mở đầu: Tổng quan tình hình nghiên cứu thuộc lĩnh vực đề tài ở trong và ngoài nước, lý do chọn đề tài, mục tiêu đề tài, phương pháp nghiên cứu, đối tượng và phạm vi nghiên cứu;

Chương 1: Mở đầu
Tổng quan tình hình nghiên cứu (Thêm tình hình thực tế từ Base.vn, Salesforce, Azure Whitepapers).

Lý do, Mục tiêu, Phạm vi, Đối tượng. Giống thuyết minh

Phương pháp nghiên cứu: Nêu rõ ở đây (Phương pháp phân tích lý thuyết, Phương pháp thực hiện, Phương pháp thực nghiệm,...)

Kết quả nghiên cứu và phân tích (bàn luận) kết quả: Trình bày thành các chương 1, 2, 3,...; nêu các kết quả nghiên cứu đạt được và đánh giá về các kết quả này

Chương 2: Cơ sở lý thuyết
Đi từ Khái niệm MT. Phân tích và chỉ ra sự khác biệt của nó. Dẫn dắt đến vì sao cách trình bày mô hình dữ liệu là tối quan trọng

Các mô hình dữ liệu trong MT: Trình bày lý thuyết 3 mô hình (Silo - Separate DB, Bridge - Separate Schema, Pool - Shared Schema).

Các thách thức kinh điển của SaaS MT: Identity, Cô lập, Noisy Neighbor, Blast Radius, Recovery, Update, Measure Usage, Pricing Model

Tech Stack

Chương 3: Pttk kiến trúc
Đánh giá và Lựa chọn mô hình dữ liệu, So sánh, vẽ sơ đồ,... Theo bài toán của mình

Kiến trúc, giải pháp cho các thách thức nêu trên

Kiến trúc tổng thể

Thiết kế Nghiệp vụ

Chương 4: Các kỹ thuật xây dựng nên kiến trúc
Các kỹ thuật giải quyết các thách thức, giải pháp được lựa chọn

Các lỗ hỏng bảo mật được nhận ra và đã vá trong quá trình thực hiện (Đến từ các cảnh báo của Azure, AWS, GCP và một số bài báo khoa học)

Chương 5: Thực nghiệm, đánh giá và bàn luận kết quả

Môi trường và cách thức thực nghiệm

Đánh giá về những tiêu chí của MT như Noisy neighbor, tốc độ triển khai - quy trình một tenant mới được tạo ra, bảo mật,...

Bàn luận

Chương 6: Kết luận và kiến nghị (chưa làm rõ, dựa trên mấy phần trên rồi làm rõ)
Phần kết luận: Kết luận về các nội dung nghiên cứu đã thực hiện. Đánh giá những đóng góp mới của đề tài và khả năng ứng dụng của kết quả nghiên cứu.

Phần kiến nghị: Các đề xuất được rút ra từ kết quả nghiên cứu. Đề xuất về các nghiên cứu tiếp theo; các biện pháp cần thiết để có thể ứng dụng KQ nghiên cứu vào thực tiễn đời sống và sản xuất; các lĩnh vực nên ứng dụng hay sử dụng kết quả nghiên cứu; các kiến nghị về cơ chế, chính sách.

Tài liệu tham khảo

Phụ lục

Các minh chứng sản phẩm cho: Ứng dụng web quản lý công việc đa thuê bao, Bản tin, Báo cáo tóm tắt, video 2 phút

kiến trúc multi-tenant trong link youtube

1. Báo cáo này trả lời điều gì?

Sau khi đọc, người mới có thể trả lời được năm câu hỏi:

Multi-tenant là gì và tenant trong video là ai?

Một yêu cầu đi từ subdomain đến đúng dữ liệu tenant như thế nào?

Dữ liệu giữa các tenant được cô lập bằng cách nào?

Video có đo mức sử dụng của tenant không?

Bridge nên học gì, giữ gì và phát triển tiếp phần nào?

2. Kết luận trước khi đi vào chi tiết

Khóa học xây một marketplace giống Gumroad, nơi nhiều người bán sử dụng chung một hệ thống:

Mỗi người bán có một gian hàng và subdomain riêng.

Tất cả người bán dùng chung ứng dụng, máy chủ và MongoDB.

Mỗi sản phẩm hoặc tệp được đánh dấu thuộc tenant nào.

Khi đọc dữ liệu, hệ thống phải lọc đúng tenant.

Stripe Connect nhận tiền cho từng người bán và chuyển phí nền tảng cho chủ ứng dụng.

Kiến trúc này là **POOL**, nghĩa là nhiều tenant dùng chung database và collection. Sự tách biệt chủ yếu do phần mềm thực hiện bằng trường tenant, access rule và điều kiện truy vấn.

Video **không xây hệ thống đo mức sử dụng theo tenant**. Video có dữ liệu đơn hàng và phí giao dịch, nhưng không đo số request, dung lượng lưu trữ, CPU, RAM hoặc transaction để tính quota hay phục vụ nghiên cứu.

Bridge không nên sao chép công nghệ của khóa học. Bridge đã có nền kiến trúc mạnh hơn với TenantContext, PostgreSQL, ba placement, RLS, schema role, datasource resolver, outbox và worker. Phần nên học là **cách tenant context xuất hiện xuyên suốt trải nghiệm;** phần cần phát triển tiếp là **metering, metric worker, kiểm thử storage và triển khai trên môi trường thật. MMARK**

3. Các thuật ngữ cần biết

Thuật ngữ	Giải thích đơn giản	Ví dụ
Tenant context	Thông tin hệ thống đã xác minh về tenant đang xử lý	Tenant ID, slug, placement, role và user hiện tại

Authorization	Kiểm tra người dùng được phép làm gì	Thành viên được xem task nhưng chỉ manager được xóa task
Isolation	Bảo đảm tenant này không đọc hoặc sửa dữ liệu tenant khác	Đổi ID trên URL vẫn không xem được tài liệu của nhóm khác
Placement	Cách bố trí dữ liệu của tenant	Chung bảng, riêng schema hoặc riêng database
Webhook	Thông báo server-to-server từ một dịch vụ ngoài	Stripe báo cho ứng dụng rằng thanh toán đã thành công
Usage metering	Ghi nhận tenant đã sử dụng bao nhiêu tài nguyên hoặc nghiệp vụ	Số request, số task tạo mới, dung lượng file đang dùng
Observability	Dữ liệu giúp vận hành và tìm lỗi	Log, metric, trace, cảnh báo

4. Ví dụ xuyên suốt: hai cửa hàng dùng chung nền tảng

Giả sử nền tảng có hai người bán:

Tenant A: **An Books**, slug an-books.

Tenant B: **Bình Design**, slug binh-design.

Hai cửa hàng có URL:

https://an-books.example.com https://binh-design.example.com

An Books tạo sản phẩm "Giáo trình Java". Bình Design tạo sản phẩm "Bộ icon Kanban". Hai sản phẩm nằm trong cùng collection MongoDB nhưng có tenant khác nhau:

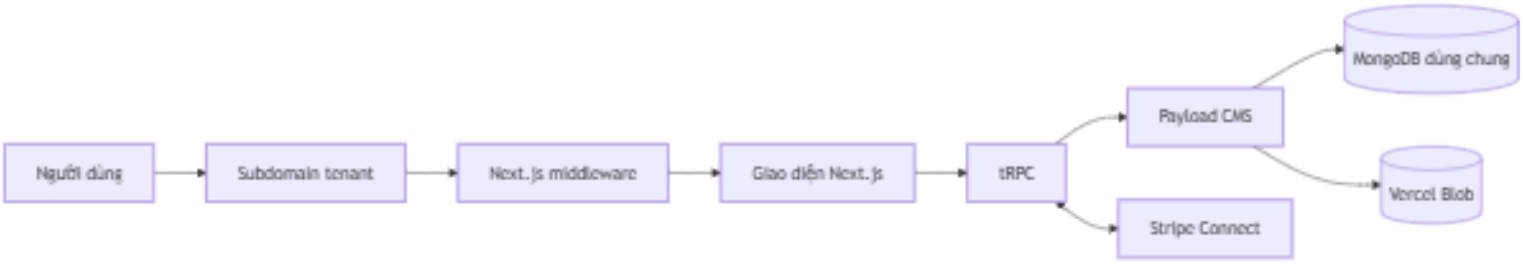
Product 1: name = "Giáo trình Java", tenant = "an-books" Product 2: name = "Bộ icon Kanban", tenant = "binh-design"

Khi người dùng mở an-books.example.com, hệ thống chỉ được trả Product 1. Product 2 vẫn tồn tại trong cùng database nhưng phải bị loại khỏi kết quả.

Đây chính là bản chất của pooled multi-tenancy: **dữ liệu ở chung một nơi nhưng mọi thao tác phải biết dữ liệu thuộc tenant nào**.

5. Kiến trúc của khóa học

5.1. Bức tranh tổng thể



Vai trò từng thành phần:

Thành phần	Làm gì?
Next.js	Chạy giao diện, route, middleware và endpoint webhook
Middleware	Đọc hostname để lấy tenant slug rồi rewrite sang route tenant nội bộ
tRPC	Nhận yêu cầu từ giao diện, kiểm input và gọi nghiệp vụ
Payload CMS	Định nghĩa collection, đăng nhập, quyền và admin dashboard
MongoDB	Lưu user, tenant, product, order, review và metadata media
Vercel Blob	Lưu tệp/ảnh bền vững ngoài filesystem serverless
Stripe Connect	Onboard người bán, tạo checkout và chia dòng tiền

Toàn bộ phần web, API, CMS và webhook nằm trong một ứng dụng Next.js. Có thể gọi đây là một **modular monolith**: một deployment nhưng bên trong chia thành các module nghiệp vụ.

5.2. Một request tìm đúng tenant như thế nào? MMARK

Ví dụ người dùng mở:

https://an-books.example.com/products/java

Luồng xử lý:

DNS/Vercel đưa request về cùng một deployment.

Middleware đọc Host = an-books.example.com.

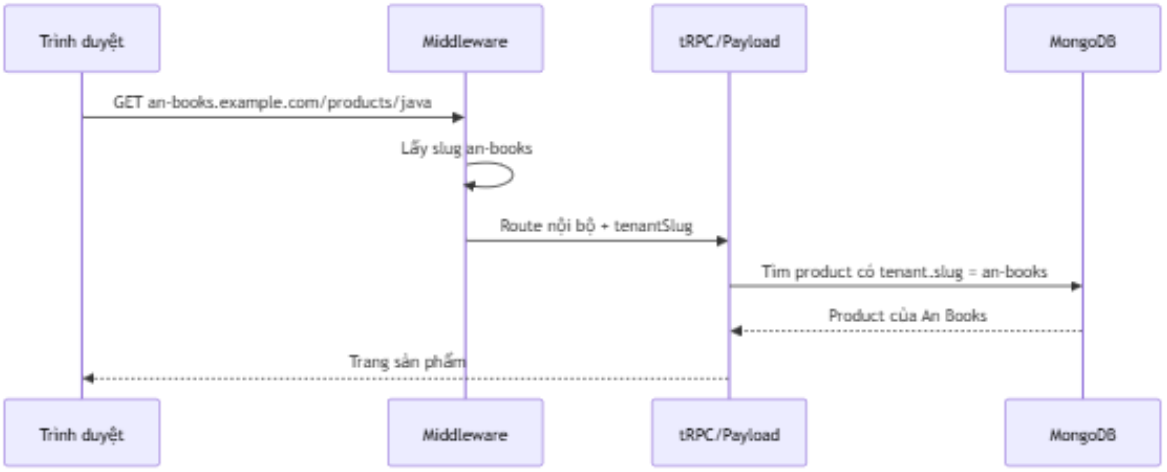
Middleware tách slug an-books.

Request được rewrite nội bộ sang /tenants/an-books/products/java.

Trang hoặc tRPC procedure dùng an-books để query Payload.

Payload query thêm điều kiện product thuộc tenant an-books.

MongoDB trả đúng sản phẩm của An Books.



Điểm quan trọng: subdomain chỉ nói người dùng đang muốn vào cửa hàng nào. Nó không tự bảo vệ dữ liệu. Điều kiện tenant trong query và access rule mới là hàng rào thật sự.

6. Khóa học cô lập tenant như thế nào?

6.1. Cô lập trong admin dashboard

Payload Multi-Tenant Plugin thêm trường tenant vào product và media. Khi merchant đăng nhập:

Merchant chỉ thấy document của tenant mình.

Document mới được gắn tenant đang chọn.

Merchant không được chuyển document sang tenant mà mình không thuộc về.

Super admin có thể xem mọi tenant.

Ví dụ: chủ An Books đăng nhập Payload Admin thì không được nhìn thấy “Bộ icon Kanban” của Bình Design.

6.2. Cô lập trong storefront và API

tRPC procedure phải tự thêm tenant filter khi query. Đây vừa là sức mạnh vừa là điểm yếu:

Đúng filter thì dữ liệu tách biệt.

Quên filter trong một endpoint có thể gây rò dữ liệu.

Ví dụ kiểm thử cần có:

Cho Product B thuộc Bình Design. Gọi API qua host an-books.example.com nhưng truyền Product B. Kết quả đúng: 404 hoặc 403, không trả Product B.

6.3. Cô lập nội dung đã mua

Nội dung số riêng tư không được trả trong product query công khai. Sau thanh toán, webhook tạo Order. **Khi user mở thư viện, hệ thống kiểm tra tồn tại Order của user và product rồi mới trả nội dung bảo vệ**. **MMARK**

Ví dụ:

Lan đã mua "Giáo trình Java": được đọc nội dung trong Library.

Minh chưa mua: dù đoán được product ID vẫn không được nhận protected content.

6.4. Cô lập thanh toán

Mỗi merchant có Stripe connected account. Checkout của An Books được tạo trong account của An Books; checkout của Bình Design nằm trong account của Bình Design.

Cart cũng được chia theo tenant vì một Checkout Session direct-charge không nên trộn sản phẩm của hai người bán.

6.5. Cô lập media chưa hoàn toàn

Metadata media được gắn tenant và được lọc trong CMS. Tuy nhiên, nếu URL tệp công khai và ai có link cũng tải được thì dữ liệu chưa thật sự riêng tư.

Ví dụ kiểm thử production:

Tenant A lấy URL tệp của Tenant B. Tenant A gửi request tải tệp. Kết quả đúng: bị từ chối hoặc signed URL không hợp lệ.

Vì vậy, private media cần kiểm quyền ở thời điểm tải và dùng private object/signed URL có thời hạn.

7. Quyền người dùng trong khóa học

Khóa học chủ yếu có:

super-admin: quản trị toàn nền tảng.

user: người dùng thông thường.

Tenant membership: user được gắn với tenant để làm merchant.

Ví dụ:

Super admin có thể quản lý tenant và xem order/review trong CMS.

Merchant chỉ cập nhật tên, ảnh và sản phẩm của cửa hàng mình.

Buyer đăng nhập để mua, xem library và viết review.

Mô hình này đủ cho tutorial nhưng marketplace thật có thể cần owner, catalog manager, support và finance. **Bridge còn cần thêm project role như manager, member và viewer. Tại sao, Bridge thì thêm role được còn Pool thì không? MMARK**

8. Thanh toán hoạt động ra sao?

8.1. Merchant onboarding

Khi merchant đăng ký, ứng dụng lần lượt:

Kiểm tra username có dùng được làm subdomain.

Tạo Stripe connected account.

Tạo Tenant với slug và Stripe account ID.

Tạo User và liên kết với Tenant.

Cho merchant hoàn tất Stripe onboarding.

Nhận webhook account.updated.

Chỉ cho tạo/bán product khi Stripe đã xác minh.

Ví dụ: An đăng ký username an-books. Tenant an-books được tạo nhưng An chưa hoàn tất Stripe. Hệ thống chưa cho An đăng sản phẩm bán. Sau khi Stripe báo đã hoàn tất, quyền bán mới được mở.

8.2. Buyer checkout

Browser gửi product ID và số lượng. Server không tin giá từ browser mà tải lại product, giá, tenant và trạng thái archive từ database.

Ví dụ: browser sửa giá từ 100.000 đồng thành 1.000 đồng. Server vẫn lấy 100.000 đồng trong database để tạo Checkout Session nên hành vi sửa giá thất bại.

8.3. Webhook tạo quyền truy cập

Sau khi Stripe báo thanh toán thành công, webhook tạo Order. Order làm bằng chứng user được xem nội dung đã mua.

Điểm còn thiếu: webhook phải chống xử lý trùng vì Stripe có thể gửi lại cùng sự kiện. Production cần lưu event ID hoặc unique entitlement để một giao dịch không tạo hai Order.

9. Video đo mức sử dụng tenant ra sao?

9.1. Câu trả lời ngắn

Video không triển khai usage metering theo tenant.

Video có thể biết một tenant có bao nhiêu product hoặc order vì các record tồn tại trong database. Nhưng video không biến chúng thành một hệ đo có thời gian, idempotency, báo cáo và đối soát.

9.2. Vì sao phí Stripe không phải usage metering?

Ví dụ một sản phẩm giá 100 USD và platform lấy 10 USD. Khoản 10 USD là phí trên giao dịch bán hàng.

Nó không trả lời được:

An Books đã gọi bao nhiêu API trong tháng?

An Books đang dùng bao nhiêu MB file?

Tenant nào tạo tải cao khiến tenant khác chậm?

Mỗi placement dùng bao nhiêu connection hoặc RAM?

Do đó:

Application fee ≠ Usage metering ≠ Operational monitoring

9.3. Một hệ đo đúng cần có gì? MMARK

Ví dụ báo cáo tháng của tenant có thể gồm:

Metric	Giá trị minh họa	Loại
API request	12.430	Counter
Task được tạo	315	Transaction counter
Dung lượng file đang dùng	824 MB	Gauge/snapshot
p95 latency	280 ms	Operational distribution

Các số trên chỉ là ví dụ cách trình bày, **không phải kết quả đo của đề tài**.

10. Khóa học làm tốt điều gì và còn thiếu gì?

10.1. Điểm làm tốt

Xây được luồng xuyên suốt MMARK từ merchant đăng ký đến buyer mua hàng.

Tenant xuất hiện trong routing, CMS, product query, cart, payment và media.

Server tải lại giá thay vì tin browser.

Webhook là nguồn xác nhận thanh toán thay vì trang success.

Order bảo vệ nội dung đã mua.

Archive product mà không phá quyền của buyer cũ.

Storefront và branding riêng nhưng dùng chung codebase.

10.2. Khoảng trống production

Không có usage metering.

Chưa thể hiện webhook idempotency đầy đủ.

Onboarding nhiều bước có thể để lại Stripe account hoặc tenant mồ côi khi lỗi giữa chừng.

Tenant filter nằm rải rác trong procedure.

Private media chưa được chứng minh an toàn khi tải.

Chưa có rate limit, audit, metric và cảnh báo đầy đủ. Noisy Neighborr MMARK

Review API chưa chứng minh người viết đã mua product. Vậy review API như thế nào mới đủ? MMARK

Chưa có refund, dispute và reconciliation hoàn chỉnh.

Chưa có ma trận kiểm thử truy cập chéo tự động cho toàn hệ thống. Cần làm rõ cái này MMARK

Vì vậy, sản phẩm phù hợp để học và demo MVP, chưa đủ bằng chứng để gọi production-ready.

11. Chuyển bài học sang Bridge

11.1. Tenant của Bridge khác tenant trong video

Trong video:

Tenant = cửa hàng/người bán

Trong Bridge:

Tenant = workspace/tổ chức cộng tác

Ví dụ Bridge có: **CHECKK**

Tenant A: Nhóm nghiên cứu SaaS.

Tenant B: Lớp Kỹ thuật phần mềm.

Nhóm nghiên cứu SaaS có project "Thực nghiệm multi-tenancy". Lớp Kỹ thuật phần mềm có project "Bài tập lớn". Hai nhóm có thể dùng chung ứng dụng nhưng không được nhìn thấy task, file, notification hoặc metric riêng của nhau.

11.2. Ba placement của Bridge qua ví dụ đời thường

Placement	Hình dung đơn giản	Cách dữ liệu được tách
POOL	Nhiều nhóm dùng chung một phòng, mọi hộp đều có nhãn chủ sở hữu	Chung database/schema/table; mọi row có `tenant_id`, có RLS bảo vệ
SCHEMA_PER_TENANT	Cùng một tòa nhà nhưng mỗi nhóm có phòng khóa riêng	Chung database, mỗi tenant có schema và runtime role riêng
SILO_DATABASE	Mỗi nhóm có một căn nhà riêng	Mỗi tenant có database và runtime role riêng

Ví dụ tenant A cố đọc task tenant B:

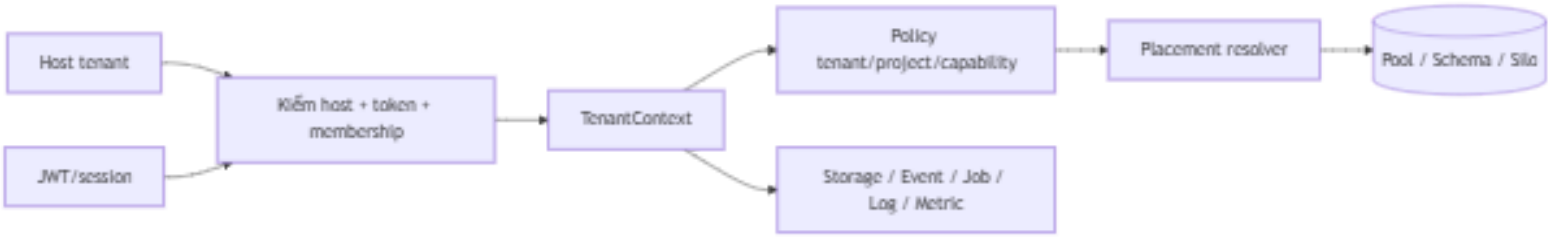
POOL: tenant predicate và RLS phải loại task B.

SCHEMA: role của A không có quyền vào schema B.

SILO: datasource của A kết nối database A, không phải database B.

Bridge nghiên cứu cả ba cách để so sánh. Video chỉ minh họa cách thứ nhất.

11.3. Luồng request an toàn của Bridge



Ví dụ user thuộc Tenant A nhưng sửa host thành Tenant B:

Host nói Tenant B.

Token nói Tenant A.

Hệ thống phát hiện không khớp.

Request bị từ chối trước khi chạy business query.

Đây là điểm Bridge mạnh hơn cách truyền tenant slug đơn thuần trong khóa học.

12. Bridge hiện có gì?

Theo checkpoint local hiện tại, Bridge đã có nền tảng:

TenantContext chứa user, tenant, tier, placement, role và correlation ID.

So khớp host, token, tenant active và membership hiện hành.

Ba placement: POOL, SCHEMA_PER_TENANT và SILO_DATABASE.

RLS cho POOL; schema/database và runtime role cho các placement còn lại.

Provisioning có checkpoint, retry và rollback.

Outbox/job và worker khôi phục tenant context.

Resource namespace theo tenant, quota và signed download flow.

Role ở cấp system, tenant và project.

Metric HTTP có tenant, tier và placement; Prometheus scrape API.

Bộ công cụ chạy thí nghiệm, lưu manifest, raw metric và resource metric.

Những điều trên là kết quả triển khai/kiểm thử local. Provider thanh toán thật, wildcard domain/TLS thật, VPS, pilot và kết quả nghiên cứu vẫn chưa được phép trình bày là đã hoàn tất.

13. Bridge còn thiếu gì quan trọng?

13.1. Usage metering bền vững

Bridge đã có metric và quota nhưng chưa có usage ledger/rollup thống nhất theo thời gian.

Thiết kế hướng tới:

tenant_usage_event

- event_id tenant_id
- metric_code

- quantity
- source_event_id
- occurred_at
- placement

tenant_usage_daily

- tenant_id
- usage_date
- metric_code
- quantity_sum
- quantity_max
- sample_count

Ví dụ task được tạo bởi một request bị retry hai lần. Cả hai lần dùng cùng source_event_id, nên hệ thống chỉ cộng TASK_CREATED = 1, không cộng thành 2.

13.2. Kiểm soát metric cardinality

Bridge đang gắn tenant_id vào HTTP metric. Với 3–5 tenant trong pilot, cách này hữu ích. Nếu có hàng nghìn tenant, số chuỗi Prometheus có thể tăng rất lớn vì kết hợp:

tenant × route × method × status × placement

Giải pháp là cấu hình chế độ nghiên cứu/production, đặt series budget và ưu tiên tổng hợp theo tier/placement. Báo cáo usage cho từng tenant nên đọc từ rollup bền vững thay vì phụ thuộc trực tiếp vào Prometheus.

13.3. Metric cho worker

Prometheus hiện scrape API nhưng worker chưa có endpoint metric riêng. Cần đo:

Outbox lag.

Số lần retry.

Delivery success/failure.

Provisioning duration.

Số job bị kẹt hoặc chuyển manual intervention.

Ví dụ API vẫn phản hồi nhanh nhưng hàng đợi email đã trễ 30 phút. Nếu chỉ đo API, nhóm sẽ tưởng hệ thống khỏe trong khi notification đang hỏng.

13.4. Kiểm chứng môi trường thật

Cần kiểm chứng thêm:

Wildcard DNS và TLS.

Reverse proxy giữ đúng host.

Cookie, CORS và CSRF giữa accounts host và tenant host.

MinIO/object storage thật và tải file chéo tenant.

Payment provider sandbox, webhook replay và reconciliation.

Cùng workload trên ba placement.

14. Những gì nên học theo, điều chỉnh hoặc loại bỏ

Ý tưởng từ video	Quyết định cho Bridge	Ví dụ áp dụng
Tenant subdomain	Học theo	nhom-saas.example.com mở đúng workspace
Storefront riêng	Điều chỉnh	Dùng branding, workspace shell và module riêng
Multi-tenant plugin	Học nguyên lý, không dùng framework	TenantContext + policy + RLS/schema/database boundary
Search/filter URL state	Học theo	Lọc task theo status, assignee, deadline và chia sẻ URL
Server kiểm lại giá	Học theo	Server quyết định tier, giá và placement surcharge
Private media	Học theo và làm chặt hơn	Kiểm membership/project trước khi cấp signed URL
Webhook thanh toán	Học state/idempotency	Callback thật mới cho provisioning tiếp tục
Stripe Connect	Không áp dụng hiện tại	Bridge không chia tiền cho người bán
Product/review/cart	Không áp dụng	Không phù hợp nghiệp vụ project/task
Next.js/Payload/MongoDB	Không đổi theo	Giữ Spring Boot/PostgreSQL/React/MUI

15. Thứ tự công việc đề xuất

Giai đoạn 1 — khóa nền an toàn

Hoàn thiện ma trận test truy cập chéo cho API, database, storage, job và notification.

Chạy cùng contract trên cả ba placement.

Ghi rõ invariant nào do application, RLS, role hoặc database đảm nhiệm.

Kết quả mong đợi: đổi host, token, path ID, query hoặc storage key đều không vượt được tenant boundary.

Giai đoạn 2 — hoàn thiện đo lường

Xây usage event/daily rollup.

Thu request, active storage và transaction metric.

Thêm metric worker/outbox/provisioning.

Kiểm soát Prometheus cardinality.

Kết quả mong đợi: nhóm trả lời được tenant dùng bao nhiêu, hệ thống đang khỏe không và số liệu nào dùng cho nghiên cứu.

Giai đoạn 3 — kiểm chứng tích hợp thật

Wildcard domain/TLS/proxy.

MinIO/private resource.

Payment sandbox và webhook reconciliation.

Runbook backup, restore và failure recovery.

Kết quả mong đợi: luồng demo không còn phụ thuộc localhost hoặc provider giả.

Giai đoạn 4 — chạy nghiên cứu

Khóa workload và môi trường.

Chạy pilot trước.

Chạy lặp trên POOL, SCHEMA_PER_TENANT và SILO_DATABASE.

Lưu manifest, raw metrics, resource metrics và QA report.

Kết quả mong đợi: kết luận dựa trên dữ liệu thật, không dựa trên cảm giác mô hình nào "tốt nhất".

16. Mười điều cần nhớ

Multi-tenant không chỉ là thêm tenant_id vào bảng.

Subdomain xác định ngữ cảnh **nhưng không thay thế authorization. MMARK**

Tenant context phải đến từ nguồn đã xác minh.

Mọi query, file, event, job, log và metric đều phải giữ tenant boundary.

UI ẩn nút không phải bảo mật; backend vẫn phải từ chối request trái quyền.

Video chỉ minh họa pooled multi-tenancy.

Stripe application fee không phải usage metering.

Bridge đã mạnh hơn video về isolation và placement nhưng metering còn cần hoàn thiện.

Không có kiểm thử tenant A chống tenant B thì chưa có bằng chứng cô lập.

Chỉ gọi production-ready sau khi DNS/TLS, provider, storage, retry, recovery và vận hành thật đã được kiểm chứng.

17. Kết luận cuối cùng

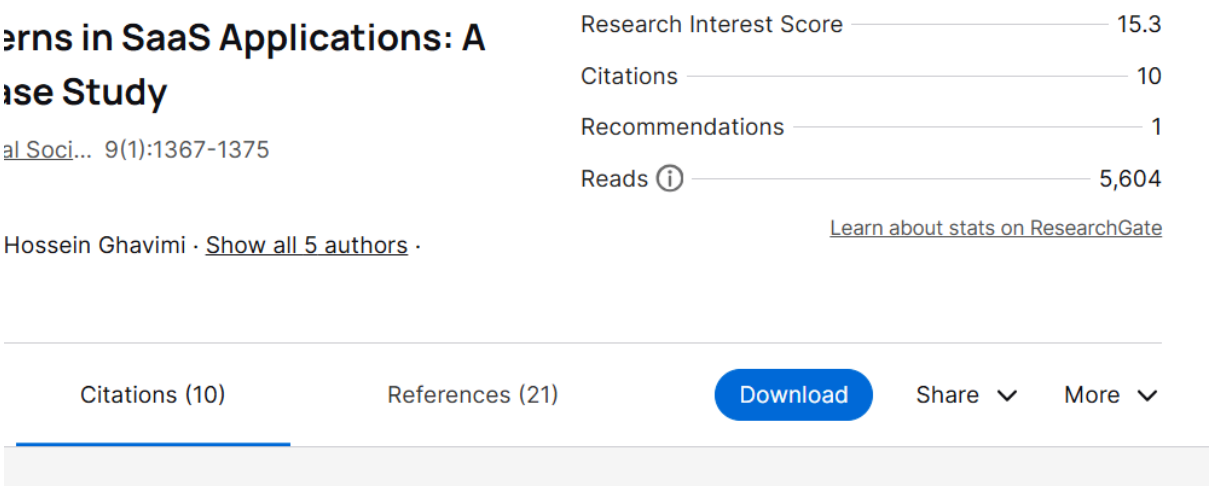
Khóa học là một ví dụ thực hành tốt vì cho thấy tenant không chỉ xuất hiện trong database mà còn đi qua subdomain, giao diện, admin, product query, cart, payment, library và media. Tuy nhiên, kiến trúc vẫn là pooled multi-tenancy dựa nhiều vào plugin và filter ở application; chưa có usage metering và chưa đủ hardening cho production.

Bridge nên học cách tổ chức trải nghiệm tenant xuyên suốt nhưng giữ kiến trúc hiện tại. Ba placement, TenantContext, RLS/schema role/database resolver, outbox và provisioning là nền nghiên cứu quan trọng hơn việc sao chép Next.js hoặc Payload. Bước phát triển có giá trị nhất tiếp theo là hoàn thiện metering, metric worker và kiểm thử truy cập chéo toàn diện, sau đó mới triển khai tích hợp thật và chạy thực nghiệm.

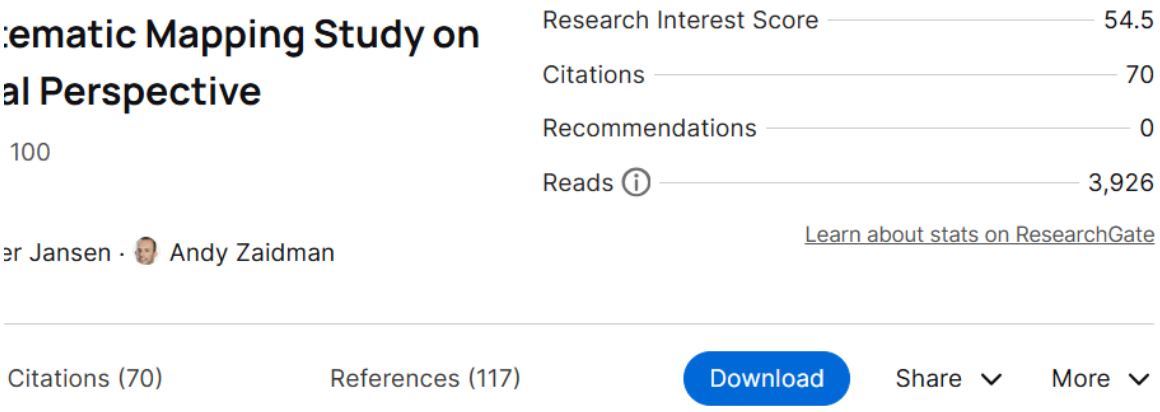
Defining Multi-Tenancy: A Systematic Mapping Study on the Academic and the Industrial Perspective

https://www.sciencedirect.com/science/article/pii/S1319157823002252

[1]
https://www.researchgate.net/publication/338216590_Multi-tenancy_Design_Patterns_in_SaaS_Applications_A_Performance_Evaluation_Case_Study -



[2]
Defining Multi-Tenancy: A Systematic Mapping Study on the Academic and the Industrial Perspective



MU và MT

This might range from tenants sharing the same user interface components, business logic right down to data layer, however each tenant only access functions or components and data belonging to their business entity. In contrast to multi-user model (traditional), where an application is designed and developed with the same functionalities but with limited configuration and hosted in a single instance. The advantage of multi-tenant application is its ability to support multi-faceted functionalities which maybe similar or varying and has the capability to scale horizontally.

Research on the Construction and Operation Optimization of E-commerce Multi-tenant Database Service Platform in Cloud-native Environment-Hill Publishing Group

After multi-tenant parallel access, resource contention no longer remains at the CPU and memory levels, but extends to connection pools, cache hits, disk I/O, and replication link stability. Once the isolation granularity is incorrect, the instantaneous fluctuations of a single tenant may spill over to adjacent instances. At the same time, the platform must also take into account permission boundaries, configuration con sistency, version upgrades, and fault recovery links for unified governance

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN TRÚC ĐA THUÊ BAO

Mã số đề tài: THS2026-67

Chủ nhiệm đề tài: Sinh viên Ngô Hưng Thịnh

Cần Thơ, tháng 11/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN TRÚC ĐA THUÊ BAO

Mã số đề tài: THS2026-67

Sinh viên chủ nhiệm đề tài:	Ngô Hưng Thịnh
Giới tính:	Nam
Dân tộc:	Kinh
Lớp:	DI2396F2
Khoa/ Trường /Viện:	Trường CNTT & TT
Năm thứ:	4/ 4.5
Ngành học:	Kỹ thuật Phần mềm CLC

Người hướng dẫn: TS. Ngô Bá Hùng

Cần Thơ, tháng 11/2026

Đề tài: Xây dựng ứng dụng quản lý công việc theo kiến trúc đa thuê bao

Giảng viên hướng dẫn: TS. Ngô Bá Hùng

Chủ nhiệm đề tài: Ngô Hưng Thịnh

Sinh viên thực hiện: Nguyễn Thanh Trọng

Trang Minh Chánh

Huỳnh Hồng Ân

Cao Tường Hưng

THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI

Thông tin chung

Mã số đề tài:

Tên đề tài:

Sinh viên chủ nhiệm đề tài:

Lớp:

Trường/ Khoa/ Viện:

Năm thứ:

Số năm đào tạo:

Người hướng dẫn:

Mục tiêu đề tài

Tính mới và sáng tạo

Kết quả nghiên cứu

Sản phẩm

Đóng góp

Hiệu quả, phương thức chuyển giao

TÓM TẮT

ABSTRACT

TÀI LIỆU THAM KHẢO

PHỤ LỤC

Nghiên cứu Kiến trúc Đa thuê bao cho Ứng dụng Quản lý Công việc: Phân tích Đa tầng và Mô hình Tối ưu

Bản chất Ranh giới Thuê bao và Thách thức Nghiệp vụ Quản lý Công việc

Kiến trúc đa thuê bao (Multi-tenancy) là mô hình cốt lõi của phần mềm dạng dịch vụ (SaaS), cho phép một phiên bản ứng dụng duy nhất vận hành trên hạ tầng dùng chung để phục vụ đồng thời nhiều tổ chức khách hàng (tenant) độc lập[1]. Việc xây dựng hệ thống đa thuê bao đòi hỏi sự phân định rạch ròi giữa hệ thống đa người dùng (Multi-user) và hệ thống đa thuê bao doanh nghiệp (Multi-tenant B2B)[2]. Trong các ứng dụng đa người dùng truyền thống, mọi tài khoản đều tồn tại trong một không gian quản trị chung; các cơ chế như kiểm soát truy cập dựa trên vai trò (RBAC) hoặc thuộc tính (ABAC) chỉ đóng vai trò phân định quyền hạn thao tác giữa các cá nhân[2]. Ngược lại, trong mô hình đa thuê bao, tenant là ranh giới quản trị, pháp lý, dữ liệu và vận hành cao nhất[1]. Mọi thực thể người dùng, dự án, luồng công việc, hạn ngạch tài nguyên và chu kỳ thanh toán đều bị ràng buộc chặt chẽ vào ngữ cảnh của tổ chức mà họ trực thuộc[2]. Một người dùng có thể tham gia nhiều tổ chức với các vai trò độc lập, nhưng các phiên làm việc và dữ liệu giữa các tổ chức tuyệt đối không được phép giao thoa[2].

Ứng dụng quản lý công việc và dự án, đặc biệt theo mô hình Kanban cộng tác thời gian thực, đặt ra những thách thức kiến trúc chuyên biệt so với các hệ thống thương mại điện tử hay xử lý giao dịch thông thường:
Đặc thù đầu tiên là tính tương tác cao và độ nhạy thời gian thực[3]. Trạng thái bảng công việc thay đổi liên tục thông qua các thao tác kéo thả thẻ, cập nhật tiến độ, gán nhãn, trao đổi bình luận và phê duyệt nhiệm vụ[3]. Hệ thống đòi hỏi cơ chế kiểm soát đồng thời tối ưu nhằm ngăn chặn triệt để hiện tượng mất dữ liệu khi cập nhật đồng thời (lost updates), đồng thời phải duy trì độ trễ xử lý cực thấp trên các kết nối API và mạng phân phối thông điệp[3].
Đặc thù thứ hai là nhu cầu tùy biến sâu sắc về mô hình dữ liệu và quy trình nghiệp vụ[2]. Mỗi tổ chức vận hành công việc theo một phương thức riêng biệt: nhóm kỹ thuật phần mềm cần các trường dữ liệu như Điểm câu chuyện (Story Points) hoặc Mức độ nghiêm trọng (Severity) với quy trình phát triển nhiều bước; trong khi nhóm Marketing cần quản lý Ngân sách, Kênh truyền thông

và phê duyệt kinh phí qua nhiều cấp quản lý[2]. Hệ thống SaaS phải đáp ứng năng lực tùy biến linh hoạt này mà không làm phân mảnh mã nguồn hoặc làm gián đoạn tính toàn vẹn của cấu trúc lưu trữ chung[3].
Đặc thù thứ ba là nguy cơ tranh chấp tài nguyên từ hiện tượng Noisy Neighbor[3]. Một tổ chức thực hiện các tác vụ nặng như xuất báo cáo tổng kết, lọc tìm kiếm trên tập dữ liệu lớn hoặc đồng bộ dữ liệu qua API có thể nhanh chóng làm cạn kiệt năng lực xử lý của CPU, bộ nhớ và hồ chứa kết nối cơ sở dữ liệu (connection pool), đe dọa trực tiếp đến chất lượng dịch vụ (QoS) của các tổ chức khác đang hoạt động trên cùng hạ tầng[3].
Đặc thù cuối cùng là yêu cầu cô lập an toàn nghiêm ngặt và giới hạn phạm vi ảnh hưởng của sự cố (Blast Radius)[1]. Kế hoạch chiến lược, tài liệu đính kèm và tiến độ dự án là các tài sản thông tin nhạy cảm của doanh nghiệp[5]. Bất kỳ sai sót nào dẫn đến việc rò rỉ dữ liệu giữa các tổ chức hoặc sự cố hạ tầng làm tê liệt toàn bộ hệ thống khách hàng đều gây ra tổn hại nghiêm trọng về mặt bảo mật và uy tín dịch vụ[3].

Tài liệu chuẩn hóa từ Amazon Web Services (AWS Well-Architected SaaS Lens), Microsoft Azure Architecture Center và tài liệu bảo mật chính thức của Kubernetes đều nhấn mạnh rằng không tồn tại một kiến trúc mẫu vạn năng[1]. Thay vào đó, kiến trúc tối ưu cho một bài toán cụ thể phải được xây dựng từ việc phân tích, đánh giá có hệ thống các đánh đổi kỹ thuật ở từng tầng hạ tầng và ứng dụng[2].

Khảo sát và Phân tích Kiến trúc Chi tiết tại Từng Tầng Hệ thống

Kiến trúc đa thuê bao cần được phân rã thành ba tầng chức năng chính của mô hình ứng dụng (Trình diễn, Điện toán, Dữ liệu) kết hợp chặt chẽ với tầng Quản trị và Vận hành xuyên suốt[3].

Tầng Trình diễn và Định tuyến (Presentation and Routing Tier)

Tầng trình diễn chịu trách nhiệm tương tác trực diện với người dùng, phân phối mã giao diện và thực hiện bước đầu tiên của cơ chế định danh thuê bao (Tenant Resolution)[3].

Phương án 1: Silo Frontend (Giao diện Độc lập theo Thuê bao)

Silo Frontend là mô hình trong đó mỗi tenant được cấp phát một bản triển khai giao diện hoàn toàn độc lập, vận hành trên các máy chủ web hoặc container riêng biệt, gắn với tên miền độc lập hoặc các bản build mã nguồn tùy biến cho từng khách hàng[6].
Ưu điểm nổi bật của mô hình này là khả năng tùy biến giao diện đạt mức tối

đa[6]. Đội ngũ phát triển có thể can thiệp sâu vào cấu trúc HTML, biểu bảng kiểu dáng (CSS) hoặc nhúng trực tiếp các đoạn mã JavaScript đặc thù cho từng tổ chức mà không ảnh hưởng đến các đối tượng khác[6]. Ranh giới sự cố giao diện được cô lập hoàn toàn; lỗi sập giao diện của một tenant không tác động đến người dùng khác[6].

Tuy nhiên, nhược điểm chí tử của Silo Frontend là làm bùng nổ chi phí hạ tầng và vận hành[3]. Quy trình tích hợp và phân phối liên tục (CI/CD) trở nên cực kỳ phân mảnh khi một bản vá giao diện nhỏ đòi hỏi phải kích hoạt hàng trăm đường ống triển khai[6]. Mô hình này làm mất đi lợi thế cốt lõi của SaaS là tính chuẩn hóa và khả năng bàn giao liên tục trên một phiên bản phần mềm duy nhất[3].

Phương án 2: Pooled Single-Page Application (SPA) Điều khiển bằng Siêu dữ liệu (Metadata-Driven UI)

Pooled SPA là mô hình chuẩn hóa trong đó toàn bộ hệ sinh thái khách hàng chia sẻ duy nhất một bản đóng gói mã nguồn giao diện (ví dụ: ứng dụng React SPA) được phân phối thông qua mạng phân phối nội dung (CDN)[2]. Khi người dùng truy cập qua tên miền con đại diện ({tenant}.taskflow.vn), lớp định tuyến biên hoặc bộ định tuyến máy khách sẽ trích xuất định danh tổ chức để truy vấn API lấy tệp siêu dữ liệu cấu hình[3]. Giao diện sau đó tự động điều chỉnh bảng màu nhận diện, biểu trưng (logo), cấu trúc các cột Kanban và bố cục các trường dữ liệu trên biểu mẫu công việc hoàn toàn dựa trên siêu dữ liệu đã tải[3].

Ưu điểm của mô hình là tối ưu hóa chi phí vận hành và tài nguyên lưu trữ biên[2]. Bản build tĩnh được phân phối hiệu quả qua CDN với chi phí mở rộng biên tế tiệm cận bằng không[6]. Đội ngũ kỹ thuật chỉ duy trì một quy trình CI/CD duy nhất, đảm bảo tính năng mới và các bản sửa lỗi bảo mật được áp dụng đồng thời cho 100% người dùng mà không tạo ra thời gian chết (zero-downtime deployment)[2].

Nhược điểm của phương án này nằm ở độ phức tạp trong việc thiết kế kiến trúc giao diện máy khách[6]. Ứng dụng phải xây dựng một bộ máy phân giải bố cục linh hoạt để kết xuất giao diện dựa trên định nghĩa dữ liệu[3]. Đồng thời, hệ thống đòi hỏi cơ chế làm sạch dữ liệu nghiêm ngặt để ngăn chặn triệt để các lỗ hổng chèn mã độc liên trang (XSS) khi cho phép quản trị viên tùy biến các nhãn hoặc cấu hình giao diện[6].

Tiêu chí Đánh giá	Silo Frontend	Pooled SPA kết hợp Metadata-Driven UI

Quy trình Phát hành CI/CD	Phân mảnh; phải biên dịch và triển khai N lần tương ứng N tenant[6]	Tối ưu; phát hành duy nhất một bản đóng gói cho toàn hệ thống[2]
Chi phí Lưu trữ & Phân phối	Tốn kém tài nguyên máy chủ web và cấu hình chứng chỉ mạng riêng[2]	Cực kỳ thấp; tận dụng tối đa năng lực bộ nhớ đệm CDN toàn cầu[2]
Tính Chuẩn hóa SaaS	Kém; dễ suy biến thành việc duy trì nhiều nhánh mã nguồn riêng lẻ[6]	Đạt chuẩn; đảm bảo nguyên tắc một phiên bản sản phẩm thống nhất[1]
Cơ chế Tùy biến Giao diện	Can thiệp trực tiếp vào mã nguồn HTML/CSS/JS của từng khách hàng[6]	Điều khiển động thông qua tệp siêu dữ liệu cấu hình từ API[3]
Mức độ An toàn Biên	Tự nhiên theo ranh giới tên miền riêng[6]	Phụ thuộc vào việc xác thực và khử độc siêu dữ liệu từ máy chủ[6]

Tầng Điện toán và Nghiệp vụ (Compute and Business Logic Tier)

Tầng điện toán đóng vai trò thực thi các ca sử dụng, kiểm tra tính hợp lệ của dữ liệu, thẩm định quyền hạn truy cập và điều phối các quy tắc chuyển dịch trạng thái công việc[3].

Phương án 1: Silo Compute (Điện toán Cô lập Tuyệt đối theo Thuê bao)

Silo Compute triển khai các cụm phiên bản ứng dụng chuyên biệt (Dedicated Containers, Pods hoặc Virtual Machines) chạy mã nguồn backend độc lập cho từng tenant cụ thể[6].

Ưu điểm lớn nhất là khả năng cô lập tài nguyên tính toán ở cấp độ vật lý hoặc tiến trình hệ điều hành[8]. Hiện tượng tranh chấp CPU hoặc tràn bộ nhớ giữa các tổ chức hoàn toàn bị triệt tiêu[8]. Mỗi tenant có thể thiết lập chính sách tự động mở rộng quy mô (autoscaling) theo tải sử dụng riêng, đồng thời đáp ứng các tiêu chuẩn tuân thủ nghiêm ngặt về phân tách môi trường điện toán[8].

Nhược điểm là hiệu quả khai thác tài nguyên rất kém[3]. Với các tổ chức có quy mô nhỏ hoặc hoạt động theo khung giờ hành chính, năng lực xử lý của máy

chủ bị bỏ phí nghiêm trọng trong thời gian nhàn rỗi[3]. Chi phí duy trì hạ tầng tính toán tăng tỷ lệ thuận với số lượng tenant, gây áp lực tài chính nặng nề cho nhà cung cấp SaaS[8].

Phương án 2: Pooled Stateless Compute (Điện toán Dùng chung Vô trạng thái)

Pooled Stateless Compute vận hành một cụm máy chủ hoặc dịch vụ backend vô trạng thái dùng chung cho tất cả các tenant[2]. Mỗi yêu cầu từ người dùng bắt buộc phải mang mã thông báo xác thực (JWT)[3]. Sau khi kiểm tra tính hợp lệ của danh tính, hệ thống trích xuất thông tin để xây dựng một đối tượng ngữ cảnh bất biến (TenantContext) gắn kèm vào luồng thực thi nhằm điều phối quyền truy cập và lọc dữ liệu tương ứng[3].

Ưu điểm vượt trội là mật độ khai thác tài nguyên tối đa và tối ưu hóa chi phí vận hành[3]. Năng lực tính toán nhàn rỗi của một tenant có thể được chia sẻ tức thời để phục vụ lưu lượng đột biến của tenant khác[13]. Cụm backend có thể tự động co giãn linh hoạt theo tổng lưu lượng truy cập toàn hệ thống mà không phụ thuộc vào số lượng tenant đăng ký[12].

Nhược điểm là mức độ phụ thuộc lớn vào tính kỷ luật của mã nguồn[3]. Một sơ suất trong logic phần mềm làm thất lạc hoặc sai lệch TenantContext có thể dẫn đến việc xử lý nhầm dữ liệu giữa các tổ chức[3]. Ngoài ra, nếu thiếu các cơ chế vách ngăn bảo vệ, nguy cơ Noisy Neighbor có thể làm tê liệt toàn bộ cụm dịch vụ dùng chung[3].

Phương án 3: Deployment Stamps (Khối Triển khai Độc lập theo Nhóm Thuê bao)

Theo mẫu thiết kế được Microsoft Azure Architecture Center chuẩn hóa, Deployment Stamps nhóm các tài nguyên tính toán, lưu trữ và mạng thành các đơn vị quy mô chuẩn hóa (gọi là Stamp hoặc Cell), mỗi đơn vị phục vụ một tập hợp tenant xác định (ví dụ: mỗi stamp phục vụ từ 50 đến 100 tenant)[7]. Bên trong mỗi stamp, các tài nguyên được chia sẻ theo mô hình pooled[7]. Khi dung lượng của stamp hiện tại tiệm cận ngưỡng thiết kế, một stamp mới sẽ được tự động khởi tạo bằng công cụ hạ tầng dưới dạng mã (IaC) để tiếp nhận các tenant tiếp theo[7].

Ưu điểm then chốt là khoanh vùng ranh giới sự cố hiệu quả[7]. Sự cố phần cứng, tắc nghẽn mạng hoặc lỗi phát sinh trong một stamp chỉ ảnh hưởng đến nhóm tenant trực thuộc, hoàn toàn không đe dọa đến tính sẵn sàng của các stamp khác[7]. Mô hình này cho phép hệ thống mở rộng quy mô tuyến tính mà không vấp phải giới hạn vật lý của một cụm máy chủ hay cơ sở dữ liệu đơn lẻ, đồng thời hỗ trợ định vị dữ liệu theo từng khu vực địa lý mà không làm phân mảnh mã nguồn[7].

Nhược điểm là đòi hỏi tầng quản trị điều phối (Control Plane) có năng lực cao

để quản lý bảng ánh xạ giữa tenant và stamp, giám sát dung lượng của từng đơn vị và xử lý các kịch bản di chuyển tenant giữa các stamp khi có nhu cầu tái cân bằng tải[3].

Khảo sát Mô hình Cô lập Điện toán trên Nền tảng Kubernetes

Khi triển khai tầng điện toán trên nền tảng điều phối container Kubernetes, ranh giới cô lập giữa các tenant được phân định thành hai trường phái kỹ thuật rõ rệt[10]:

Trường phái Cô lập Mềm (Soft Multi-tenancy) sử dụng chung một Kubernetes Control Plane (API Server, etcd, Scheduler) và cụm Worker Node[10]. Các tenant được phân tách logic thông qua cấu trúc Không gian tên (Namespaces)[10]. Bảo mật và kiểm soát tài nguyên được thiết lập thông qua việc áp dụng nghiêm ngặt các chính sách RBAC (chỉ cấp quyền trong phạm vi namespace), NetworkPolicies (thiết lập mặc định cấm toàn bộ lưu lượng mạng ra vào giữa các namespace khác nhau), và ResourceQuotas (giới hạn trần dung lượng CPU, bộ nhớ và số lượng Pod)[10]. Mô hình này mang lại hiệu quả chi phí tối ưu, triển khai thuận tiện nhưng vẫn chia sẻ nhân hệ điều hành (Linux Kernel), tiềm ẩn rủi ro bảo mật nếu xảy ra lỗ hổng thoát hộp cát (container escape) hoặc nghẽn API Server dùng chung[15].

Trường phái Cô lập Cứng (Hard Multi-tenancy) áp dụng cho các môi trường có mức độ tin cậy thấp hoặc đòi hỏi tuân thủ pháp lý khắt khe[10]. Các kỹ thuật triển khai bao gồm: cấp phát các Node Pool chuyên biệt thông qua cơ chế gắn nhãn và loại trừ (Taints and Tolerations kết hợp NodeAffinity) để đảm bảo Pod của các tenant không bao giờ chạy chung máy chủ vật lý; áp dụng môi trường thực thi hộp cát bảo mật cấp nhân ảo hóa (Sandboxed Runtimes như gVisor hoặc Kata Containers) để chặn đứng việc gọi hàm hệ thống trực tiếp xuống nhân máy chủ; triển khai các cụm điều khiển ảo hóa (Virtual Clusters thông qua vCluster hoặc Capsule) nhằm cấp cho mỗi tenant một API Server riêng biệt; hoặc tạo lập các cụm Kubernetes hoàn toàn độc lập cho từng khách hàng[10]. Mô hình này triệt tiêu rủi ro bảo mật hạ tầng nhưng chi phí vận hành và tài nguyên duy trì tăng cao đột biến[14].

Tiêu chí Đánh giá	Silo Compute	Pooled Stateless Compute	Deployment Stamps
Hiệu suất Khai thác CPU/RAM	Thấp; lãng phí năng lực tính toán nhàn rỗi[3]	Tối ưu; chia sẻ năng lực linh hoạt giữa các	Cân bằng; tối ưu hóa trong phạm vi từng stamp[7]

		tenant[3]	
Ranh giới Sự cố (Blast Radius)	Rất nhỏ; giới hạn trong một tenant duy nhất[8]	Toàn hệ thống; sự cố compute ảnh hưởng mọi tenant[3]	Có kiểm soát; cô lập hoàn toàn trong stamp bị lỗi[7]
Khả năng Mở rộng Hạ tầng	Kém; số lượng đơn vị triển khai bùng nổ theo khách hàng[12]	Phụ thuộc vào giới hạn mở rộng của các điểm nghẽn chung[3]	Mở rộng tuyến tính bằng cách nhân bản các stamp chuẩn hóa[7]
Mức độ Phức tạp Vận hành	Cao; quản lý phân tán nhiều cụm tài nguyên độc lập[12]	Thấp nhất; duy trì một cụm dịch vụ đồng nhất[3]	Cần lớp Router biên và Control Plane điều phối stamp[3]

Tầng Dữ liệu và Lưu trữ (Data Placement and Storage Tier)

Tầng dữ liệu là trọng tâm bảo đảm tính toàn vẹn, bảo mật và hiệu năng lâu dài của toàn bộ giải pháp đa thuê bao[3].

Phương án 1: Silo Database (Cơ sở Dữ liệu Độc lập theo Thuê bao)

Mỗi tenant được cấp phát một phiên bản cơ sở dữ liệu (Database Instance) hoàn toàn riêng biệt về mặt vật lý hoặc logic[5].

Ưu điểm nổi bật là ranh giới cô lập dữ liệu đạt mức tối đa[5]. Nguy cơ truy vấn nhầm lẫn dữ liệu giữa các tổ chức bị loại bỏ ở cấp độ kiến trúc lưu trữ[5]. Việc sao lưu dữ liệu độc lập và phục hồi thảm họa về một thời điểm xác định (Point-in-Time Recovery) cho một khách hàng cụ thể được thực hiện trực tiếp mà không ảnh hưởng đến bất kỳ ai khác[5].

Nhược điểm là vấn đề cạn kiệt tài nguyên kết nối và chi phí vận hành tăng vọt[3]. Mỗi cơ sở dữ liệu đòi hỏi lượng bộ nhớ đệm cơ sở và số lượng kết nối tối thiểu; khi số lượng tenant đạt hàng trăm hoặc hàng nghìn, chi phí hạ tầng trở nên khổng lồ[3]. Hơn nữa, việc thực thi các kịch bản nâng cấp lược đồ (Schema Migration) biến thành cơn ác mộng vận hành khi phải điều phối đồng bộ hàng nghìn kết nối với nguy cơ thất bại cục bộ làm phân mảnh phiên bản dữ liệu[3].

Phương án 2: Bridge (Lược đồ Riêng trên Cùng Cơ sở Dữ liệu - Schema-per-tenant)

Tất cả các tenant chia sẻ chung một máy chủ cơ sở dữ liệu nhưng mỗi tenant được định danh bằng một vùng lược đồ riêng biệt (tenant_a.tasks,

tenant_b.tasks)[5].
Ưu điểm là cung cấp ranh giới phân tách logic rõ ràng ở cấp độ không gian tên (namespace) của hệ quản trị cơ sở dữ liệu[5]. Phương án này giải quyết bài toán lãng phí kết nối vật lý so với Silo Database và cho phép thiết lập các quyền hạn bảo mật cơ sở dữ liệu riêng biệt cho từng vai trò người dùng[5].
Nhược điểm xuất hiện khi số lượng tenant gia tăng nhanh chóng, dẫn đến hiện tượng phình to danh mục hệ thống (PostgreSQL System Catalog Bloat)[3]. Việc hệ quản trị phải quản lý hàng chục nghìn bảng và chỉ mục làm suy giảm nghiêm trọng hiệu năng của bộ lập lịch truy vấn[3]. Ngoài ra, thời gian thực thi migration vẫn tăng tuyến tính theo số lượng tenant và không giải quyết được vấn đề tranh chấp bộ nhớ chia sẻ và dung lượng vào/ra (I/O) của đĩa từ[3].

Phương án 3: Pooled Database (Lược đồ Dùng chung kết hợp Cô lập Đa tầng - Shared Schema + Multi-layer Isolation)

Toàn bộ các tenant lưu trữ dữ liệu chung trong một lược đồ quan hệ duy nhất, trong đó mọi bảng dữ liệu nghiệp vụ đều bắt buộc chứa cột định danh tenant_id[3]. Ranh giới an toàn được bảo vệ bởi chiến lược phòng thủ theo chiều sâu (Defense in Depth) kết hợp giữa ràng buộc toàn vẹn quan hệ và chính sách Row-Level Security (RLS) của hệ quản trị cơ sở dữ liệu[3].
Ưu điểm là đạt hiệu quả sử dụng tài nguyên tối đa[3]. Hồ chứa kết nối cơ sở dữ liệu và bộ nhớ đệm trang dữ liệu (Buffer Pool) được chia sẻ tối ưu cho toàn bộ các truy vấn[3]. Quy trình CI/CD lược đồ dữ liệu đạt mức tinh gọn nhất: mọi thay đổi cấu trúc bảng chỉ cần thực thi một lần duy nhất thông qua một tệp mã kịch bản di chuyển (migration script)[3].
Nhược điểm là áp lực bảo mật dồn hoàn toàn vào tính kỷ luật của các lớp phòng thủ[3]. Nếu các lớp kiểm soát logic tại ứng dụng bị vượt qua và cơ sở dữ liệu thiếu cấu hình RLS chặt chẽ, nguy cơ rò rỉ dữ liệu giữa các tổ chức có thể xảy ra ngay lập tức[3].

Cơ chế Thực thi Cô lập Dữ liệu Đa tầng và Vai trò của RLS trong PostgreSQL

Nhằm vô hiệu hóa hoàn toàn rủi ro rò rỉ dữ liệu trong mô hình Pooled Database, kiến trúc phải triển khai năm lớp bảo vệ độc lập và tương hỗ[3]:
Lớp đầu tiên là thẩm định ngữ cảnh tại biên (Edge Validation)[3]. Ngay tại bộ lọc cổng vào, hệ thống đối chiếu tính nhất quán giữa tên miền con của yêu cầu, thông tin mã thông báo JWT và trạng thái hoạt động thực tế của tài khoản người dùng; yêu cầu bị từ chối ngay lập tức nếu có sự sai lệch[2].
Lớp thứ hai là ủy quyền tại tầng dịch vụ (Application Authorization)[3]. Tầng nghiệp vụ chỉ tiếp nhận đối tượng TenantContext đã được hệ thống xác thực an toàn, không bao giờ tin cậy hoặc nhận tham số tenant_id được truyền tự do từ phía máy khách[2]. Mọi truy vấn hướng cơ sở dữ liệu đều được tự động

chèn mệnh đề lọc tenant (WHERE tenant_id = :currentTenantId)[2].
Lớp thứ ba là ràng buộc toàn vẹn lược đồ (Schema Relational Constraints)[3]. Toàn bộ các bảng nghiệp vụ liên quan đến tổ chức đều bắt buộc áp dụng khóa chính kết hợp (Composite Primary Key) và khóa ngoại kết hợp (Composite Foreign Key) dạng (tenant_id, id)[2]. Ràng buộc này đảm bảo ở cấp độ cấu trúc dữ liệu rằng một công việc thuộc Tenant A tuyệt đối không thể tham chiếu hoặc liên kết sang một cột hay dự án thuộc Tenant B[2].
Lớp thứ tư là chính sách bảo mật cấp hàng của cơ sở dữ liệu (PostgreSQL Row-Level Security - RLS)[3]. RLS hoạt động như một lưới an toàn độc lập với mã nguồn ứng dụng[3]. Khi RLS được kích hoạt, hệ quản trị cơ sở dữ liệu tự động gắn thêm các điều kiện lọc vô hình vào mọi câu lệnh truy vấn[5]. Chính sách được thiết lập bằng câu lệnh kiểm tra điều kiện dòng dữ liệu với biến phiên làm việc: tenant_id = current_setting('app.tenant_id')::uuid, áp dụng bắt buộc cho cả chiều đọc dữ liệu (USING) và chiều ghi dữ liệu (WITH CHECK)[5].
Để RLS phát huy hiệu lực an toàn tuyệt đối và ngăn ngừa các lỗi rò rỉ phổ biến, các điều kiện kỹ thuật bắt buộc phải được thỏa mãn[5]:

- Tài khoản kết nối của ứng dụng (Runtime Role) phải tuân thủ nguyên tắc đặc quyền tối thiểu: không phải là Superuser, không sở hữu các bảng dữ liệu, và tuyệt đối không mang cờ BYPASSRLS[5].
- Bắt buộc thực thi lệnh FORCE ROW LEVEL SECURITY trên tất cả các bảng dữ liệu của tenant nhằm ngăn chặn tình trạng chính sách bị vô hiệu hóa nếu bảng vô tình thuộc quyền sở hữu của người dùng hiện hành[5].
- Khi ứng dụng mượn kết nối từ Connection Pool để thực thi yêu cầu, việc thiết lập biến phiên bắt buộc phải dùng lệnh có phạm vi giao dịch cục bộ: SET LOCAL app.tenant_id = '...'[5]. Lệnh SET LOCAL đảm bảo giá trị này tự động bị hủy bỏ ngay khi giao dịch kết thúc (COMMIT hoặc ROLLBACK), triệt tiêu hoàn toàn nguy cơ rò rỉ ngữ cảnh sang yêu cầu tiếp theo khi tái sử dụng kết nối[5].
- Cần nhìn nhận khách quan rằng RLS không phải cơ chế phần cứng và không triệt tiêu mọi rủi ro: RLS tạo ra tải phụ (overhead) tính toán nhất định lên bộ lập lịch truy vấn, và không bảo vệ được hệ thống nếu quản trị viên vô tình cấu hình sai quyền hạn của vai trò kết nối[3].

Lớp thứ năm là phân vùng không gian tên tài nguyên phụ trợ (Auxiliary Namespace Isolation)[3]. Các khóa lưu trữ trên bộ nhớ đệm Redis bắt buộc phải có tiền tố tenant:{id}:..., và các tệp đính kèm trên hệ thống lưu trữ đối tượng bắt buộc phải tuân theo tiền tố đường dẫn riêng biệt do máy chủ kiểm soát[3].

Mô hình Lưu trữ Dữ liệu Tùy biến cho Ứng dụng Quản lý Công việc

Để hỗ trợ khả năng mở rộng các trường dữ liệu linh hoạt theo đặc thù của

từng tổ chức mà không làm biến tướng mô hình Pooled Database thành việc can thiệp cấu hình vật lý liên tục, bốn giải pháp kỹ thuật được đặt lên bàn cân so sánh[3]:

Giải pháp Tạo Bảng/Cột Vật lý Động (Physical DDL at Runtime) thực thi các lệnh ALTER TABLE ADD COLUMN hoặc tạo bảng mới mỗi khi người dùng cấu hình trường dữ liệu[3]. Phương án này bị loại bỏ hoàn toàn do gây ra hiện tượng khóa bảng (Table Lock), làm gián đoạn truy cập, phân mảnh cấu trúc dữ liệu và khiến việc bảo trì, cập nhật phiên bản phần mềm trở nên bất khả thi[3].

Giải pháp Thực thể - Thuộc tính - Giá trị Truyền thống (Untyped EAV) lưu trữ các thuộc tính tùy biến vào bảng mở rộng dưới dạng chuỗi văn bản thuần túy[3]. Mô hình này bộc lộ nhược điểm nghiêm trọng khi ứng dụng Kanban cần lọc và sắp xếp công việc theo nhiều kiểu dữ liệu khác nhau[2]. Việc ép kiểu dữ liệu từ chuỗi ký tự sang số học hoặc ngày tháng làm mất đi tính toàn vẹn dữ liệu và triệt tiêu khả năng tối ưu hóa của các chỉ mục cơ sở dữ liệu quan hệ[2].

Giải pháp Lưu trữ Cột Tài liệu Không định dạng (Pure JSONB Column) gom toàn bộ các trường tùy biến vào một cột JSONB duy nhất trong bảng công việc[2]. Mặc dù rất linh hoạt trong lưu trữ tài liệu tĩnh, việc truy vấn lọc dải (range query) và sắp xếp phức tạp trên các trường số hoặc ngày tháng lồng nhau trong JSONB tiêu tốn nhiều tài nguyên bộ nhớ và kém hiệu quả hơn các chỉ mục B-tree chuyên biệt[2].

Giải pháp Siêu dữ liệu Định kiểu và Bảng Giá trị (Typed Metadata/Value Table) tách biệt rõ ràng giữa cấu trúc định nghĩa và dữ liệu thực tế[2]. Bảng custom_field_definitions lưu trữ định nghĩa trường (tên trường, kiểu dữ liệu, tùy chọn, cờ bắt buộc)[2]. Bảng task_custom_field_values lưu trữ dữ liệu thực thi với các cột được định kiểu rõ ràng: text_value, number_value, date_value, boolean_value, và option_id, đi kèm ràng buộc kiểm tra (Check Constraint) chỉ cho phép một cột duy nhất có dữ liệu trên mỗi dòng[2]. Mô hình này duy trì tính toàn vẹn kiểu dữ liệu của cơ sở dữ liệu quan hệ, cho phép tạo các chỉ mục B-tree kết hợp tối ưu như (tenant_id, field_id, number_value) để phục vụ việc lọc và sắp xếp tốc độ cao mà tuyệt đối không đòi hỏi thay đổi cấu trúc DDL ở thời gian chạy[2].

Quản lý Lưu trữ Tệp đính kèm (Object Storage)

Đối với các tệp tài liệu và hình ảnh đính kèm trên bảng Kanban, việc cấp phát vùng chứa riêng (Bucket-per-tenant) gây lãng phí lớn và nhanh chóng chạm trần hạn ngạch tài nguyên đám mây[2]. Giải pháp chuẩn hóa là sử dụng một Bucket dùng chung duy nhất, trong đó tiền tố đường dẫn tệp (Object Key Prefix) bắt buộc phải do máy chủ tạo lập theo cấu trúc phân cấp: tenants/{tenant_id}/projects/{project_id}/{resource_id}[2].

Hệ thống áp dụng mẫu thiết kế Chìa khóa Ủy quyền (Valet Key Pattern)[17]. Ứng dụng phía máy khách không bao giờ tải tệp trực tiếp thông qua thông tin xác thực dùng chung[2]. Khi có nhu cầu tải lên hoặc tải xuống, tầng nghiệp vụ tiến hành xác thực quyền thành viên đối với công việc tương ứng, sau đó tạo ra một liên kết truy cập có chữ ký số (Presigned URL) với thời gian hiệu lực cực ngắn (vài phút) và giới hạn quyền truy cập nghiêm ngặt trong phạm vi tiền tố của tổ chức đó[3].

Mô hình Dữ liệu	Chi phí & Sử dụng Kết nối	Mức độ Cô lập Dữ liệu	Độ phức tạp Nâng cấp (Migration)	Khả năng Tùy biến Nghiệp vụ
Silo Database	Kém nhất; lãng phí bộ nhớ đệm và cạn kết nối[3]	Tối đa; phân tách vật lý hoàn toàn[5]	Cực kỳ phức tạp; chạy nâng cấp N lần độc lập[3]	Tự do can thiệp DDL vật lý theo nhu cầu riêng[6]
Schema-per-tenant	Trung bình; chia sẻ máy chủ nhưng phân mảnh đệm[5]	Tốt; phân ranh giới theo namespace cơ sở dữ liệu[5]	Rất khó; phình to danh mục hệ thống (catalog bloat)[3]	Có thể chỉnh sửa DDL cục bộ trong schema riêng[6]
Pooled (Shared Schema + RLS)	Tối ưu nhất; chia sẻ hoàn toàn kết nối và bộ nhớ đệm[3]	Rất cao khi kết hợp RLS, Composite Key và App Authorization[3]	Thấp nhất; thực thi duy nhất một kịch bản cập nhật[3]	Tùy biến an toàn qua Typed Metadata/Value Table[2]

Tầng Quản trị và Vận hành (Control Plane, Governance and Noisy Neighbor Tier)

Hệ thống SaaS chuyên nghiệp đòi hỏi sự phân tách kiến trúc rõ ràng giữa Mặt phẳng Ứng dụng (Application Plane) và Mặt phẳng Quản trị (Control Plane)[1].

Phân định Ranh giới Mặt phẳng Quản trị và Mặt phẳng Ứng dụng

Mặt phẳng Ứng dụng là nơi người dùng cuối thực hiện các thao tác quản lý công việc hàng ngày như tạo thẻ, cập nhật trạng thái và trao đổi thông tin[6]. Toàn bộ các dịch vụ tại đây vận hành hoàn toàn trong ranh giới của một tenant xác định[1].
Mặt phẳng Quản trị đóng vai trò là cơ quan điều hành trung tâm của nhà cung cấp dịch vụ SaaS, tách biệt khỏi luồng dữ liệu nghiệp vụ của khách hàng[1]. Control Plane đảm nhiệm toàn bộ vòng đời của các tổ chức:

- Quy trình Cấp phát Tự động (Self-service Onboarding): Tiếp nhận đăng ký mới, khởi tạo mã định danh duy nhất tenant_id, tạo tài khoản quản trị viên đầu tiên, thiết lập cấu hình mặc định, ghi nhận ánh xạ vào Stamp phù hợp còn năng lực tiếp nhận, và khởi tạo các dữ liệu mẫu cần thiết[2].
- Quản lý Vòng đời Thuê bao: Điều phối các trạng thái hoạt động của tổ chức từ Khởi tạo (PROVISIONING), Hoạt động bình thường (ACTIVE), Tạm ngưng do vi phạm hoặc quá hạn thanh toán (SUSPENDED), cho đến Thanh lọc dữ liệu (PURGED)[2]. Khi một tổ chức chuyển sang trạng thái tạm ngưng, bộ lọc tại tầng biên lập tức chặn đứng mọi truy cập vào Application Plane mà hoàn toàn không ảnh hưởng đến hoạt động của các tổ chức khác[2].
- Đo lường và Giám sát Tài nguyên (Resource Metering): Thu thập các chỉ số tiêu thụ tài nguyên đại diện (dung lượng lưu trữ tệp, số lượng công việc/dự án đang quản lý, tần suất gọi API) nhằm phục vụ quản trị dung lượng hệ thống và giám sát chi phí vận hành[3].

Kiểm soát và Hạn chế Tác động "Kẻ láng giềng Ồn ào" (Noisy Neighbor Mitigation)

Để bảo vệ chất lượng dịch vụ chung trên môi trường chia sẻ tài nguyên, hệ thống phải triển khai đồng bộ các cơ chế kiểm soát lưu lượng và cô lập tài nguyên[3]:

- Giới hạn Tốc độ Phân tán (Distributed Rate Limiting): Triển khai thuật toán Thùng chứa Mã thông báo (Token Bucket) trên cụm bộ nhớ đệm phân tán Redis tại tầng API Gateway hoặc bộ lọc ứng dụng[3]. Thay vì giới hạn theo địa chỉ IP thông thường, hạn ngạch được kiểm soát nghiêm ngặt theo từng định danh tenant_id[3]. Khi lưu lượng vượt ngưỡng thiết kế, hệ thống lập tức phản hồi mã trạng thái HTTP 429 (Too Many Requests), ngăn chặn yêu cầu bất thường đi sâu vào hạ tầng tính toán và cơ sở dữ liệu[3].
- Vách ngăn Tài nguyên Đồng thời (Concurrency Bulkheads): Thiết lập giới hạn số lượng luồng xử lý hoặc số lượng kết nối cơ sở dữ liệu đồng thời tối đa mà một tenant được phép chiếm dụng trong Connection Pool[3]. Cơ chế này ngăn chặn tình huống một tổ chức thực hiện hàng loạt

yêu cầu phức tạp làm cạn kiệt toàn bộ hồ chứa kết nối của ứng dụng[3].

- Thời gian Ngắt Lệnh và Phân trang Bắt buộc (Statement Timeouts & Keyset Pagination): Thiết lập cấu hình ngắt lệnh nghiêm ngặt trên cơ sở dữ liệu cho mọi kết nối của ứng dụng (ví dụ: tự động hủy các truy vấn đọc vượt quá 3 giây) nhằm loại trừ nguy cơ các câu lệnh quét bảng phức tạp làm nghẽn CPU[3]. Mọi API trả về danh sách thẻ việc hoặc lịch sử hoạt động đều bắt buộc phải áp dụng kỹ thuật phân trang theo con trỏ (keyset/cursor pagination) để kiểm soát kích thước tập dữ liệu trả về[3].
- Lập lịch Hàng đợi Công bằng cho Tác vụ Bất đồng bộ (Fair Queueing): Đối với các tác vụ xử lý nền tốn nhiều thời gian (gửi email thông báo, xuất báo cáo, kích hoạt tự động hóa), hệ thống sử dụng hàng đợi thông điệp có phân vùng logic theo tenant[2]. Bộ điều phối xử lý luân phiên công bằng (Fair Scheduling) giữa các tổ chức, ngăn ngừa tình trạng một tenant đưa hàng loạt thông điệp vào hàng đợi làm tắc nghẽn toàn bộ luồng xử lý của hệ thống (Head-of-Line Blocking)[2].

Cơ chế Quản trị	Vị trí Thực thi	Mục tiêu Bảo vệ	Giới hạn Cần Lưu ý
Distributed Rate Limiting	Edge Gateway / Filter[6]	Ngăn chặn các đợt bùng nổ lưu lượng API vượt hạn ngạch[3]	Chỉ hạn chế yêu cầu mới; không giải quyết được các truy vấn nặng đã lọt vào cơ sở dữ liệu[3]
Concurrency Bulkhead	Connection Pool / Threads[3]	Đảm bảo kết nối cơ sở dữ liệu luôn sẵn sàng cho các tenant khác[3]	Cần hiệu chuẩn ngưỡng chính xác để tránh từ chối nhầm các yêu cầu hợp lệ[3]
Statement Timeout	Database Session[3]	Hủy kịp thời các câu lệnh truy vấn tiêu tốn quá nhiều tài nguyên tính toán[3]	Phía máy khách cần xử lý lỗi ngắt kết nối một cách an toàn[3]
Fair Queue	Message Broker	Đảm bảo tính	Tăng độ phức

Scheduling	& Workers[2]	công bằng trong việc thực thi các tác vụ nền bất đồng bộ[2]	tạp trong việc thiết kế và lập lịch hàng đợi[2]

Đánh giá So sánh Đa Tiêu chí giữa các Mô hình Kiến trúc Tổng thể

Việc kết hợp các giải pháp ở từng tầng tạo nên bốn phương án kiến trúc tổng thể tiêu biểu[3]:

- Phương án A (Full-Stack Silo per Tenant): Mỗi tenant sở hữu một hạ tầng độc lập hoàn toàn từ giao diện, API đến cơ sở dữ liệu riêng biệt[3].
- Phương án B (Một Cụm Pooled Toàn cục - Global Shared Pool): Toàn bộ tenant dùng chung một cụm giao diện, một cụm API duy nhất và một cơ sở dữ liệu dùng chung duy nhất trên toàn cầu[3].
- Phương án C (Bridge Cố định theo Gói Dịch vụ): Dùng chung tầng giao diện và điện toán, nhưng phân tách cơ sở dữ liệu thành ba nhánh: Shared Schema cho gói khởi đầu, Schema riêng cho gói chuyên nghiệp, và Database riêng cho gói doanh nghiệp lớn[3].
- Phương án D (Pooled Multi-Tenancy theo Deployment Stamps): Một Mặt phẳng Quản trị dùng chung điều phối nhiều Stamp tiêu chuẩn[2]. Trong mỗi Stamp, các tenant chia sẻ hạ tầng API vô trạng thái và chia sẻ một cơ sở dữ liệu duy nhất với cơ chế cô lập đa tầng (Shared Schema + RLS + Typed Metadata/Value Table)[2]. Hệ thống mở rộng bằng cách bổ sung thêm các Stamp mới khi quy mô phát triển[7].

Tiêu chí Đánh giá	Phương án A: Full-Stack Silo	Phương án B: Global Pool	Phương án C: Bridge Cố định	Phương án D: Pooled Deployment Stamps
Mức độ Cô lập & An toàn	Cao nhất về mặt vật lý[5]	Phụ thuộc hoàn toàn vào logic ứng dụng[3]	Phân mảnh; chỉ an toàn ở nhánh dữ liệu riêng[3]	Rất cao; cô lập đa tầng nội tại kết hợp ranh giới stamp[2]
Ranh giới	Rất nhỏ; giới hạn trong	Toàn hệ thống; sự cố	Lớn ở tầng compute	Có kiểm soát; cô lập

Sự cố (Blast Radius)	một tenant[8]	ảnh hưởng mọi khách hàng[3]	dùng chung[3]	hoàn toàn trong phạm vi một stamp[7]
Hiệu quả Chi phí (TCO)	Rất kém; chi phí cơ sở tăng đột biến theo số tenant[3]	Tối đa ở giai đoạn đầu; giảm chi phí vận hành[3]	Phức tạp; chi phí duy trì nhiều loại hạ tầng[3]	Rất cao; tận dụng chia sẻ tài nguyên trong từng stamp[3]
Độ phức tạp CI/CD & Vận hành	Khủng hoảng vận hành khi số tenant lớn[6]	Rất đơn giản; chỉ có một đường ống triển khai[3]	Rất khó; phải bảo trì đồng thời ba loại lược đồ[3]	Tiêu chuẩn hóa; phân phối đồng bộ theo từng stamp[7]
Khả năng Mở rộng Quy mô	Giới hạn bởi năng lực quản trị cụm[12]	Vấp phải giới hạn vật lý của cơ sở dữ liệu duy nhất[3]	Bị tắc nghẽn ở khâu điều phối dữ liệu không nhất quán[3]	Mở rộng tuyến tính không giới hạn bằng việc nhân bản stamp[7]
Tính Nhất quán Nghiệp vụ	Dễ phân mảnh phiên bản phần mềm[6]	Nhất quán hoàn toàn trên toàn hệ thống[3]	Dễ gán nhầm tính năng vào loại hạ tầng[3]	Một mã nguồn duy nhất, tùy biến hoàn toàn qua siêu dữ liệu[2]

Kết luận Kiến trúc Đích cho Ứng dụng Quản lý Công việc

Dựa trên việc đối chiếu có hệ thống giữa các tiêu chuẩn kỹ thuật đám mây từ AWS, Azure, Kubernetes và đặc thù nghiệp vụ của ứng dụng quản lý công việc cộng tác, kiến trúc đích tối ưu duy nhất được xác lập là:
Kiến trúc Pooled Multi-Tenancy theo mô hình Deployment Stamps kết hợp Cô lập Đa tầng (Multi-layer Isolation) và Tùy biến Dựa trên Siêu dữ liệu (Metadata-Driven Customization)[2].
Kiến trúc này loại bỏ cách tiếp cận chia tách sản phẩm thành ba gói hạ tầng

phân mảnh (Bridge ba nhánh), đồng thời khắc phục triệt để nguy cơ tê liệt toàn hệ thống của mô hình Global Pool đơn lẻ[2]. Ba nhánh triển khai POOL, SCHEMA_PER_TENANT và SILO_DATABASE trong mã nguồn được định vị lại đúng với giá trị khoa học: đóng vai trò là **các kịch bản cơ sở thực nghiệm đối chứng (Baseline Benchmarks)** để đo lường định lượng và làm nổi bật tính ưu việt của kiến trúc đích được đề xuất[2].

Đặc tả Chi tiết Bản thiết kế Hệ thống Đích

Toàn bộ luồng xử lý và ranh giới cấu trúc của kiến trúc đích được tích hợp chặt chẽ từ cổng vào mạng đến tầng lưu trữ bền vững theo các thành phần chức năng:

Tại Tầng Biên và Định tuyến Toàn cục (Global Edge & Routing), hệ thống tiếp nhận lưu lượng tập trung thông qua chứng chỉ ký số Wildcard SSL/TLS (*.taskflow.vn)[2]. Bộ định tuyến biên trích xuất tên miền con đại diện của tổ chức và truy vấn danh bạ quản trị để phân giải định danh theo luồng hai bước: từ tenant_slug xác định tenant_id, và từ tenant_id xác định stamp_id đích[2]. Yêu cầu mạng sau đó được chuyển tiếp chính xác đến cổng vào của Stamp tương ứng mà không làm lộ cấu trúc liên mạng nội bộ[2].

Tại Tầng Trình diễn Thống nhất (Shared Presentation), người dùng tải về bản đóng gói tĩnh duy nhất của ứng dụng React SPA từ mạng phân phối nội dung CDN[2]. Khi khởi chạy trên trình duyệt, ứng dụng gửi yêu cầu lấy siêu dữ liệu cấu hình theo tenant để tự động dựng giao diện bảng Kanban, màu sắc thương hiệu và cấu trúc các biểu mẫu nhập liệu[3]. Mọi quyền hạn thao tác đều được kiểm soát độc lập tại máy chủ; việc ẩn các nút bấm trên giao diện chỉ đóng vai trò tối ưu hóa trải nghiệm người dùng, không đại diện cho ranh giới an toàn[2].

Tại Tầng Điện toán Vô trạng thái theo Stamp (Stateless Compute per Stamp), backend ứng dụng được triển khai dưới dạng cụm dịch vụ vô trạng thái (Stateless Services) vận hành bên trong từng Stamp[2]. Bộ lọc ngữ cảnh (Tenant Context Filter) tiến hành đối chiếu chéo bắt buộc giữa tên miền truy cập, mã xác thực JWT và trạng thái hoạt động của tổ chức[2]. Khi xác thực thành công, đối tượng ngữ cảnh TenantContext được khởi tạo và gắn vào luồng xử lý[2]. Các tác vụ xử lý bất đồng bộ bắt buộc phải đóng gói tenant_id trong nội dung thông điệp; các luồng xử lý nền khôi phục ngữ cảnh độc lập và thực thi cơ chế loại trừ trùng lặp (Idempotency) dựa trên khóa định danh theo tenant[2]. Quy trình phê duyệt nhiều cấp và tự động hóa được điều phối thông qua Công cụ Quy tắc Động (Policy Engine) dùng chung, đảm bảo tuân thủ quy tắc nghiệp vụ mà không làm thay đổi mã nguồn[2].

Tại Tầng Dữ liệu Quan hệ với Cô lập Đa tầng (Shared PostgreSQL Schema with Multi-Layer Isolation), mỗi Stamp sở hữu một cụm cơ sở dữ liệu

PostgreSQL dùng chung cho toàn bộ các tenant trực thuộc stamp đó[2]. Toàn bộ các bảng nghiệp vụ (Dự án, Bảng, Cột, Công việc, Bình luận) đều bắt buộc sử dụng khóa chính kết hợp và khóa ngoại kết hợp dạng (tenant_id, id)[2]. Hệ thống kích hoạt đồng thời ENABLE ROW LEVEL SECURITY và FORCE ROW LEVEL SECURITY trên tất cả các bảng dữ liệu của tenant, áp dụng chính sách kiểm tra biến phiên: USING (tenant_id = current_setting('app.tenant_id')::uuid) và WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid)[3]. Tài khoản kết nối của ứng dụng bị tước bỏ quyền Superuser, không sở hữu bảng và không có cờ BYPASSRLS[3]. Biến phiên làm việc bắt buộc phải được thiết lập trong phạm vi giao dịch bằng lệnh SET LOCAL app.tenant_id = '...' để ngăn ngừa rò rỉ trạng thái giữa các phiên kết nối[5]. Dữ liệu tùy biến mở rộng được lưu trữ theo mô hình Typed Metadata/Value Table; không thực thi DDL ở thời gian chạy, đảm bảo toàn vẹn kiểu dữ liệu và tối ưu hóa chỉ mục B-tree sẵn có[2].

Tại Tầng Bộ nhớ đệm và Lưu trữ Đối tượng Bổ trợ (Auxiliary Storage & Caching), cụm Redis áp dụng quy ước tiền tố bắt buộc: tenant:{tenant_id}:{resource_type}:{resource_id} nhằm ngăn chặn triệt để xung đột khóa và rò rỉ dữ liệu đệm[3]. Các tệp đính kèm công việc được lưu trữ trên một Bucket dùng chung, phân vùng đường dẫn tệp nghiêm ngặt theo tiền tố tenants/{tenant_id}/...[2]. Quyền truy cập tệp chỉ được cấp phát thông qua các liên kết có chữ ký số xác thực (Presigned URLs) có thời hạn hiệu lực ngắn sau khi đã thẩm định quyền hạn thành viên[3].

Tại Mặt phẳng Quản trị Độc lập (Decoupled SaaS Control Plane), dịch vụ Control Plane chạy tách biệt hoàn toàn khỏi Application Plane, nắm giữ cơ sở dữ liệu quản trị riêng biệt chứa danh mục tổ chức, chính sách định tuyến và dung lượng hiện thời của các Stamp[2]. Control Plane cung cấp công cụ tự động hóa toàn bộ quy trình cấp phát tổ chức mới, điều phối vòng đời dịch vụ, thu thập các chỉ số tài nguyên tiêu thụ, và điều phối việc bổ sung Stamp mới khi dung lượng hệ thống tiệm cận ngưỡng cảnh báo[2].

Tầng Hệ thống	Thành phần Triển khai Đích	Cơ chế Bảo mật & Cô lập Thực thi	Cơ chế Tùy biến Nghiệp vụ
Biên & Định tuyến	Wildcard Ingress + Edge Router[2]	Phân giải hai bước (slug -> tenant_id -> stamp_id), từ	Định tuyến động theo tên miền con đại diện[6]

		chối kết nối sai lệch[2]	
Trình diễn	Single-build React SPA trên CDN[2]	Khử độc dữ liệu, xác thực phiên bảo mật, server-side authorization[2]	Metadata-Driven UI (tự dựng bảng, cột, biểu mẫu)[3]
Điện toán	Stateless Spring Boot API theo Stamp[2]	TenantContext bất biến, Token Bucket rate limit, Concurrency Bulkhead[3]	Policy Engine điều phối quy trình và phê duyệt động[2]
Dữ liệu Quan hệ	Shared PostgreSQL Schema trong Stamp[2]	Composite Keys (tenant_id, id), FORCE RLS, SET LOCAL biến phiên[3]	Typed Metadata/Value Table (không DDL ở runtime)[2]
Bộ nhớ đệm & Tệp	Redis & Shared Object Storage[3]	Tiền tố khóa tenant:{id}:..., Presigned URL theo Valet Key Pattern[3]	Cấu hình hạn ngạch dung lượng lưu trữ theo tenant[3]
Quản trị & Vận hành	Decoupled SaaS Control Plane[1]	Tách biệt Application Plane, kiểm soát vòng đời, giám sát Noisy Neighbor[3]	Cấu hình hạn ngạch và mẫu dự án khởi tạo[2]

Luận điểm Khoa học Bảo vệ Lựa chọn Kiến trúc Đích

Năm luận điểm khoa học có tính liên kết chặt chẽ xác lập nền tảng bảo vệ cho phương án kiến trúc đích được lựa chọn:

Thứ nhất là sự tương thích tối ưu với bản chất nghiệp vụ quản lý công việc[2]. Hệ thống Kanban phục vụ nhu cầu cộng tác thời gian thực với các giao dịch ngắn, đọc/ghi liên tục và chia sẻ chung một mô hình dữ liệu cốt lõi (dự án, bảng, cột, thẻ việc, thành viên)[2]. Nhóm khách hàng mục tiêu là các tổ chức

vừa và nhỏ, không có yêu cầu bắt buộc về hạ tầng điện toán vật lý riêng biệt[2]. Việc cấp phát cơ sở dữ liệu riêng hay máy chủ riêng cho từng khách hàng gây lãng phí nghiêm trọng tài nguyên nhàn rỗi và làm chi phí vận hành tăng vọt một cách không cần thiết[3]. Kiến trúc Pooled trong từng Stamp mang lại mật độ khai thác tài nguyên tối ưu và hiệu quả kinh tế cao nhất[3].

Thứ hai là khả năng mở rộng quy mô thực tế và khống chế ranh giới sự cố[7]. Mô hình Global Pool đơn lẻ chứa đựng rủi ro chí tử về ranh giới sự cố: khi cơ sở dữ liệu hoặc cụm API chạm trần giới hạn vật lý, sự cố sẽ làm tê liệt toàn bộ khách hàng trên toàn cầu[3]. Bằng cách áp dụng mẫu thiết kế Deployment Stamps, bài toán mở rộng quy mô được giải quyết bằng phương thức phân vùng cell-based[7]. Mỗi Stamp đóng vai trò là một đơn vị quy mô độc lập có dung lượng hữu hạn; sự cố phần cứng, tắc nghẽn chỉ mục hay bùng nổ lưu lượng tại một Stamp được cô lập hoàn toàn, bảo vệ tuyệt đối tính sẵn sàng của các Stamp còn lại[7].

Thứ ba là nguyên tắc phòng thủ theo chiều sâu (Defense in Depth) loại trừ điểm lỗi đơn lẻ[3]. Kiến trúc không đặt toàn bộ niềm tin bảo mật vào một cơ chế duy nhất[3]. Dữ liệu của tổ chức không chỉ trông cậy vào logic ứng dụng (vốn có thể phát sinh lỗi trong quá trình lập trình) hay chỉ dựa vào RLS (vốn có thể bị vô hiệu hóa nếu cấu hình sai quyền quản trị)[3]. Bằng cách thiết lập năm lớp ranh giới độc lập: Thẩm định biên, Ủy quyền dịch vụ, Khóa ngoại kết hợp, Ép buộc RLS cơ sở dữ liệu và Phân vùng tài nguyên phụ trợ; dữ liệu của từng tổ chức luôn được bảo vệ an toàn ngay cả khi một trong các lớp phòng thủ gặp sự cố[3].

Thứ tư là giải quyết triệt để bài toán tùy biến mà không phá vỡ tính chuẩn hóa của SaaS[3]. Việc cho phép can thiệp DDL vật lý ở thời gian chạy là sai lầm phổ biến khiến lược đồ bị phân mảnh và làm tê liệt quy trình CI/CD[3]. Bằng việc kết hợp kiến trúc giao diện điều khiển bằng siêu dữ liệu (Metadata-Driven UI) với mô hình dữ liệu giá trị định kiểu (Typed Metadata/Value Table), ứng dụng cho phép các tổ chức tự do định nghĩa các trường dữ liệu Kanban và quy trình phê duyệt đặc thù trong một cấu trúc quan hệ chuẩn mực mà không cần thay đổi bất kỳ cấu trúc bảng vật lý nào ở thời gian chạy[2].

Thứ năm là duy trì một dòng sản phẩm thống nhất và đảm bảo chất lượng vận hành[2]. Kiến trúc duy trì một phiên bản mã nguồn duy nhất cho toàn bộ hệ thống khách hàng[2]. Việc phát hành các tính năng mới hoặc nâng cấp lược đồ được thực hiện an toàn theo phương thức cuốn chiếu (Canary/Ring Deployment) từ Stamp thử nghiệm sang các Stamp chính thức, loại bỏ hoàn toàn hiện tượng trôi dạt cấu hình (configuration drift) và giảm thiểu tối đa chi phí hỗ trợ kỹ thuật lâu dài[7].

Mô hình Pooled Multi-Tenancy theo Deployment Stamps kết hợp Cô lập Đa

tầng và Tùy biến Dựa trên Siêu dữ liệu thiết lập một giải pháp kiến trúc toàn diện, dung hòa tối ưu giữa yêu cầu an toàn dữ liệu nghiêm ngặt, hiệu quả khai thác tài nguyên và năng lực vận hành bền vững cho ứng dụng quản lý công việc đa thuê bao[2].

References

1. COLLECTED_SOURCES_MULTITENANCY.md
2. LUAN_GIAI_KIEN_TRUC_MULTI_TENANT_TOI_UU.md
3. KIEN_TRUC_MULTI_TENANT_BAN_HOP_NHAT.md
4. thuyetMinhSaasMultiTenancy.md
5. Chapter 27 — Multi-tenant data isolation · Security in Depth, https://frontendlabs.xyz/security-in-depth/chapter-27-multi-tenant-data-isolation/
6. PHAN_TICH_KIEN_TRUC_VA_LAP_LUAN_BAO_VE.md
7. Deployment Stamps Pattern - Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/patterns/deployment-stamp
8. Tenancy Models for a Multitenant Solution - Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models
9. PostgreSQL Row-Level Security for AI Agents: A Testable Setup, https://datamcp.app/blog/postgresql-row-level-security-ai-agents
10. Multi-tenancy | Kubernetes, https://kubernetes.io/docs/concepts/security/multi-tenancy/
11. Identity management with Amazon Cognito user pools - AWS, https://aws.amazon.com/blogs/security/saas-authentication-identity-management-with-amazon-cognito-user-pools/
12. Architectural Approaches for Compute in Multitenant Solutions, https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/compute
13. Compute Resource Consolidation Pattern - Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/patterns/compute-resource-consolidation
14. Kubernetes Multi-Tenancy: We Secured 20+ Shared Clusters (Guide), https://tasrieit.com/blog/kubernetes-multi-tenancy-shared-clusters-guide-2026
15. Multi-Tenancy Patterns and Which One Is Correct for You, https://liquidreply.net/blog/multi-tenancy-patterns-and-which-one-is-the-correct-one-for-you

16. The deployment stamp advantage - Parloa, https://www.parloa.com/labs/insights/scaling-parloa-when-the-platform-becomes-the-product/
17. Multitenancy and Azure Storage - Microsoft Learn, https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/service/storage
18. Chapter 21 — Multi-tenancy and isolation models · Security in Depth, https://frontendlabs.xyz/security-in-depth/chapter-21-multi-tenancy-and-isolation-models/
19. Kubernetes Rooms: A Comprehensive Review of Multi-Tenancy, https://www.preprints.org/manuscript/202606.0625
20. Mastering Kubernetes Multi-Tenancy - Signiance Technologies, https://signiance.com/mastering-kubernetes-multi-tenancy/
21. Multi-tenant — co to jest i jak wybrać model SaaS - Digital Vantage, https://www.digitalvantage.pl/artykuly/saas/multi-tenant
22. PostgreSQL RLS - Tenancy for Laravel v4, https://v4.tenancyforlaravel.com/postgres-rls/
23. Multi-tenant data isolation with PostgreSQL Row Level Security - AWS, https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/

Đánh giá: “Kiến Trúc Multi-Tenant SaaS”

> **Lưu ý quan trọng:** Đây là nhận xét độc lập. File nguồn được giữ nguyên; tài liệu này không xác nhận các tính năng đã được cài trong ứng dụng.

1. Kết luận điều hành

Tài liệu bao quát đúng nhiều vấn đề quan trọng: ranh giới tenant, tùy biến nghiệp vụ, cô lập dữ liệu, noisy neighbor, Control Plane và triển khai từ giao diện tới hạ tầng. Hướng **dùng chung sản phẩm và runtime, áp dụng tenant isolation nhiều lớp, bổ sung deployment stamp khi có nhu cầu phân vùng đã đo** là một phương án có cơ sở để tiếp tục xem xét.

Nhưng kết luận “Pooled Multi-Tenancy theo Deployment Stamps là kiến trúc tối ưu duy nhất” hiện **mạnh hơn bằng chứng**. Tài liệu chưa chứng minh workload cần nhiều stamp; một stamp về thực chất vẫn là một deployment pool. Stamp cũng không tự cô lập tenant bên trong stamp, không tự loại bỏ noisy neighbor, và không mở rộng vô hạn. Tài liệu còn có các con số, ví dụ và kết luận an toàn/hiệu năng không dẫn được tới phép đo hoặc nguồn trực tiếp.

Đánh giá tổng thể: tốt về độ bao quát và làm bản đồ chủ đề; cần sửa đáng kể về cách phân biệt các trục kiến trúc, độ chính xác của một số khẳng định, và chuỗi bằng chứng. Nên xem đây là **bản khảo sát/đề xuất kiến trúc**, chưa phải quyết định kiến trúc cuối cùng hay cơ sở để tuyên bố phương án đã được chứng minh tối ưu.

Khuyến nghị cho đề tài hiện tại: trước mắt thiết kế một deployment dùng chung có tenant boundary xuyên suốt (một stamp nếu dùng thuật ngữ stamp), tập trung chứng minh định danh, authorization, cô lập dữ liệu, tùy biến có giới hạn và kiểm soát noisy neighbor. Chỉ đầu tư vào nhiều deployment stamp sau khi có yêu cầu hoặc kết quả đo cho thấy một deployment đã chạm ngưỡng. Phương án này là khuyến nghị có điều kiện; thay đổi so với Thuyết minh đã duyệt, đặc biệt việc không cấp database/tên miền riêng cho từng tenant như tài liệu gốc mô tả, phải được nhóm và thầy chấp thuận trước.

2. Những điểm tài liệu làm tốt

1. **Không giới hạn Multi-tenancy ở database.** Tài liệu nhìn từ UI/routing, business logic, data, storage, Control Plane và hạ tầng. Đây là khung tốt để trả lời câu hỏi của thầy về cái gì dùng chung và cái gì khác nhau theo tenant.

2. **Đã nêu đúng nhóm thách thức của sản phẩm quản lý công việc.** Workflow và field tùy biến, cộng tác, tệp, tải nền và tranh chấp tài nguyên có liên hệ thực tế với Kanban/dự án.

3. **Có tư duy phòng thủ nhiều lớp.** Kết hợp kiểm tra host/token/membership, authorization nghiệp vụ, ràng buộc quan hệ, RLS và namespace cho cache/file là hướng tốt, miễn là mỗi lớp được thiết kế và kiểm thử đúng.

4. Nhận ra Control Plane có thể xử lý tenant lifecycle, cấu hình, placement, telemetry và consumption. Đây là trách nhiệm quản trị dịch vụ, không nên để màn hình quản trị hệ thống lộ board/task/file nghiệp vụ của tenant.

5. Có nêu hạn chế của các lựa chọn. Tài liệu không chỉ nói về lợi ích pool; đã nhắc đến bảo mật, vận hành, migration và noisy neighbor.

3. Những vấn đề cần sửa trước khi dùng làm kết luận

3.1. “Deployment stamp” và “shared schema” là hai trục khác nhau

Deployment stamp/cell mô tả cách đóng gói và nhân bản một nhóm thành phần triển khai để làm scale unit hoặc giới hạn blast radius. Shared schema, schema-per-tenant và database-per-tenant mô tả vị trí/ranh giới của dữ liệu. Hai quyết định này có thể kết hợp theo nhiều cách; không phải bốn lựa chọn loại trừ nhau cùng một tầng.

Ví dụ: một stamp có thể dùng shared database/shared schema; stamp khác có thể chứa database riêng cho một khách hàng đặc biệt. Microsoft cũng mô tả stamps như các bản sao độc lập có capacity hữu hạn; phải tính chi phí, routing và quy trình di chuyển tenant. Họ cảnh báo noisy neighbor vẫn có thể xảy ra bên trong stamp và việc scale bằng stamp gần tuyến tính trong giới hạn, không vô hạn (tham khảo Deployment Stamps pattern, Azure multitenant storage/data approaches).

Cách sửa lập luận: so sánh các kiến trúc tổng thể theo cùng một phạm vi, rồi quyết định riêng từng trục: mức chia sẻ compute, data placement, routing, Control Plane, storage, queue/cache và vận hành. Đừng để “pooled-by-stamp” che khuất phần nào thực sự tạo isolation.

3.2. Stamp không tự động hóa onboarding hoặc giải quyết noisy neighbor

Tài liệu đưa ví dụ stamp phục vụ 50–100 tenant và tự động tạo stamp mới khi gần ngưỡng. Đây chỉ có thể là **giả định minh họa**; tài liệu nguồn của Azure không đưa ra một ngưỡng phổ quát 50–100 tenant. Năng lực stamp phải được xác định bằng workload, SLO, giới hạn dịch vụ và capacity thực tế. Stamp mới chỉ được cấp nếu có pipeline/IaC, tenant catalog và placement logic để làm việc đó.

Một stamp hữu hạn còn chia sẻ CPU, database, queue, cache và network giữa các tenant trong chính stamp ấy. Vì thế, sự cố có thể được khoanh vùng giữa các stamp, nhưng tenant trong cùng stamp vẫn có thể ảnh hưởng nhau. Tuyên bố “cô lập hoàn toàn” hay “mở rộng tuyến tính không giới hạn” cần bỏ. Microsoft mô tả scale unit có giới hạn, chi phí lặp hạ tầng, router/routing bổ sung và tenant migration phức tạp; thậm chí khuyến cáo triển khai ít nhất hai stamp khi thực sự áp dụng để tránh code/config chỉ chạy trong trường hợp một stamp.

Với phạm vi nghiên cứu 3–5 tenant và 10–20 người dùng đồng thời/tenant đã ghi trong Thuyết minh, **nhiều stamp không phải tiền đề cần có để chứng minh Multi-tenancy**. Một stamp hoặc một deployment pooled có thể đủ cho prototype; kiểm thử placement/routing nhiều stamp là nội dung mở rộng nếu được duyệt.

3.3. RLS hữu ích nhưng không phải "hàng rào tối hậu" độc lập với mọi rủi ro

Phần mô tả ENABLE ROW LEVEL SECURITY, FORCE ROW LEVEL SECURITY, runtime role không phải superuser/BYPASSRLS, và đặt tenant context theo transaction là khung kỹ thuật đáng xem xét. Tuy nhiên:

• **Quy định bypass:** PostgreSQL quy định superuser và role có BYPASSRLS luôn bỏ qua RLS; table owner thường cũng bỏ qua, trừ khi dùng FORCE ROW LEVEL SECURITY. FORCE không biến policy thành ranh giới vật lý và không thay việc quản trị quyền đúng.

• **Nguồn gốc Tenant ID:** SET LOCAL/set_config(..., true) giúp cấu hình có hiệu lực cục bộ theo transaction và giảm rủi ro lưu context trên connection được tái sử dụng. Nó không tự xác minh nguồn gốc tenant ID; code phải chỉ nhận context từ xác thực/ủy quyền tin cậy, gắn với transaction đúng và kiểm thử cả rollback/autocommit/connection pool. PostgreSQL cho phép session parameter được đặt trong một số điều kiện bởi user, nên đừng diễn giải GUC đơn thuần như một bí mật chống SQL injection.

• **Chi phí và giới hạn:** RLS có thể là hàng rào bổ sung chống lỗi truy vấn bỏ quên tenant predicate, nhưng không thay authorization, quyền tối thiểu, parameterized SQL, kiểm soát truy vấn đặc quyền hay kiểm thử. Azure cũng lưu ý RLS đòi hỏi truyền đúng user/tenant identity vào data store và có chi phí thiết kế, kiểm thử, bảo trì.

• **Composite Key:** Composite key/foreign key theo (tenant_id, id) có thể ngăn liên kết bản ghi nhầm tenant; nó không tự ngăn người dùng đọc một hàng của tenant khác nếu truy vấn được phép.

Vì vậy, thay "vô hiệu hóa hoàn toàn rủi ro" bằng "giảm một số loại lỗi truy cập chéo khi được cấu hình và kiểm thử đúng; các giới hạn còn lại được nêu rõ". Chỉ ghi "đã bật RLS" khi migration, database role, policy và test hiện hành xác nhận.

3.4. Silo và Kubernetes không đồng nghĩa cô lập tuyệt đối

Tài liệu đôi lúc gọi Silo Compute là "cô lập tuyệt đối", hoặc nói contention CPU/RAM bị triệt tiêu. Điều này phụ thuộc ranh giới thực tế: container riêng nhưng chung node/kernel/network không ngang với VM riêng hay cluster riêng. Database riêng cũng không cô lập API, DNS, network, broker hay control plane vẫn dùng chung.

Phần Kubernetes nên được mô tả như **một phổ mức cô lập**, không phải hai nhãn mềm/cứng có nghĩa phổ quát. Namespace chỉ là một phần; cần RBAC, ResourceQuota, requests/limits, NetworkPolicy được CNI hỗ trợ, secrets, storage và chính sách admission phù hợp. Quota cũng không bao phủ tất cả loại tranh chấp (ví dụ network).

3.5. Các con số và từ tuyệt đối chưa có cơ sở

Những nội dung sau cần chuyển thành giả định cần đo hoặc loại bỏ nếu không tìm được nguồn/benchmark phù hợp:

• 50–100 tenant mỗi stamp;

• Rate/timeout ví dụ 60 request/phút, truy vấn tự hủy sau 3 giây, presigned URL “vài phút” như mặc định;

• “Chi phí CDN tiệm cận 0”, “phát hành cho 100% người dùng không downtime”, “RLS overhead” hoặc catalog bloat ở mức định lượng/khẳng định chung;

• “Mở rộng tuyến tính không giới hạn”, “nguy cơ bằng 0”, “triệt tiêu”, “hoàn toàn không ảnh hưởng”.

Những giá trị có thể xuất hiện trong thiết kế mẫu, nhưng phải gắn nhãn ví dụ; ngưỡng sản xuất cần lấy từ SLO, threat model, workload và phép đo.

3.6. So sánh schema/database còn khái quát hóa

• **Database-per-tenant:** Cần nói rõ đó là database logic trên cùng server, DB server/cluster riêng hay toàn bộ hạ tầng riêng; chi phí và isolation khác nhau đáng kể.

• **Schema-per-tenant:** Không mặc định sẽ làm PostgreSQL suy giảm nghiêm trọng ở một con số tenant cố định. Catalog size, migration duration và query performance phải benchmark với số bảng/index và phiên bản PostgreSQL cụ thể.

• **Shared schema:** Giảm số đường migration nhưng làm tăng yêu cầu kỷ luật về tenant context, query scoping, index tenant-first, backup/export/delete tenant và điều tra sự cố.

• **Tính khách quan:** Không thể chỉ từ kiến trúc suy ra pooled “tối ưu nhất” về hiệu năng hay chi phí. Phải so sánh cùng dữ liệu, workload, phần cứng, connection policy và số lần lặp.

3.7. Typed EAV/Value Table chưa được chứng minh là lựa chọn tốt nhất

Typed metadata/value tables giữ cấu trúc DDL ổn định và có thể giúp lọc/sắp xếp theo kiểu dữ liệu khi được thiết kế phù hợp. Đổi lại, truy vấn thường cần join/pivot, mỗi kiểu field tạo logic index riêng, cần constraint để bảo đảm kiểu và cardinality; báo cáo/điều kiện tùy biến có thể phức tạp.

JSONB không mặc định chậm hơn trong mọi tình huống: nếu tập field thưa, truy vấn chủ yếu theo key và workload phù hợp, JSONB cùng loại index/biểu thức thích hợp có thể là phương án hợp lý. EAV, JSONB và cột quan hệ chuẩn nên được chọn theo các truy vấn dự kiến, validation, sort/range/filter, export/report, khả năng migration và phép benchmark — không theo tuyên bố tổng quát “EAV mất index” hoặc “typed table tối ưu tốc độ”.

Với app hiện đã có tùy biến DDL theo placement (theo mô tả trong các tài liệu dự án), trước khi đổi mô hình cần xác định migration dữ liệu, giữ tương thích API/UI, backup/restore và test cô lập; không biến phần này thành refactor bắt buộc nếu nó không nằm trong phạm vi nghiên cứu được duyệt.

3.8. Presigned URL không tự giới hạn theo prefix tenant

Ví dụ object key dạng tenants/{tenant_id}/... là quy ước namespace tốt nếu server tự tạo key và kiểm tra quyền trước khi ký. Nhưng presigned URL của S3 cấp quyền tạm thời cho một

object/operation cụ thể; bản thân URL không tự hiểu tenant prefix hay membership của ứng dụng. Vì vậy cần authorize trước khi tạo URL, không nhận object key tùy ý từ client, giới hạn method/content type/size khi có thể, chọn expiry phù hợp và không log/chia sẻ URL nhạy cảm.

3.9. Control Plane đang bị mô tả như một hệ thống lớn hơn nhu cầu trước mắt

Control Plane là tập trách nhiệm quản trị tenant/resource/lifecycle/configuration/telemetry/consumption. Nó không bắt buộc phải là microservice hay database hoàn toàn riêng trong prototype. Azure nêu rõ không có một mẫu Control Plane duy nhất; với SaaS fully shared và ít tenant, catalog đơn giản cùng quy trình/script onboarding có thể đủ. Nhiều control plane global/stamp làm tăng độ phức tạp; chỉ tách khi nhu cầu cần.

Đối với demo app hiện tại, Control Plane nên làm rõ các chức năng như tenant catalog/state, quota/policy, usage/health/incident, cảnh báo và hành động vận hành. Không hiển thị task, board, tệp hay nội dung nghiệp vụ của tenant trong admin hệ thống. Một tenant noisy nên tạo incident/telemetry rõ ràng và cho admin áp dụng policy; đây là cảnh báo/quản trị, không phải bằng chứng rằng các tenant còn lại không bị ảnh hưởng.

Rate limit, concurrency bulkhead, query timeout, pagination và fair queue là các biện pháp bổ sung khác tầng. Rate limit chỉ ngăn thêm một số request; không thu hồi truy vấn đã chạy. Timeout giới hạn thời gian truy vấn chứ không tự đảm bảo fairness; keyset pagination phù hợp với một số kiểu duyệt dữ liệu nhưng không thay thế mọi API pagination. Ngưỡng phải xuất phát từ tải đo.

4. So sánh lại bốn kiến trúc tổng thể

Phương án	Bản chất	Ưu điểm chính	Hạn chế/rủi ro	Mức phù hợp hiện tại
A. Full-stack silo	Compute/data/network/UI tách phần lớn theo tenant; cần định nghĩa cụ thể tách tới đâu	Ranh giới dữ liệu/tài nguyên mạnh hơn ở các thành phần thực sự dedicated; rollout hoặc phục hồi riêng dễ hơn	Tốn tài nguyên và vận hành; còn shared global dependency thì blast radius vẫn tồn tại	Không làm default cho 3–5 tenant; dùng khi compliance, region hoặc hợp đồng cần dedicated isolation

B. Một pooled deployment	Nhiều tenant chia sẻ artifact/runtime và thường cả data store, có tenant-aware isolation	Ít bản triển khai, hiệu quả chia sẻ và đơn giản để bắt đầu	Blast radius rộng; cần isolation, metering, noisy-neighbor controls và capacity monitoring	Baseline thực dụng để xây/test core Multi-tenancy
C. Bridge cố định theo gói	Một số tenant dùng pool, một số schema/database riêng theo gói	Có thể bán/đáp ứng nhiều isolation tier nếu business thực sự yêu cầu	Nhiều code path, provisioning/migration/backup/restore và dễ gắn capability nghiệp vụ sai với data placement	Không khuyến nghị nếu mục tiêu nghiên cứu là chọn một kiến trúc sản phẩm duy nhất; vẫn có thể làm baseline
D. Pooled theo stamps/cells	Nhân bản data-plane thành các đơn vị có capacity; nhiều tenant trong mỗi stamp, có resolver/catalog	Chia partition, rollout hoặc sự cố ở phạm vi stamp; có đường scale-out và region placement	Chi phí lặp hạ tầng, cần routing/catalog/capacity, aggregate observability và tenant move; tenant trong cùng stamp còn noisy neighbor	Kiến trúc tăng trưởng tiềm năng, chưa cần triển khai nhiều stamp nếu chưa chứng minh giới hạn một deployment

Điểm mấu chốt: D không phải "pool cộng RLS" một cách tự nhiên; D là mô hình vận hành/scale-unit. Có thể dùng data placement pool, schema hoặc silo bên trong stamp tùy yêu cầu. Tương tự, shared schema không phải đối thủ trực tiếp của deployment stamps.

5. Đề xuất kiến trúc cho app và đề tài

5.1. Mục tiêu kiến trúc cần chứng minh

App phải cho thấy tenant là ranh giới tổ chức xuyên suốt, chứ không chỉ là nhiều người dùng hoặc role. Dùng chung code/hạ tầng không mâu thuẫn với Multi-tenancy; điều quan trọng là mỗi request được gắn tenant tin cậy, mọi thao tác/quan hệ dữ liệu được authorize trong tenant, cấu hình và hạn mức được quản lý theo tenant, và lỗi/tải được theo dõi theo tenant.

5.2. Phương án khuyến nghị giai đoạn nghiên cứu

Tầng/trục	Dùng chung	Riêng/được cấu hình theo tenant	Cơ chế cần chứng minh

Cloud/network	Một môi trường nghiên cứu, ingress/TLS và runtime dùng chung	Tenant hostname/slug và trạng thái tenant trong catalog	Không tin host/header từ client nếu chưa được proxy tin cậy xác thực; kiểm tra host–tenant–membership
Presentation	Một artifact SPA/CDN	Branding nhẹ, bố cục/field/workflow config có giới hạn	UI nhận config theo tenant; chỉ là UX, không phải security boundary; cấm tenant cung cấp JS tùy ý
Business/API	Một codebase và API stateless	Membership, policy, quota, workflow config và tenant context	Context lấy từ identity/membership tin cậy; kiểm tra ở mỗi use case; async jobs truyền tenant id và kiểm soát idempotency
Relational data	Một PostgreSQL database/schema cho đường pooled nếu scope/decision được duyệt	Mỗi bản ghi, khóa quan hệ, query scope, export và retention thuộc tenant	Tenant-first unique/foreign keys; app auth; RLS chỉ khi migration/runtime role/context có test chứng minh
Custom data	Một engine/metadata schema được kiểm soát	Field/workflow definitions và values từng tenant	Chốt theo các truy vấn thực tế; validate kiểu/quyền/sort/filter; không cho DDL/SQL tùy ý
Cache/event/file	Dịch vụ dùng chung ở prototype	Tenant namespace server-generated, quota/retention/access	Cache key và message chứa tenant context; object key do server tạo; authorize trước presign
Control Plane	Catalog và công cụ vận hành tối thiểu	Tenant state/quota/policy/usage/incident	Chỉ dữ liệu quản trị; không lộ board/task/file; audit thao tác và cảnh báo
Noisy neighbor	CPU/DB/queue có thể chia sẻ	Budget/limit/incident theo tenant	Đo baseline và noisy workload; request rate + concurrency/query/queue controls; theo dõi ảnh hưởng của tenant lành

Đây là kiến trúc **một pooled deployment (có thể gọi là một stamp)**, không đòi hỏi sẵn nhiều stamp. Không tạo Codespace/VPS/database riêng cho từng tenant trừ khi Thuyết minh sau khi sửa, yêu cầu bảo mật/tuân thủ hoặc kết quả đo dẫn tới quyết định đó.

5.3. Lộ trình mở rộng có điều kiện

Chỉ xem xét multi-stamp khi benchmark cho thấy một deployment chạm capacity/SLO, cần phân vùng địa lý, hoặc phải giảm blast radius. Khi ấy cần tối thiểu:

1. tenant_id -> stamp_id catalog và resolver HA; quy trình cấp stamp bằng IaC.

2. Capacity model theo throughput, concurrent work, DB connections/storage/queue, không chỉ đếm tenant.

3. Health/capacity monitoring từng stamp và tổng hợp telemetry toàn dịch vụ.

4. Quy trình route/drain/move tenant, backup/restore và xử lý lỗi provisioning.

5. Ít nhất hai stamp trong test/pilot nếu muốn kiểm tra routing, giới hạn blast radius và hoạt động khi một stamp lỗi.

6. Đánh giá mức sẵn sàng của tài liệu nguồn

Tiêu chí	Nhận xét	Mức hiện tại
Bao phủ tầng kiến trúc	Rộng, chạm đủ UI/API/data/control plane/infra	**Tốt**
Phân biệt Multi-user/Multi-tenant	Có ý đúng nhưng đôi lúc mô tả tenant như ranh giới duy nhất, bỏ qua mixed/shared dependencies	**Khá**
So sánh phương án	Có nhiều chiều, nhưng trộn data placement với deployment scale unit và có kết luận vượt quá trade-off	**Cần sửa**
Độ chính xác kỹ thuật	Một số điểm đúng, song còn tuyệt đối hóa RLS, silo, stamp, K8s và hiệu năng	**Cần sửa đáng kể**
Chất lượng dẫn chứng	Danh mục đánh số trộn tài liệu chính thức, blog, tài liệu nội bộ; các nguồn nội bộ trích dẫn vòng lẫn nhau	**Yếu cho báo cáo học thuật**
Gắn với app hiện tại	Chưa tách rõ “đã cài”, “đề xuất”, “ví dụ minh họa”, “chưa đo”; có nguy cơ mô tả kiến trúc tương lai như hiện trạng	**Chưa sẵn sàng**
Kết luận cuối	Hướng pooled có lý do để thử, nhưng chưa chứng minh stamp là cần thiết hoặc pooled thắng các baseline	**Chưa đủ để chốt**

Trạng thái khuyến nghị: giữ tài liệu gốc làm bản tổng hợp tham khảo; trước khi dùng bảo vệ, sửa theo các finding ở mục 3, thay các khẳng định không có nguồn/đo, xác minh từng trạng thái triển khai và cập nhật nguồn. File review này không ghi điểm số tổng vì chưa có rubric hoặc dữ liệu thực nghiệm; mục trên là đánh giá mức sẵn sàng tài liệu, không phải điểm chất lượng app.

7. Các quyết định nhóm cần chốt trước khi sửa thiết kế

1. Phạm vi được duyệt: Thuyết minh hiện tại có mô tả cấp database/tên miền riêng trong onboarding. Nhóm có được phép đổi sang shared database/domain + logical tenant namespace không? Nếu có, sửa Thuyết minh/ADR trước khi gọi đây là kiến trúc chính thức.

2. Bài toán tenant cụ thể: Tenant là trường/khoa, lớp/nhóm nghiên cứu hay workspace tổ chức? Một user có thể thuộc nhiều tenant? Ai là tenant admin, ai là platform admin?

3. Customization có giá trị nghiệp vụ: Chọn field/task schema và workflow/approval nào có thể vận hành, kiểm thử được; không coi logo/màu là bằng chứng duy nhất.

4. Data placement của prototype: Ba mode hiện tại là baseline nghiên cứu hay lựa chọn tính năng? Nếu baseline, định nghĩa cách giữ cùng schema, seed, workload và migration để so sánh công bằng.

5. Có thực sự thử stamps không: Nếu không được duyệt mở rộng workload, để stamp ở sơ đồ kiến trúc dài hạn và không tuyên bố đã benchmark.

6. Evidence: Protocol, seed data, workload, warm-up, số lần lặp, p50/p95/p99, throughput, error rate, CPU/RAM, DB pool/latency, queue lag và isolation tests; lưu raw data cùng cấu hình.

8. Nguồn kỹ thuật nên dùng để hiệu chỉnh bản gốc

• **Microsoft Azure — Tenancy models:** isolation spectrum, cost/performance trade-offs, shared and dedicated components. (https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models)

• **Microsoft Azure — Deployment Stamps pattern:** scale units, finite capacity, extra costs, routing, tenant moves and blast-radius boundaries. (https://learn.microsoft.com/en-us/azure/architecture/patterns/deployment-stamp)

• **Microsoft Azure — Control plane considerations:** control-plane responsibilities depend on architecture; tenant catalog, lifecycle, telemetry, consumption and placement. (https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/control-planes)

• **Microsoft Azure — Control plane approaches:** simple/manual approaches can suit a small tenant count; multiple control planes add complexity. (https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/control-planes)

• **Microsoft Azure — Storage and data approaches:** shared DB/RLS trade-offs, stamp use, noisy neighbor and data-placement choices. (https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/storage-data)

• **PostgreSQL — Row Security Policies & configuration settings:** RLS behavior, owner/superuser/BYPASSRLS limitations and session settings. (https://www.postgresql.org/docs/current/ddl-rowsecurity.html, https://www.postgresql.org/docs/current/config-setting.html)

• **Kubernetes — Multi-tenancy:** namespaces are one control, with additional access, quota, network and data-plane isolation requirements. (https://kubernetes.io/docs/concepts/security/multi-tenancy/)

• **AWS S3 — Presigned URLs:** temporary access to a specific object; application authorization and object-key design remain necessary. (https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html)

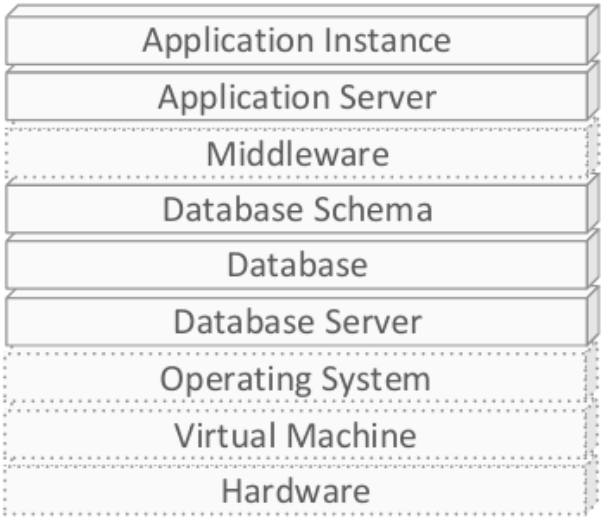


Figure 2: Software Stack: The different system levels where multi-tenancy can be applied to share resources.

Table 5: Difference overview for multi-tenant, multi-user, and multi-instance systems.

Multi	**Shared resources**	**Configurable at runtime**
-tenant	Yes	Fully
-user	Yes	Partly
-instance	Possibly	Possibly

1. Định Nghĩa Và Phân Định Kiến Trúc: MI, MU Và MT

Trong kỷ nguyên điện toán đám mây và mô hình Phần mềm dạng Dịch vụ (SaaS), việc lựa chọn kiến trúc hệ thống giữ vai trò quyết định đối với khả năng mở rộng, chi phí vận hành và mức độ an toàn thông tin. Để xây dựng một hệ thống tối ưu, việc làm rõ các khái niệm kiến trúc căn bản như Multi-Tenant (MT), Multi-Instance (MI) và Multi-User (MU) là điều kiện tiên quyết.

Multi-Instance (MI) là mô hình kiến trúc trong đó mỗi tenant được cung cấp một thể hiện ứng dụng (software instance) riêng biệt, nhưng các thể hiện này có thể được triển khai trên cùng một hạ tầng phần cứng hoặc tài nguyên ảo hóa dùng chung. Khác với MT thuần túy (vốn chia sẻ cả ứng dụng lẫn cơ sở dữ liệu), MI tách biệt mã nguồn đang chạy (runtime execution) của từng tenant. Mô hình này thường là bước chuyển tiếp cho các ứng dụng truyền thống (legacy) khi chuyển hướng sang SaaS mà chưa kịp tái cấu trúc mã nguồn để hỗ trợ đa tenant.

Multi-User (MU) chỉ năng lực của một ứng dụng trong việc cho phép nhiều người dùng (users) đăng nhập và thao tác đồng thời. MU là một thuộc tính mang tính chức năng của phần mềm, hoàn toàn không quy định hay phản ánh kiến trúc hạ tầng bên dưới. Một ứng dụng xử lý văn bản đơn lẻ cài trên máy chủ nội bộ hoặc một ứng dụng nhắn tin cá nhân đều là hệ thống Multi-User, nhưng chúng không phải là Multi-Tenant vì không có sự phân tách ranh giới giữa các tổ chức khách hàng khác nhau trên cùng một instance dùng chung.

Multi-Tenancy (MT) là một nguyên lý kiến trúc phần mềm trong đó một thể hiện (instance) duy nhất của ứng dụng chạy trên hạ tầng của nhà cung cấp dịch vụ để phục vụ nhiều tổ chức khách hàng khác nhau, được gọi là các "tenant" (bên thuê). Trong mô hình MT thuần túy, các tenant chia sẻ chung tài nguyên phần cứng, không gian tính toán, ứng dụng và cơ sở dữ liệu. Mặc dù chia sẻ tài nguyên vật lý và logic, hệ thống vẫn đảm bảo mỗi tenant có một môi trường hoạt động độc lập, an toàn và có khả năng tùy biến cấu hình riêng biệt như thể ứng dụng đang vận hành trên một hạ tầng dành riêng.

Table 5: Difference overview for multi-tenant, multi-user, and multi-instance systems.

Multi	Shared resources	Configurable at runtime
-tenant	Yes	Fully
-user	Yes	Partly
-instance	Possibly	Possibly

To clarify, we have conceptualized a system, such that we can reuse it for the definition later in Figure 2. The dotted boxes are parts of the system that are not influenced by software level multi-tenancy. Efforts exist to apply multi-tenancy at the middle-ware level (Strauch et al., 2013), but we did not explicitly analyse this, for the sake of creating a high level general definition. link báo

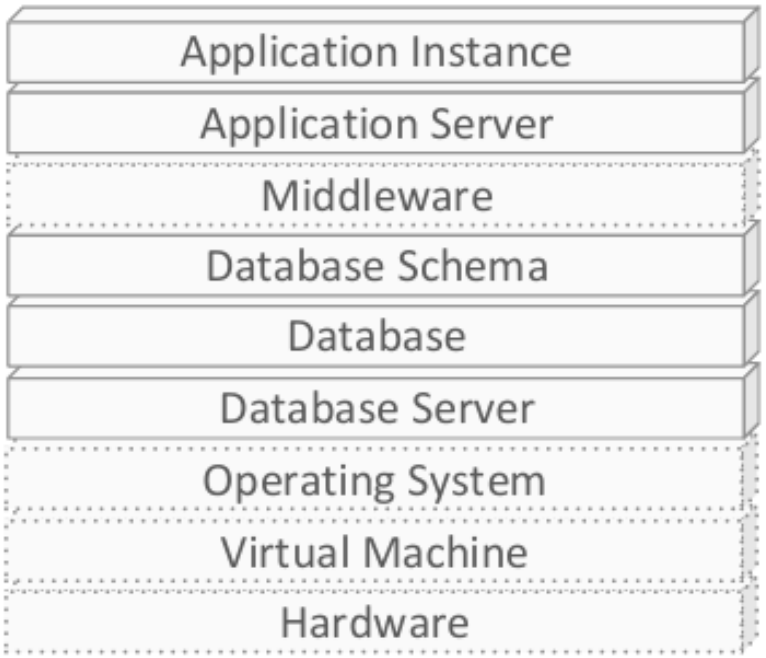
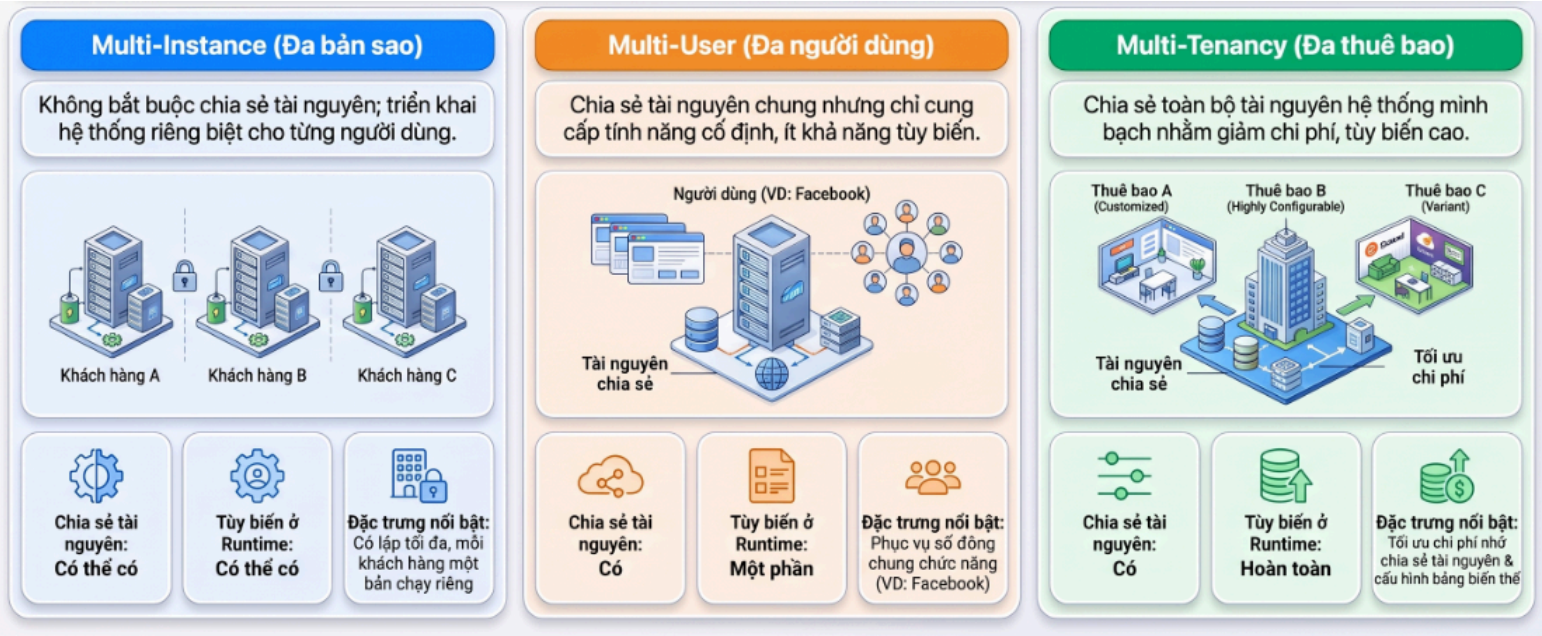


Figure 2: Software Stack: The different system levels where multi-tenancy can be applied to share resources.



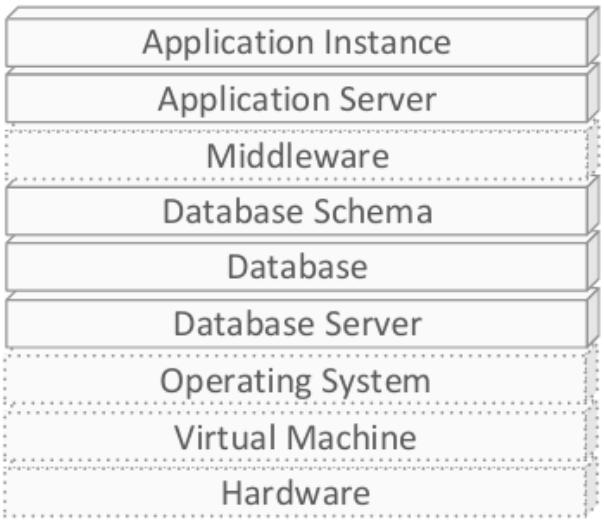


Figure 2: Software Stack: The different system levels where multi-tenancy can be applied to share resources.

Tiêu Chí Đánh Giá	Multi-Tenant (MT)	Multi-Instance (MI)	Multi-User (MU)
Chia sẻ Ứng dụng (App Shared)	Có (Dùng chung 1 instance duy nhất)	Không (Mỗi tenant 1 instance riêng)	Có thể có hoặc không tùy kiến trúc bên dưới

Chia sẻ Cơ sở dữ liệu (DB Shared)	Thường dùng chung (Shared DB / Shared Schema)	Thường tách biệt theo từng instance	Dùng chung một cơ sở dữ liệu duy nhất
Khả năng Cấu hình (Configurability)	Cấu hình linh hoạt qua Metadata (không sửa code)	Cấu hình độc lập trên từng instance riêng	Hạn chế, cấu hình theo thiết lập cá nhân
Tổng Chi Phí Sở Hữu (TCO)	Tối ưu nhất do hiệu ứng quy mô hạ tầng	Trung bình đến cao (tốn phí quản lý nhiều instance)	Phụ thuộc vào kiến trúc ST hay MT bên dưới
Độ Phức Tạp Cài Đặt / Nâng Cấp	Rất thấp (Chỉ cần nâng cấp 1 instance duy nhất) [cite: 1]	Cao (Phải triển khai bản vá cho từng instance)	Phụ thuộc vào mô hình triển khai hạ tầng
Mức Độ Cách Ly Dữ Liệu (Isolation)	Cách ly logic (Phụ thuộc vào thuật toán & DB policy)	Cách ly ứng dụng, có thể chung hoặc riêng DB	Không có khái niệm cách ly tổ chức (Tenant)

---

2. Bản Chất Sự Khác Biệt Giữa Multi-Tenant (MT) Và Multi-User (MU)

Về mặt bản chất hệ thống, Multi-Tenancy bao hàm Multi-User, nhưng Multi-User không đương nhiên tạo thành Multi-Tenancy.

Tenant được định nghĩa là một thực thể tổ chức (doanh nghiệp, tập đoàn, công ty con, hoặc tổ chức phi lợi nhuận) đăng ký sử dụng giải pháp SaaS. Trong mỗi tenant lại bao gồm một tập hợp nhiều người dùng (users) đại diện cho các nhân viên, quản lý hoặc khách hàng của tổ chức đó. Do đó, kiến trúc MT xử lý bài toán quản lý phân quyền hai lớp (Two-Tier Management): lớp tổ chức (Tenant Level) để quản lý ranh giới dữ liệu, gói dịch vụ, cấu hình hệ thống và thanh toán; và lớp người dùng (User Level) nằm bên trong từng tenant để quản lý định danh và quyền hạn cá nhân.

Ngược lại, hệ thống MU thuần túy chỉ có một lớp phẳng (Single-Tier) quản lý các tài khoản người dùng cá nhân. Hệ thống MU không có khái niệm về ranh giới tổ chức dùng chung giữa một nhóm người dùng, cũng như không cung cấp khả năng cô lập dữ liệu hay tùy biến quy trình nghiệp vụ độc lập cho từng nhóm khách hàng doanh nghiệp.

Trong các hệ thống MU truyền thống, cơ chế kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) là phương thức tiêu chuẩn. RBAC xác định một người

dùng có vai trò gì, từ đó quyết định người dùng đó có quyền thực hiện hành động trên một tài nguyên phần mềm hay không. Tuy nhiên, RBAC chỉ giải quyết bài toán cấp phép chức năng cá nhân mà không thể tự động giới hạn không gian dữ liệu giữa các tổ chức.

Trong kiến trúc Multi-Tenant, việc phân quyền không chỉ dựa vào RBAC mà bắt buộc phải dựa vào Ngữ Cảnh Tách Biệt Tenant (Tenant Isolation Context) kết hợp với Kiểm Soát Truy Cập Dựa Trên Thuộc Tính (Attribute-Based Access Control - ABAC), Policy-Based Access Control (PBAC), và Động Cơ Phân Quyền Hạn Mức (Entitlement Engine). Quyết định truy cập trong hệ thống MT tuân theo ma trận đánh giá logic phức hợp:

$$AccessGranted = ValidateTenantContext \land ValidateRBACPermission \land ValidateTenantEntitlement$$

Trong đó, bước kiểm tra ngữ cảnh tenant có vai trò tiên quyết. Hệ thống sẽ kiểm tra xem dữ liệu được yêu cầu có khớp chính xác với ngữ cảnh tenant hiện tại của không gian yêu cầu hiện tại hay không. Nếu không trùng khớp, truy cập bị từ chối ngay lập tức ở tầng hạ tầng hoặc cơ sở dữ liệu mà không cần xét tiếp các quyền hạn RBAC khác. Sau đó, hệ thống mới kiểm tra quyền thao tác nghiệp vụ của người dùng nội bộ trong tenant (như Admin, Editor, Viewer) và kiểm tra gói cước mà tenant đã mua (Gói Basic, Pro, Enterprise) có cho phép sử dụng tính năng này hay không.

Ba Ranh Giới Cốt Lõi Giữa Các Tenant

Việc thiết lập thành công kiến trúc MT đòi hỏi sự phân tách rõ ràng trên ba ranh giới chuyên biệt bao gồm ranh giới nghiệp vụ, ranh giới truy cập, và ranh giới dữ liệu.

Ranh giới nghiệp vụ (Business Boundary) định hình cách thức ứng dụng thích ứng với các quy trình làm việc khác nhau của từng tenant. Hai tenant hoạt động trên cùng một phiên bản ứng dụng có thể có quy trình phê duyệt đơn hàng hoàn toàn khác nhau, các trường dữ liệu tùy biến (custom fields) riêng biệt, cũng như thỏa thuận chất lượng dịch vụ (SLA) khác nhau [cite: 1, 5]. Hệ thống phải đảm bảo việc thay đổi quy tắc nghiệp vụ của một tenant tuyệt đối không làm ảnh hưởng đến luồng xử lý dữ liệu của tenant khác.

Ranh giới truy cập (Access Boundary) quản lý luồng định danh và đường truyền dữ liệu đi vào hệ thống. Mỗi tenant có ranh giới truy cập riêng thông qua định danh tên miền (như portal.tenantA.com), cơ sở hạ tầng xác thực riêng, và không gian chứa mã định danh tenant đã được ký số mã hóa.

Ranh giới dữ liệu (Data Boundary) là ranh giới quan trọng nhất, quyết định sự an toàn của hệ thống. Dữ liệu của một tenant (bao gồm dữ liệu quan hệ, dữ liệu phi cấu trúc, chỉ mục tìm kiếm, bộ nhớ đệm cache và tệp tin lưu trữ) phải được cô lập logic hoặc vật lý để ngăn chặn tuyệt đối tình trạng truy nhập chéo dữ liệu. Sự vi phạm ranh giới dữ liệu dù chỉ một bản ghi đơn lẻ cũng có thể dẫn đến hậu quả pháp lý nghiêm trọng và phá hủy uy tín của nhà cung cấp SaaS.

---

3. Các Đặc Trưng Cốt Lõi Của Kiến Trúc Multi-Tenant

Khả Năng Tùy Biến Cấu Hình

Trong kiến trúc Multi-Instance, khi một khách hàng yêu cầu tính năng riêng, đội ngũ phát triển thường rẽ nhánh mã nguồn (code branching) hoặc duy trì các bản build riêng biệt.

Ngược lại, đối với Multi-Tenant, nếu phải sửa đổi mã nguồn cho từng khách hàng thì nó không còn là MT nữa. Mọi sự khác biệt về giao diện, quy trình, trường dữ liệu mở rộng và quy tắc tính toán phải được trừu tượng hóa dưới dạng Metadata (Dữ liệu tả dữ liệu). Hệ thống sẽ đọc cấu hình metadata của tenant đang thực thi tại thời điểm runtime để dựng giao diện và thực thi luồng xử lý tương ứng.

Co giãn linh hoạt & Phân bổ tài nguyên động

Hệ thống tự động điều chỉnh hạn mức và di chuyển tài nguyên để xử lý các đợt tăng vọt lưu lượng (order flow pulse) mà không cần dự phòng tài nguyên quá mức gây lãng phí

An toàn dữ liệu & Bảo mật đa tầng
Đảm bảo dữ liệu riêng tư của từng tổ chức được bảo mật chống lại cả các mối đe dọa bên ngoài lẫn rủi ro rò rỉ chéo bên trong hạ tầng dùng chung

Quản lý tải, Hiện tượng "Láng giềng ồn ào" & Bán kính ảnh hưởng
Ngăn chặn tình trạng một tenant tăng đột biến lưu lượng chiếm dụng toàn bộ CPU, RAM, I/O đĩa hoặc kết nối CSDL, làm suy giảm dịch vụ của các tenant khác dùng chung hạ tầng (*Noisy Neighbor Effect*)

Phân cụm tenant thành các nhóm nhỏ (Pods/Cell-based architecture) để khi xảy ra sự cố phần cứng hay lỗi phần mềm, ảnh hưởng chỉ giới hạn trong một nhóm tenant thay vì làm sập toàn bộ hệ thống

Lan Truyền Ngữ Cảnh Tenant
Trong toàn bộ vòng đời của một yêu cầu (request lifecycle), định danh của tenant (Tenant Context) phải được lan truyền liên tục và trong suốt qua tất cả các tầng kiến trúc: từ API Gateway, các dịch vụ xử lý microservices, các tác vụ chạy ngầm bất đồng bộ (background queued jobs), cho đến tầng lưu trữ database và cache. Nếu ngữ cảnh tenant bị mất ở bất kỳ tầng nào (ví dụ: một luồng bất đồng bộ gửi email quên truyền tenant_id), dữ liệu có nguy cơ bị xử lý sai không gian hoặc rò rỉ sang tenant khác.

Phục hồi sự cố & Sao lưu cấp Tenant
Khả năng sao lưu, khôi phục hoặc chuyển đổi dự phòng (failover) cho một tenant cụ thể mà không làm gián đoạn hay phải khôi phục toàn bộ hệ thống của các tenant khác

Quản Lý Tải Vùng Và Bán Kính Ảnh Hưởng
Chia sẻ tài nguyên giúp giảm chi phí hạ tầng trên mỗi tenant xuống mức tối thiểu nhờ tận dụng công suất nhàn rỗi của hệ thống. Tuy nhiên, việc chia sẻ này nảy sinh hiện tượng "hàng xóm ồn ào" (Noisy Neighbor Effect), khi một tenant đột ngột tăng đột biến lưu lượng truy cập hoặc thực thi các truy vấn nặng có thể chiếm dụng toàn bộ tài nguyên CPU, RAM, I/O của cơ sở dữ liệu hoặc băng thông mạng, làm ảnh hưởng đến dịch vụ của các tenant khác đang dùng chung hạ tầng. Hệ thống MT bắt buộc phải tích hợp các cơ chế Rate Limiting, Throttling và Quota Enforcement theo tenant để duy trì sự ổn định. Đồng thời, các kiến trúc sư phải tính toán bán kính ảnh hưởng (Blast Radius) để đảm bảo một sự cố sập

ứng dụng hay lỗi phần mềm trên instance dùng chung không làm gián đoạn dịch vụ của tất cả khách hàng cùng lúc.

Nâng cấp ứng dụng & Quản lý vòng đời
Tự động hóa hoàn toàn các khâu khởi tạo (*provisioning*), nâng cấp phần mềm không gián đoạn (*zero-downtime upgrade*), di trú (*migration*) và hủy bỏ (*decommissioning*) tenant.

- **Nâng cấp không gián đoạn (Rolling / Blue-Green Upgrades):** Khi cập nhật phiên bản phần mềm mới, hệ thống triển khai theo mô hình Blue-Green hoặc Canary[44]. Phiên bản ứng dụng mới phải giữ tính tương thích ngược (*backward compatibility*) với lược đồ metadata của tất cả các tenant[9].
- **Tự động hóa bằng Infrastructure as Code (IaC):** Sử dụng các mẫu thiết lập chuẩn hóa giúp rút ngắn thời gian bàn giao thể hiện ứng dụng/CSDL cho tenant mới (từ 15 phút xuống chỉ còn 3-5 phút)

Nghiên Cứu Kiến Trúc Phần Mềm Đa Thuê Bao và Mô Hình Cô Lập

1. Định Nghĩa Ba Khái Niệm Cốt Lõi: Service, Tenant và Instance

Để xây dựng một lập luận kiến trúc chặt chẽ, yêu cầu tiên quyết là phải định nghĩa chính xác các thuật ngữ nền tảng, loại bỏ sự nhập nhằng ngữ nghĩa thường gặp giữa góc nhìn người dùng cuối và góc nhìn kỹ thuật hệ thống[2].

1. Trong ngữ cảnh hệ thống quản lý công việc theo mô hình Kanban phục vụ môi trường giáo dục đại học, khái niệm Dịch vụ (Service) được định nghĩa là:

- Là tập hợp các tính năng và giá trị phần mềm được cung cấp qua Internet (như quản lý không gian làm việc, lập kế hoạch Kanban, phân quyền, tự động nhắc hạn).

Góc nhìn: Người dùng chỉ trải nghiệm kết quả và giá trị mà dịch vụ mang lại chứ không cần biết về hệ thống máy chủ bên dưới.

Điểm cốt lõi cần phân biệt là khách hàng thụ hưởng và trả phí (hoặc được cấp quyền sử dụng) cho Service—tức là trải nghiệm và kết quả nghiệp vụ—chứ không trực tiếp quản lý hay nhận biết cấu trúc máy chủ vật lý bên dưới đang vận hành dịch vụ đó[2].

2. Khác với khái niệm người dùng cá nhân (User), Thuê bao (Tenant):

- Là một đơn vị tổ chức, tập thể, hoặc chủ thể đại diện (ví dụ: một Bộ môn, Phòng thí nghiệm, nhóm nghiên cứu, hay lớp học phần đồ án tại Đại học Cần Thơ) đứng ra đăng ký không gian làm việc (Workspace).

Góc nhìn: Có ranh giới quản trị riêng, toàn quyền quản lý thành viên, tùy chỉnh giao diện và sở hữu tuyệt đối dữ liệu dự án của đơn vị mình mà không bị các Tenant khác can thiệp.

Mỗi Tenant sở hữu ranh giới quản trị riêng biệt, trong đó chủ thuê bao có toàn quyền mời hoặc thu hồi quyền truy cập của thành viên, thiết lập vai trò nội bộ, tùy chỉnh nhận diện thương hiệu như logo và màu sắc của đơn vị, cũng như nắm quyền sở hữu tuyệt đối đối với dữ liệu dự án bên trong không gian đó mà không bị bất kỳ thuê bao nào khác nhìn thấy hay can thiệp[1].

3. Để vận hành Service phục vụ các Tenant, hệ thống cần đến Thể hiện (Instance), được định nghĩa là:

- Là thực thể vật chất gồm các tài nguyên tính toán vật lý hoặc ảo hóa (tiến trình ứng dụng, RAM heap, luồng CPU, cơ sở dữ liệu) được khởi tạo để thực thi mã nguồn.

Góc nhìn: Đây là lớp hạ tầng kỹ thuật phía dưới để vận hành dịch vụ phục vụ cho các Tenant.

Nếu Service là khái niệm ở mặt phẳng giá trị nghiệp vụ và Tenant là khái niệm ở mặt phẳng ranh giới tổ chức, thì Instance chính là thực thể vật chất ở mặt phẳng vận hành kỹ thuật (runtime execution)[2].

4. Nếu *Service* mới chỉ là tập hợp tính năng trừu tượng trên giấy hoặc trong mã nguồn, và *Tenant* là ranh giới tổ chức sử dụng, thì ***Deployment*** chính là cầu nối chuyển đổi mã nguồn thành các *Instance* vật lý/ảo hóa đang chạy (running processes, RAM, CPU, database).

2. Bản Chất Kỹ Thuật Của Tài Nguyên Chia Được (Sharable) và

Không Chia Được (Non-sharable / Isolated)

1. Tài nguyên "Chia được" (Sharable / Pooled Resource) là những thành phần mà tại tầng đó, một thể hiện vật lý hoặc một tiến trình logic duy nhất có khả năng tiếp nhận, xử lý và phục vụ đồng thời các yêu cầu đến từ nhiều thuê bao khác nhau[2]. Bản chất kỹ thuật của trạng thái "Chia được" là sự đa hợp (multiplexing): vì phần cứng hoặc tiến trình bên dưới không tự nhận biết khái niệm thuê bao của nghiệp vụ, nên để chia sẻ tài nguyên mà không làm trộn lẫn hay rò rỉ dữ liệu, hệ thống bắt buộc phải thiết lập cơ chế kiểm soát bằng phần mềm và logic (software-enforced isolation)[2]. Các cơ chế này bao gồm việc đóng gói định danh vào mã thông báo (Tenant-scoped JWT), duy trì biến ngữ cảnh theo luồng thực thi (ThreadLocal TenantContext), gắn nhãn phân biệt tenant_id trên từng bản ghi, cưỡng chế chính sách bảo mật cấp dòng (Row-Level Security – RLS) trong nhân cơ sở dữ liệu, hoặc phân bổ hạn ngạch bằng thuật toán Token Bucket[2].

Ưu điểm & Rủi ro:

- *Ưu điểm:* Tối ưu hóa triệt để chi phí máy chủ, tận dụng tối đa bộ nhớ (RAM pool, JVM heap) và đơn giản hóa vận hành.
- *Rủi ro:* Dễ xuất hiện hiện tượng **"Hàng xóm ồn áo" (Noisy Neighbor)** khi một tenant tiêu thụ quá tài nguyên làm giảm chất lượng dịch vụ chung, hoặc rủi ro **rò rỉ dữ liệu chéo** nếu lập trình thiếu sót.

Tóm gọn: Tài nguyên chia được là một thể hiện vật lý hoặc tiến trình duy nhất phục vụ đồng thời nhiều thuê bao (multiplexing)
2.Tài nguyên "Không chia được" (Non-sharable / Isolated / Siloed Resource) là những thành phần mà tại tầng đó, mỗi thuê bao bắt buộc phải sở hữu một thể hiện độc lập và biệt lập hoàn toàn, tuyệt đối không cho phép các luồng thực thi hoặc dữ liệu của thuê bao khác đi vào cùng không gian định danh đó[2].
Thay vì phải viết code cẩn thận để các tenant không nhìn thấy dữ liệu của nhau (giống như ở chung một nhà trọ mà dùng chung phòng khách, chỉ nhắc nhau không được lục đồ của người khác), phương pháp **"Không chia được"** giống như việc **xây cho mỗi nhà một căn hộ độc lập có khóa riêng**:

- **Ở tầng Máy chủ ảo (VM):** Dùng phần mềm quản lý máy chủ (Hypervisor) để chia hẳn một máy tính ảo độc lập cho từng tenant. Máy ảo của ai người nấy dùng, sập máy này không ảnh hưởng máy kia.
- **Ở tầng Hệ điều hành (Container):** Mỗi tenant chạy trong một "chiếc hộp" (container) riêng biệt với không gian bộ nhớ và hệ thống file độc lập.
- **Ở tầng Cơ sở dữ liệu (Database):** Thay vì tất cả các tenant cùng chui vào một bảng dữ liệu rồi dùng câu lệnh `Select` để lọc (rất dễ sơ hở lộ dữ liệu), hệ thống sẽ cấp hẳn một file cơ sở dữ liệu riêng hoặc phân quyền tài khoản chặt chẽ ở tầng quản trị database. Tên tenant này tuyệt đối không có quyền sờ vào dữ liệu của tên tenant khác, dù code ứng dụng có viết lỗi đi chăng nữa.

Ưu điểm & Rủi ro:

- *Ưu điểm:* Bảo mật tuyệt đối, cách ly hoàn toàn, không sợ ảnh hưởng hiệu năng từ các tenant khác.

- *Rủi ro:* Tốn kém nhiều tài nguyên phần cứng và phức tạp hơn trong việc quản lý, vận hành hàng loạt instance riêng lẻ. Chi phí vận hành như chạy kịch bản nâng cấp cấu trúc bảng tăng lên theo cấp số nhân $O(N)$ với N là số lượng thuê bao[2].

3. Xây Dựng Ngăn Xếp Kiến Trúc và Phân Tích Khả Năng Chia Sẻ Từng Thành Phần

Mọi hình ảnh liên quan bên dưới nằm trong:
https://research-portal.uu.nl/ws/files/6122833/Kabbedijk.pdf

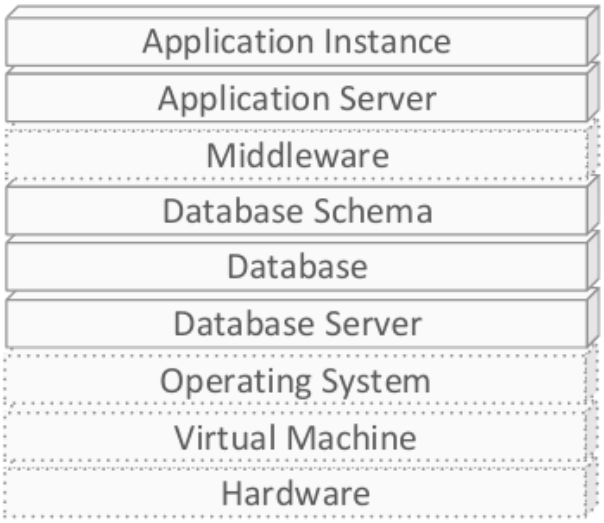


Figure 2: Software Stack: The different system levels where multi-tenancy can be applied to share resources.

Multi-tenancy level	N
Application Instance	16
Database Server	16
Database	15
Operating System	15
Hardware	14
Schema	14
Middleware	12
Virtual Machine	9
Application Server	4

TABLE 6.1: Multi-Tenancy levels identified in literature

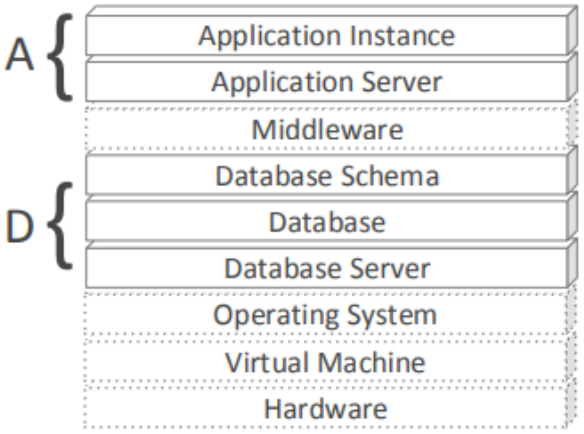


FIGURE 6.2: Multi-tenancy computing stack. *'A' and 'D' relatively indicate the Application and Data related layer sets*

Nét Liền – Nét Đứt

Nhóm 4 tầng "Nét đứt" (Hạ tầng và đường ống kỹ thuật): Gồm *Hardware (Phần cứng), Virtual Machine (Máy ảo), Operating System (Hệ điều hành)* và *Middleware (Phần mềm trung gian)*. Các tầng này được vẽ bằng nét đứt vì trong điện toán đám mây, chúng thuộc về lớp hạ tầng có sẵn do các nhà cung cấp nền tảng đảm nhiệm. Đối với người phát triển phần mềm, chúng chỉ đóng vai trò là "khung nhà" hoặc "đường ống dẫn" cơ bản, cung cấp sự cách ly máy móc chứ không can thiệp hay quyết định thiết kế kiến trúc bên trong của ứng dụng.

Ngược lại, năm khối nét liền đại diện cho hai tập hợp tầng cốt lõi mà người thiết kế kiến trúc phần mềm SaaS trực tiếp kiểm soát và cấu hình: Tập hợp tầng Ứng dụng (A – Application Layer Set) gồm Application Server và Application Instance; cùng với Tập hợp tầng Dữ liệu (D – Data Layer Set) gồm Database Server, Database, và Database Schema[2]. Theo lý thuyết của Kabbedijk và Bezemer, mỗi kiến trúc đa thuê bao (MTA) được định nghĩa toán học bằng một bộ đôi $\mathrm{MTA} = \langle A, D \rangle$ nghĩa là cấu trúc hệ thống là sự kết hợp giữa cách tổ chức tầng ứng dụng và tầng dữ liệu,

trong đó $A \in \{AD, AS, AI\}$

AD **(App Dedicated - Tách riêng):** Mỗi thuê bao (Tenant) có một Application Server riêng biệt, không dùng chung với ai.

AS **(App Shared Server - Dùng chung máy chủ, tách tiến trình):** Các thuê bao dùng chung một cụm máy chủ ứng dụng nhưng chạy các tiến trình (Instance) xử lý khác nhau.

AI **(App Instance Shared - Dùng chung hoàn toàn Instance):** Tất cả các thuê bao dùng chung một thể hiện ứng dụng chạy đồng thời, cùng xử lý yêu cầu cho mọi người.

Và $D \in \{DD, DS, DB, DC\}$

DD **(Data Dedicated Server - Tách riêng hẳn máy chủ CSDL):** Mỗi thuê bao có một máy

chủ cơ sở dữ liệu vật lý hoàn toàn độc lập.
DS **(Shared Database Server):** Chung DB Server, nhưng **mỗi tenant một Database riêng** .
DB (**Shared Database**) Chung một Database, nhưng **mỗi tenant một Schema / bộ bảng riêng**
DC **(Shared Schema):** Chung cả Schema và bảng, phân biệt bằng cột tenant_id
→ Suy ra được rằng kiến trúc multi-tenants (MTA) có 12 cách tiếp cận được biểu diễn thành ma trận 3x4
Một trong những cách tiếp cận bài báo đưa ra:

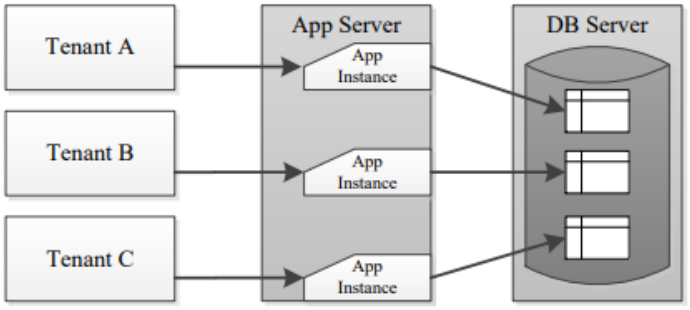


FIGURE 6.3: MTA$\langle AS, DB\rangle$ - Shared Application Server & Shared Database

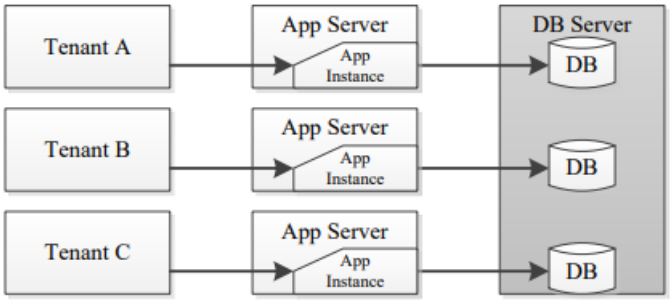


FIGURE 6.4: MTA$\langle AD, DS\rangle$ - Dedicated Application Server & Shared Database Server

MTA Decision Matrix

Decision Criterion	$\langle AD, DD \rangle$	$\langle AS, DD \rangle$	$\langle AI, DD \rangle$	$\langle AD, DS \rangle$	$\langle AS, DS \rangle$	$\langle AI, DS \rangle$	$\langle AD, DB \rangle$	$\langle AS, DB \rangle$	$\langle AI, DB \rangle$	$\langle AD, DC \rangle$	$\langle AS, DC \rangle$	$\langle AI, DC \rangle$	Dist. Factor (σ^2)
Time Behavior	5.0	4.0	4.0	4.0	3.0	3.0	4.0	3.0	3.0	3.5	3.0	2.5	0.5
Resource Utilization	2.0	2.5	3.0	2.5	3.0	3.0	3.0	3.0	4.0	3.0	3.0	4.5	0.4
Throughput	4.5	3.0	3.0	4.0	3.0	3.0	3.5	3.5	3.0	3.0	3.0	3.0	0.2
Number of Tenants	1.0	3.0	3.0	3.0	3.5	4.0	3.0	4.0	4.0	4.0	4.0	5.0	1.0
Number of End-Users	2.5	3.5	3.0	3.0	3.5	3.5	3.5	3.5	4.0	3.5	4.0	4.5	0.3
Availability	4.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.5	3.0	3.0	3.0	0.1
Recoverability	5.0	4.5	4.5	4.0	4.0	4.0	3.0	3.0	3.0	2.0	2.0	2.0	1.1
Confidentiality	5.0	4.5	4.0	4.0	4.0	4.0	3.5	3.0	3.0	2.0	2.0	2.5	0.9
Integrity	4.5	4.0	3.0	4.0	3.5	3.0	3.5	3.0	3.0	3.0	2.5	2.0	0.5
Authenticity	4.5	3.5	3.0	3.5	3.0	3.0	4.0	3.5	3.0	3.0	3.0	2.5	0.3
Maintainability	1.5	2.5	3.0	2.5	3.0	3.5	2.5	4.0	4.5	3.0	4.0	5.0	1.0
Portability	5.0	5.0	4.5	4.5	4.5	4.5	4.0	4.0	4.0	3.0	3.0	2.5	0.7
Deployment Time	1.5	3.0	3.0	2.5	3.5	4.0	3.0	4.0	4.0	3.0	4.0	5.0	0.8
Variability	5.0	4.0	2.5	5.0	4.0	2.0	4.5	3.5	2.0	2.5	2.0	1.0	1.8
Diverse SLA	4.5	4.0	3.0	4.0	3.5	2.5	4.0	3.0	3.0	3.0	2.5	2.0	0.6
Software Complexity	5.0	4.5	4.0	4.5	4.5	3.5	4.0	4.0	3.0	2.5	2.5	2.0	0.9
Monitoring	1.0	2.0	3.0	2.5	3.0	3.0	3.0	4.0	4.0	3.5	4.0	5.0	1.1

TABLE 6.3: Multi-Tenant Architecture Decision Matrix (In color) - $n = 16$

Cấu trúc cốt lõi của ma trận được định nghĩa qua các trục sau:

- **Trục cột (12 Phương án Kiến trúc {A, D}):** Thể hiện các tổ hợp từ cô lập vật lý hoàn toàn ({AD, DD} - siloed) đến chia sẻ logic hoàn toàn ({AI, DC} - pooled), kết hợp giữa tầng Ứng dụng (AD, AS, AI) và tầng Dữ liệu (DD, DS, DB, DC).
- **Trục hàng (Hệ thống Tiêu chí Quyết định - Decision Criteria):** Bao gồm các thuộc tính chất lượng (như hiệu năng, khả năng phục hồi, bảo mật/cô lập, tính bảo trì, khả năng mở rộng, độ phức tạp phần mềm) và các chỉ số tải/chi phí (số lượng tenant, mức tận dụng tài nguyên).
- **Ô giá trị (Điểm số tác động):** Thể hiện mức độ đánh đổi (trade-offs) theo thang đo chuẩn hóa (thường từ 1.0 đến 5.0), giúp chỉ rõ một mô hình là rất bất lợi (điểm thấp) hay là điểm mạnh vượt trội (điểm cao) đối với tiêu chí tương ứng.
- **Hệ số phân tán (σ^2 - Distribution Factor):** Đo lường mức độ biến thiên của điểm số giữa các kiến trúc, giúp kiến trúc sư nhận diện những tiêu chí nào bị ảnh hưởng mạnh nhất bởi quyết định thiết kế (như bảo mật, khả năng phục hồi, giới hạn số lượng tenant).

Ý nghĩa cốt lõi: Ma trận này loại bỏ cảm tính trong thiết kế phần mềm bằng cách biến việc chọn kiến trúc thành một bài toán đánh đổi minh bạch, cung cấp cơ sở lý luận khoa học vững chắc để biện minh cho việc chọn mô hình triển khai phù hợp nhất với ràng buộc tài nguyên và mục tiêu bài toán.

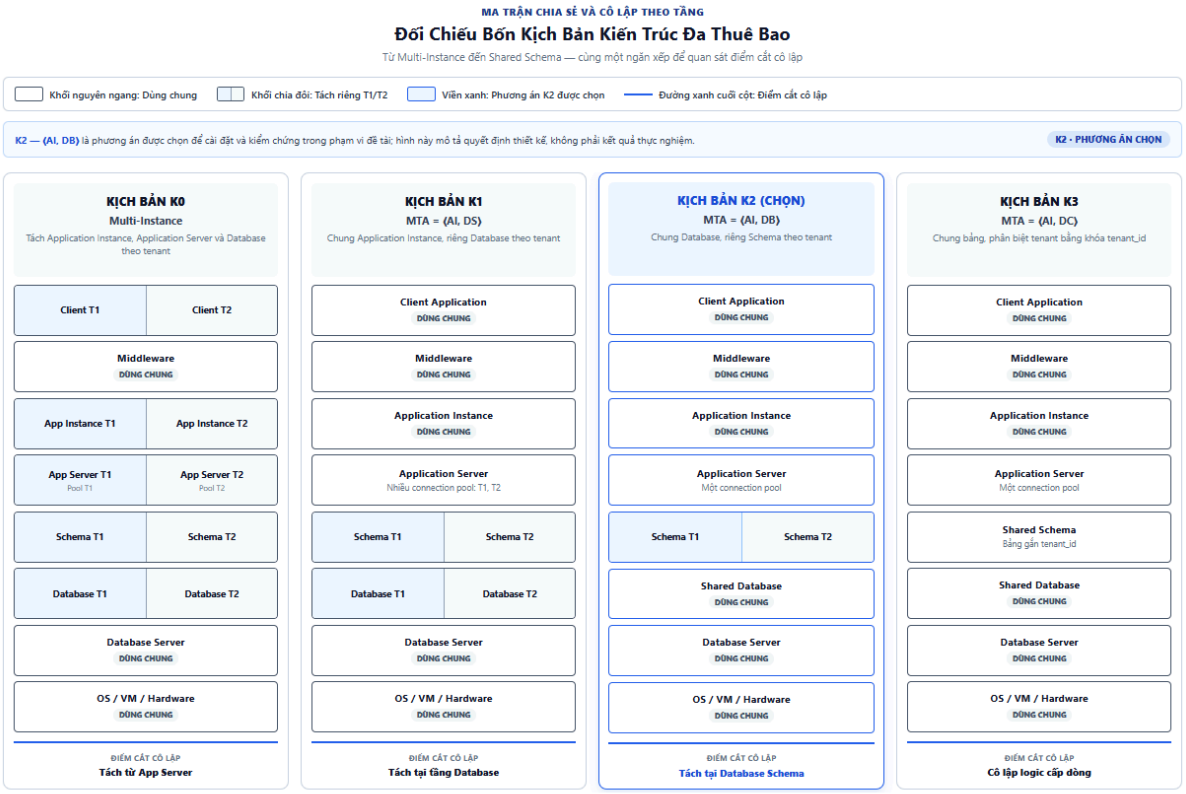
4. Lập luận lựa chọn phương án kiến trúc đề xuất (đối chiếu Table 6.3 – MTA Decision Matrix)

Phương án chọn: K2 – ⟨AI, DB⟩ (*Shared Application Instance, Shared Database – Separate Schema*). Đây là giả thuyết nghiên cứu có điều kiện, nhóm sẽ kiểm chứng bằng đo đạc thực nghiệm.

1. Cơ sở sàng lọc

Table 6.3 (Kabbedijk et al., n = 16 chuyên gia, thang 1.0 – rất bất lợi đến 5.0 – rất tích cực) đánh giá 12 tổ hợp ⟨A, D⟩ trên 17 tiêu chí. Nhóm giữ lại các tổ hợp dùng chung thể hiện ứng dụng (AI), vì đó là bản chất của đa thuê bao ở tầng nghiệp vụ. Còn lại ba ứng viên:

Ký hiệu	Tầng dữ liệu
K1 – ⟨AI, DS⟩	Chung app, chung DB server, **riêng database**
K2 – ⟨AI, DB⟩ (Được chọn)	Chung app, chung database, **riêng schema**
K3 – ⟨AI, DC⟩	Chung app, chung database, **chung schema**



Hình X. Đối chiếu mức chia sẻ và điểm cắt cô lập của bốn kịch bản K0–K3 trên cùng ngăn xếp ứng dụng–dữ liệu–hạ tầng. Khối chia đôi biểu diễn tài nguyên riêng cho Tenant 1 và Tenant 2; khối nguyên ngang biểu diễn tài nguyên dùng chung.

2. Số liệu đối chiếu (xếp theo hệ số phân tán σ² giảm dần)

Tiêu chí có σ² cao là nơi các chuyên gia bất đồng nhiều nhất, nên cũng là nơi phương án thật sự khác nhau.

Tiêu chí	σ²	K2 (Selected)	K1	K3
Variability	1.8	**2.0**	2.0	1.0
Recoverability	1.1	**3.0**	4.0	2.0
Monitoring	1.1	**4.0**	3.0	5.0
Number of Tenants	1.0	**4.0**	4.0	5.0
Maintainability	1.0	**4.5**	3.5	5.0
Confidentiality	0.9	**3.0**	4.0	2.5
Software Complexity	0.9	**3.0**	3.5	2.0
Deployment Time	0.8	**4.0**	4.0	5.0
Portability	0.7	**4.0**	4.5	2.5
Diverse SLA	0.6	**3.0**	2.5	2.0

Time Behavior	0.5	**3.0**	3.0	2.5
Integrity	0.5	**3.0**	3.0	2.0
Resource Utilization	0.4	**4.0**	3.0	4.5
Điểm trung bình (17 tiêu chí)		**≈ 3.4**	**≈ 3.3**	**≈ 3.2**

Điểm trung bình chỉ mang tính tham khảo vì chênh lệch nhỏ và các tiêu chí không có trọng số. Nó cho thấy K2 không bị bỏ xa ở phương án nào, chứ không chứng minh K2 "thắng tuyệt đối".

3. Vì sao K2 thay vì K3

- Phù hợp với quy mô thực nghiệm của thuyết minh: Thuyết minh đề tài đặt trọng tâm vào việc khảo sát ranh giới cô lập dữ liệu và định danh thuê bao với quy mô dự kiến **3–5 workspace thử nghiệm** (mỗi workspace 10–20 người dùng) trên một máy chủ VPS giới hạn. Ở quy mô này, những lợi thế điểm 5.0 của K3 về vận hành hàng trăm tenant (*Number of Tenants*, *Deployment Time*, *Monitoring*) chưa tạo ra khác biệt thực tế vì mức điểm **4.0 – 4.5** của K2 đã hoàn toàn dư sức đáp ứng mà chưa chạm tới điểm nghẽn Catalog Bloat hay Migration O(N).
- Thắng thế ở các tiêu chí cốt lõi về dữ liệu: K2 vượt qua K3 ở toàn bộ các tiêu chí quyết định chất lượng quản trị dữ liệu đơn vị: *Recoverability* (**3.0** so với **2.0**), *Portability* (**4.0** so với **2.5**), *Confidentiality* (**3.0** so với **2.5**), *Integrity* (**3.0** so với **2.0**), *Variability* (**2.0** so với **1.0**) và *Software Complexity* (**3.0** so với **2.0**). Trong môi trường đại học, mỗi Khoa hoặc Phòng thí nghiệm cần một ranh giới dữ liệu tường minh để có thể sao lưu, khôi phục hoặc tùy biến độc lập.

4. Vì sao K2 thay vì K1 (thừa nhận đánh đổi)

- Nhóm hoàn toàn thừa nhận K1 có điểm *Recoverability* (4.0) và *Confidentiality* (4.0) cao hơn K2 (3.0). Tuy nhiên, trên một VPS có tài nguyên RAM và CPU giới hạn, K2 vượt K1 ở toàn bộ nhóm tiêu chí sống còn về hiệu quả khai thác hạ tầng: *Resource Utilization* (**4.0** so với **3.0**), *Maintainability* (**4.5** so với **3.5**), *Monitoring* (**4.0** so với **3.0**), *Throughput* (**3.5** so với **3.0**) và *Number of End-Users* (**4.0** so với **3.0**).
- Về mặt kỹ thuật, K2 gom toàn bộ các workspace về **1 Connection Pool chung** và chia sẻ chung bộ nhớ đệm của một PostgreSQL Database duy nhất, trong khi K1 buộc phải nhân bản Connection Pool và phân mảnh bộ đệm cho từng Database riêng lẻ.

5. Giá trị thực nghiệm và giới hạn chủ động thừa nhận

- **Giá trị:** K2 có ranh giới schema rõ ràng để kiểm thử truy cập chéo giữa các workspace. Các tenant vẫn dùng chung tiến trình ứng dụng và PostgreSQL, nên vẫn có điều kiện thực tế để đo hiện tượng *Noisy Neighbor* (Time Behavior chỉ 3.0).

- **Giới hạn:**
    - Cấp phát và migration theo từng schema phức tạp hơn K3.
    - Schema riêng không tự chặn truy cập chéo nếu cấp quyền sai.
    - Các tenant vẫn tranh chấp cùng DB server.
    - Điểm trong Table 6.3 là đánh giá chuyên gia (n = 16), không phải số đo trên hệ thống của nhóm.

6. Tuyên bố định hướng

*"Mục đích của đề tài không chỉ là xây dựng một ứng dụng Kanban có nhiều người dùng, mà là **đề xuất và kiểm chứng một kiến trúc đa thuê bao phù hợp cho bài toán quản lý công việc**. Kiến trúc đó phải làm rõ, từ tầng giao diện, nghiệp vụ, dữ liệu đến hạ tầng, thành phần nào được nhiều tenant dùng chung, thành phần nào được tách riêng, cơ chế nào duy trì đúng ranh giới tenant, và việc chia sẻ tài nguyên tạo ra lợi ích cũng như đánh đổi gì.*

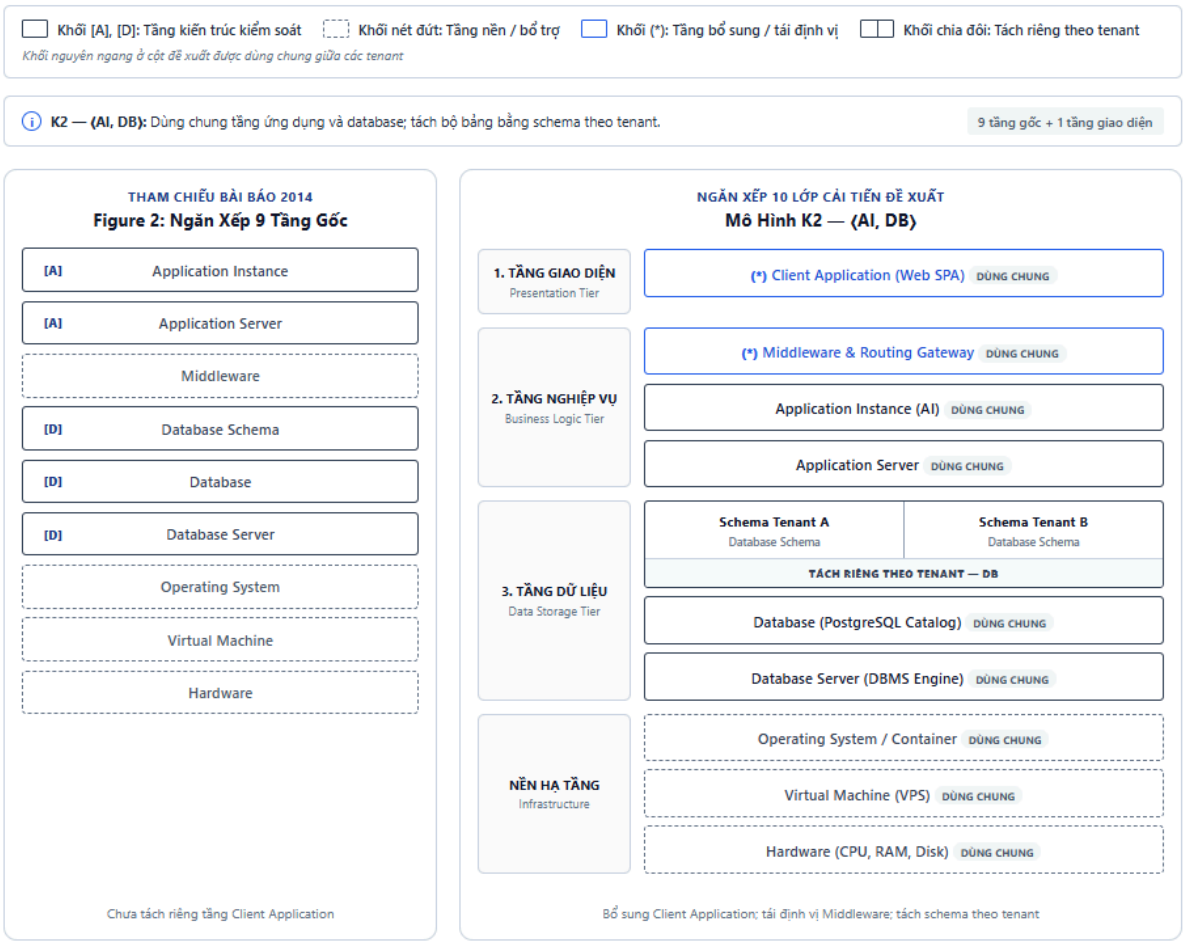
*Để đạt mục đích này, nhóm so sánh ba phương án cùng chia sẻ một phiên bản ứng dụng nhưng có ranh giới dữ liệu khác nhau: database riêng trên DB server chung (K1 — ⟨AI, DS⟩), bộ bảng riêng trong database chung (K2 — ⟨AI, DB⟩), và bộ bảng dùng chung (K3 — ⟨AI, DC⟩). Ma trận MTA của Kabbedijk và cộng sự được sử dụng như **cơ sở tham khảo về xu hướng đánh đổi theo nhận định của 10 chuyên gia**, không phải kết quả đo trên ứng dụng của nhóm và không đủ để khái quát cho mọi hệ thống. Trong phạm vi đánh giá đó, chia sẻ sâu hơn được nhận định thuận lợi hơn cho khai thác tài nguyên, triển khai và quản trị nhiều tenant; ranh giới dữ liệu tách hơn được nhận định thuận lợi hơn cho bảo mật, phục hồi và khả năng biến đổi.*

*Dưới các ràng buộc của đề tài — 3–5 workspace thử nghiệm, 10–20 người dùng đồng thời cho mỗi workspace và một VPS giới hạn — nhóm **đề xuất có điều kiện K2 — ⟨AI, DB⟩** làm kiến trúc mục tiêu: dùng chung mã nguồn, phiên bản ứng dụng, tài nguyên tính toán, DB server và PostgreSQL database; mỗi workspace được ánh xạ vào một schema/bộ bảng riêng. So với K3, K2 tạo một ranh giới dữ liệu riêng mà nhóm "dự kiến" sẽ thuận lợi hơn cho việc quan sát, kiểm thử và quản lý vòng đời tenant, trong khi vẫn duy trì mức chia sẻ tài nguyên cao hơn K1. So với K1, K2 giảm số database phải cấp phát và quản trị, nhưng chấp nhận ranh giới cô lập và khả năng phục hồi yếu hơn. Các nhận định này là giả thuyết cần đo, không phải kết quả đã có. Vì vậy, K2 không được tuyên bố là kiến trúc tốt nhất nói chung; nó là **giả thuyết kiến trúc phù hợp nhất với mục tiêu và phạm vi hiện tại để nhóm đưa vào cài đặt và kiểm chứng**."*

Phần Quan Trọng nhất tìm thấy trong bài báo:

Nguyên lý bất biến của ngăn xếp: Theo Kabbedijk et al. (Chương 6, tr. 93–94), khi tính đa thuê bao (chia sẻ) được áp dụng tại một tầng nào đó, toàn bộ các tầng nằm bên dưới nó **trong cùng cụm (A hoặc D)** bắt buộc phải được dùng chung, trong khi các tầng nằm bên trên nó trong cụm đó sẽ được tách riêng cho từng thuê bao. Từ sự kết hợp giữa A (3 mức) và D (4 mức), kiến trúc đa thuê bao hình thành **12 tổ hợp MTA = (A, D)**.

Dựa vào nguyên lý bất biến đó ta có thể thiết kế một ngăn xếp khi chọn *(K2 — ⟨AI, DB⟩)* làm một mô hình đề xuất:



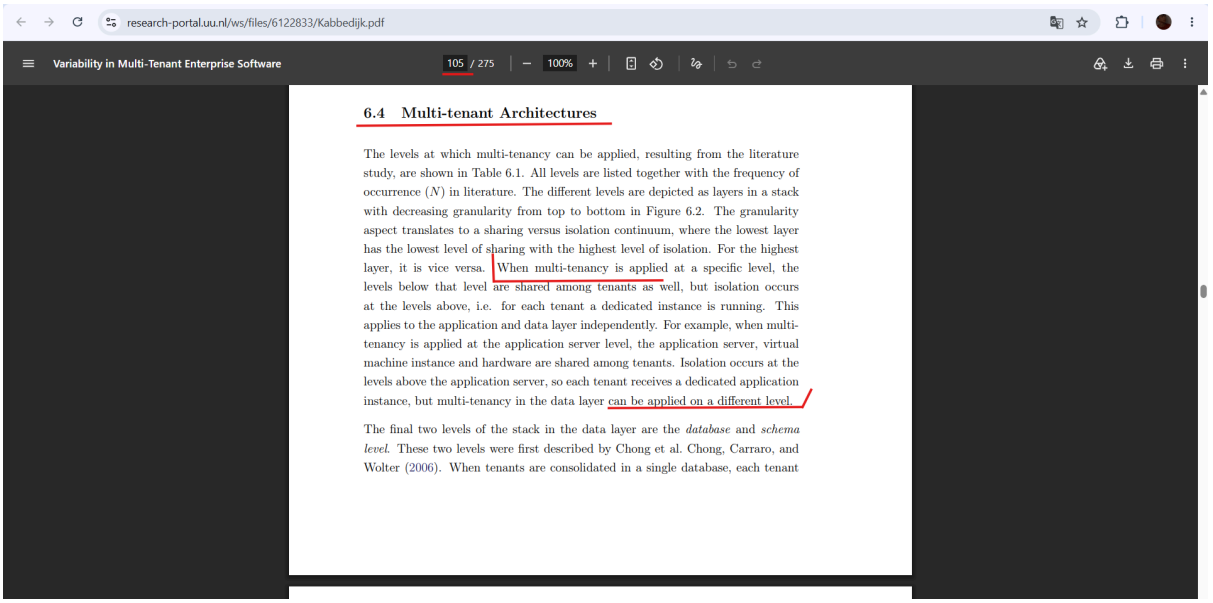
CỤM ỨNG DỤNG — A = AI

Application Instance CHUNG ← điểm bắt đầu chia sẻ A thì ở dưới “Application Server” theo nguyên tắc nó phải là dùng chung
Application Server CHUNG

CỤM DỮ LIỆU — D = DB

Schema tenant A	RIÊNG
Schema tenant B	RIÊNG
Database	CHUNG ← điểm bắt đầu chia sẻ D theo nguyên lý thì “Database Server” phải dùng chung
Database Server	CHUNG

Phần nguyên lý được nhắc trong bài nghiên cứu:



6.4 Multi-tenant Architectures

The levels at which multi-tenancy can be applied, resulting from the literature study, are shown in Table 6.1. All levels are listed together with the frequency of occurrence (N) in literature. The different levels are depicted as layers in a stack with decreasing granularity from top to bottom in Figure 6.2. The granularity aspect translates to a sharing versus isolation continuum, where the lowest layer has the lowest level of sharing with the highest level of isolation. For the highest layer, it is vice versa. When multi-tenancy is applied at a specific level, the levels below that level are shared among tenants as well, but isolation occurs at the levels above, i.e. for each tenant a dedicated instance is running. This applies to the application and data layer independently. For example, when multi-tenancy is applied at the application server level, the application server, virtual machine instance and hardware are shared among tenants. Isolation occurs at the levels above the application server, so each tenant receives a dedicated application instance, but multi-tenancy in the data layer can be applied on a different level.

The final two levels of the stack in the data layer are the *database* and *schema level*. These two levels were first described by Chong et al. Chong, Carraro, and Wolter (2006). When tenants are consolidated in a single database, each tenant

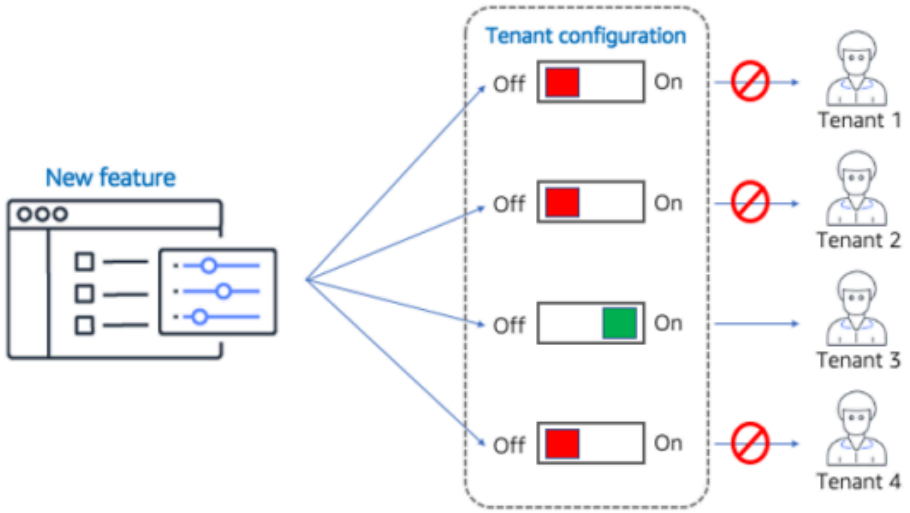


Figure 14: Managing tenant needs with feature flags

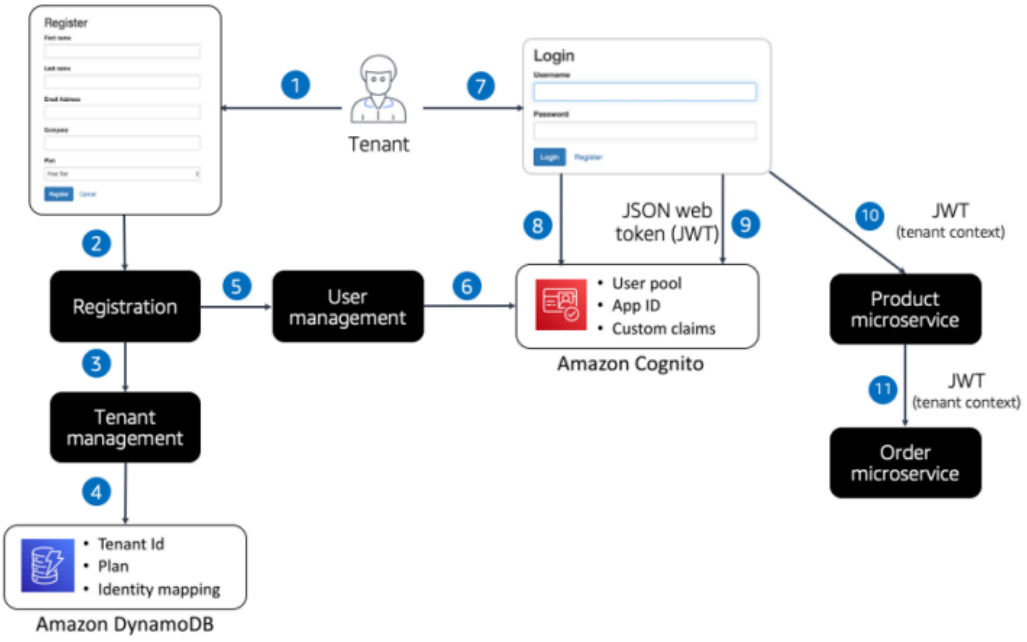


Figure 15: Injecting tenant content

Sửa lại quy trình:
- **Định nghĩa khái niệm nền tảng rõ ràng:** *Service*, *Tenant*, *Instance*, *Deployment*, *Sharable vs. Non-sharable*. (Dẫn chứng từ nguồn ra trước tổng hợp các định nghĩa từ nguồn chính thống khác nhau) -> lập khái niệm tổng hợp
- Tạo/ vẽ một mô hình MTA hoàn chỉnh: nó bao gồm các Tenant, Service, Instance. Một số thứ khác nữa nhưng tui chỉ biết một MTA không chỉ dừng lại ở 3 khái niệm trên. Mô hình MTA bạn tạo không đào sâu vào kĩ thuật dì hết chỉ là những thành phần cần có cho MTA thôi, những thành phần đó như Tenant Service Instance vẽ thành những thành phần khối

vuông hay dì đó cũng được nhưng nó phải có sự liên kết chặt chẽ với nhau. Đảm bảo đầu ra của mô hình là dòm vào không bị nhầm lẫn với các mô hình khác như Multi-user, multi-instance, 2 mô hình khác ấy vẽ ra luôn phân tách ra và chỉ rõ khác những gì so với MTA đã vẽ. (Khuyến nghị tham khảo các nguồn tinh cậy để lập mô hình)

- Từ cái MTA tổng thể được vẽ ở trên Đưa ra Ngăn xếp kiến trúc hiện đại của nhóm (cải tiến từ Figure 2): Chỉ ra các thành phần trong một hệ thống Multi-Tenant hoàn chỉnh hiện nay gồm những tầng nào.Rút gọn thuần tiếng Anh học thuật. Mỗi tầng bạn vẽ ra ấy nó phải cho biết là các thành phần trong mỗi tầng đó gồm những thứ gì. (Khuyến nghị tham khảo các nguồn tinh cậy trước khi lập mô hình)

- Dựng các kịch bản kiến trúc (Scenarios): Dựa trên ngăn xếp mới đó, nếu chia sẻ ở tầng này / cô lập ở tầng kia thì sinh ra những mô hình (kịch bản) nào?

- Phân tích đánh đổi (Trade-offs) của từng kịch bản: Khi chia sẻ hay tách riêng ở mỗi kịch bản thì đạt được đặc trưng gì của Multi-Tenant, ưu/nhược điểm ra sao (kết hợp tham khảo số liệu Table 6.3 và kiến thức thực tế hiện nay).

- Chốt lựa chọn kiến trúc cho đề tài: Dựa trên quy mô và ràng buộc thực tế của nhóm (số lượng tenant, hạ tầng VPS...), giải thích vì sao nhóm chọn kịch bản đó để cài đặt.

- Cài đặt & Thảo luận kết quả: Trình bày cách cài đặt kịch bản đã chọn, đo đạc thực nghiệm, thảo luận xem kết quả đạt được giải quyết các đặc trưng Multi-Tenant như thế nào và rút ra hướng phát triển.